

UNIVERSIDADE ESTADUAL DE PONTA GROSSA – UEPG
SETOR DE ENGENHARIAS, CIÊNCIAS AGRÁRIAS E TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

JOÃO PEDRO BORSATO DE RAMOS
JOÃO VITOR CARVALHO DE ANDRADE

OASIS:
PROJETO DE SOFTWARE DE UMA APLICAÇÃO WEB PARA GESTÃO DE
RESERVAS
E CONTROLE DE ACESSO EM CONDOMÍNIOS RESIDENCIAIS

PONTA GROSSA
2026

JOÃO PEDRO BORSATO DE RAMOS
JOÃO VITOR CARVALHO DE ANDRADE

OASIS:
PROJETO DE SOFTWARE DE UMA APLICAÇÃO WEB PARA GESTÃO DE
RESERVAS
E CONTROLE DE ACESSO EM CONDOMÍNIOS RESIDENCIAIS

Projeto de Software apresentado ao curso de Engenharia de Computação da Universidade Estadual de Ponta Grossa como requisito parcial na disciplina de Projeto de Software.

Professores: Diolete Marcante Lati Cerutti, Ezequiel Gueiber e Idomar Augusto Cerutti.

PONTA GROSSA

2026

SUMÁRIO

1 DIAGNÓSTICO ATUAL.....	4
2 OBJETIVOS DO PROJETO	5
2.1 OBJETIVO GERAL	5
2.2 OBJETIVOS ESPECÍFICOS	5
3 ESCOPO	6
4 PROPOSTA DE PROJETO.....	8
4.1 BENEFÍCIOS	10
4.2 CUSTOS	11
4.3 RISCOS	12
4.4 MODELAGEM DO SISTEMA	13
4.4.1 Diagrama de casos de uso	13
4.4.2 Diagrama de classes	23
4.4.3 Diagramas de sequência	28
4.5 MODELAGEM DO BANCO DE DADOS.....	33
4.5.1 Diagrama entidade-relacionamento	34
4.5.2 Diagrama lógico	36
4.5.3 Domínios e tipos enumerados	39
4.5.4 Dicionário de dados	40
4.5.5 Scripts do banco de dados.....	49
4.5.6 Regras de negócio e gatilhos.....	54
4.6 INTERFACE DO SISTEMA	59
4.6.1 Mapa de navegação	60
4.6.2 Telas comuns aos perfis	61
4.6.3 Telas do perfil Morador	63
4.6.4 Telas do perfil Porteiro.....	65
4.6.5 Telas do perfil Síndico e manutenção de cadastros	67
4.7 PLANO DE TESTES	69
4.7.1 Objetivo	69
4.7.2 Escopo.....	70
4.7.3 Estratégia e critérios de aceitação	70
4.7.4 Massa de dados	70
4.7.5 Casos de teste	71
5 CRONOGRAMA.....	73
6 CONCLUSÃO.....	74
6.1 TRABALHOS FUTUROS	75
7 REFERÊNCIAS	76
8 RESPONSABILIDADES.....	76

1 DIAGNÓSTICO ATUAL

O cliente deste projeto é um condomínio residencial localizado em Ponta Grossa, representado pela síndica responsável pela administração. Atualmente, o condomínio utiliza o aplicativo comercial BRCONDOS para a gestão de reservas (agendamentos) de espaços comuns. Embora a ferramenta disponibilize um conjunto de funcionalidades, a síndica relatou que o aplicativo não contempla o acesso biométrico associado à reserva do espaço (academia, piscina), nem o acesso por reconhecimento facial à garagem e à porta principal de entrada do condomínio.

Outro problema relatado é que o sistema atual restringe cada apartamento a somente dois logins de acesso, mesmo quando há mais de dois moradores na unidade. Um terceiro morador não pode ter *login* para fazer a reserva em seu nome. Desse modo, caso um morador de determinado apartamento efetue uma reserva da academia no período matutino, os demais moradores da mesma unidade ficam impedidos de realizar uma reserva complementar para outro horário do mesmo dia, mesmo que os espaços estejam disponíveis. Essa limitação gera conflitos internos e insatisfação entre os condôminos.

Quanto ao sistema de comunicação e notificações, a transmissão de avisos da administração aos moradores ocorre atualmente de forma informal, por meio de aplicativos de mensagens (*WhatsApp*) e cartazes físicos afixados em áreas comuns. Não há um canal centralizado, oficial e rastreável para a comunicação condominial, tampouco mecanismos automatizados de notificação para a chegada de encomendas, o que dificulta a gestão de informações e o acompanhamento pelos moradores.

Por fim, o controle de empréstimo de chaves de áreas comuns e o registro de recebimento de encomendas na portaria são feitos em cadernos de papel, sem qualquer rastreabilidade sobre quem retirou determinado item, em que data e em que horário, o que impede a apuração de responsabilidades em caso de extravio. A portaria também não dispõe de meios para saber quais áreas estão reservadas ou em uso em determinado momento, nem quem é o morador responsável por cada uso.

2 OBJETIVOS DO PROJETO

2.1 OBJETIVO GERAL

Desenvolver uma aplicação *web* denominada OASIS destinada à gestão integrada de reservas de espaços coletivos, ao controle de acesso às áreas comuns e à rotina da portaria em condomínios residenciais, promovendo maior praticidade para os moradores e maior organização, transparência e monitoramento para a administração condominial.

2.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos do projeto são:

- Eliminar a restrição de logins por apartamento, possibilitando o controle individualizado de reservas por morador, inclusive dos dependentes vinculados à unidade;
- Oferecer autenticação própria e segura, com ativação de conta por código de primeiro acesso, recuperação de senha e armazenamento das senhas apenas na forma de resumo criptográfico (*hash*);
- Centralizar e formalizar a comunicação entre a administração condominial e os moradores, substituindo o uso de canais informais por um mural oficial e rastreável, com registro de leitura;
- Reduzir conflitos no uso dos espaços coletivos por meio de regras configuráveis por área: antecedência, capacidade, idade mínima, limite de reservas por período e quantidade de reservas simultâneas;
- Controlar logicamente o acesso às áreas comuns por meio de reservas, bloqueios de área e penalidades aplicadas a moradores, deixando o sistema preparado para uma futura integração com dispositivos físicos de controle de acesso;
- Rastrear o empréstimo e a devolução de chaves e a movimentação de encomendas, registrando o morador, o funcionário responsável e os horários de cada operação;
- Fornecer à portaria uma visão em tempo real da ocupação das áreas comuns e da agenda de reservas do dia;

- Fornecer à administração condominial indicadores consolidados sobre o uso dos espaços e a movimentação da portaria, apoiando decisões de gestão;
- Automatizar a notificação dos moradores no mural sobre a chegada de encomendas e sobre os comunicados da administração.

3 ESCOPO

O OASIS abrangerá as seguintes funcionalidades dentro de seu escopo de desenvolvimento:

- Autenticação de três perfis distintos de usuários — morador, administração (síndica) e porteiro —, com suporte a pessoas que acumulam mais de um perfil, limitação de tentativas de acesso, ativação de conta por código de primeiro acesso e recuperação de senha por código;
- Manutenção (cadastro, consulta, edição, inativação e exclusão) de blocos, unidades, vagas de garagem, pessoas e perfis, com proteção contra a remoção do único administrador do sistema;
- Cadastro de visitantes e prestadores de serviço, sem acesso ao sistema, com vínculo opcional a uma unidade e descrição do serviço prestado, permitindo que a portaria os localize e registre o empréstimo de chaves a eles;
- Gestão de dependentes pelo morador titular, com aprovação do cadastro pela administração antes da liberação do acesso;
- Gerenciamento de espaços coletivos, incluindo cadastro, edição, imagem ilustrativa com compressão automática, horários de funcionamento por dia da semana, utensílios disponíveis, chaves físicas e exclusão condicionada à inexistência de histórico;
- Configuração, por área comum, das regras de reserva: antecedência mínima e máxima, duração do intervalo reservável, reserva por dia inteiro, prazo de cancelamento, idade mínima, limite de reservas por unidade em período diário, semanal ou mensal, e quantidade de unidades que podem reservar o mesmo horário;
- Reserva de espaços coletivos por morador (não por apartamento), com visualização de disponibilidade em calendário, controle de conflitos de horário e validação automática das regras configuradas;

- Cancelamento de reserva dentro do prazo configurável definido pela administração, com conclusão automática das reservas cujo horário já se encerrou;
- Agendamento de horários para mudanças por meio da reserva do elevador de serviço;
- Mural de avisos: publicação e remoção de comunicados pela administração, com visualização pelos moradores, registro individual de leitura e separação entre comunicados da administração e avisos de encomenda;
- Controle de recebimento e entrega de encomendas pela portaria, com registro de data de recebimento, data de retirada, identificação de quem retirou e aviso automático ao morador destinatário, que também pode consultar as próprias encomendas;
- Controle de empréstimo e devolução de chaves das áreas comuns, com rastreamento do solicitante e do funcionário responsável, e consulta, pelo morador, das chaves em posse de sua família;
- Controle do estado de ocupação das áreas de uso livre, que dispensam reserva, pela portaria;
- Painel da portaria com a ocupação das áreas em tempo real, a agenda de reservas do dia e a busca de moradores com sua atividade atual e seu histórico recente;
- Bloqueio de áreas comuns para manutenção e aplicação de penalidades a moradores, com cancelamento automático das reservas afetadas;
- Painel administrativo com indicadores, gráficos e filtros sobre o uso dos espaços, encomendas e chaves.

Estão fora do escopo do presente projeto:

- Aplicativo *mobile* nativo (Android ou iOS) — a aplicação é exclusivamente *web*, com *layout* responsivo para acesso via navegadores em dispositivos móveis;
- Integração com sistemas de cobrança ou pagamento de taxas condominiais, e gestão financeira do condomínio;
- Integração com a plataforma Intelbras para liberação física de acesso por biometria e reconhecimento facial;
- Envio de notificações *push* aos dispositivos dos moradores;
- Envio do código de recuperação de senha por e-mail;

- Abertura e acompanhamento de chamados pelos moradores;
- Assistente automatizado (*bot*) de atendimento via *WhatsApp*;
- Relatório de uso da própria aplicação (acessos, telas mais utilizadas e horários de pico);
- Módulo de votação em assembleia e chat em tempo real entre moradores;
- Integração com o sistema BRCONDOS.

As funcionalidades fora do escopo não serão atendidas neste projeto em razão das restrições de tempo e de recursos da equipe de desenvolvimento. A integração com a plataforma Intelbras, as notificações *push*, o envio de e-mail, os chamados, o assistente via *WhatsApp* e o relatório de uso da aplicação constituem a próxima etapa de evolução do sistema e são detalhados como trabalhos futuros na seção 6.1.

4 PROPOSTA DE PROJETO

O OASIS é desenvolvido como uma aplicação *web* responsiva, acessível através de navegadores em computadores e dispositivos móveis. A escolha pela plataforma *web* exclusiva visa reduzir os custos do projeto e simplificar o processo de manutenção e atualização do sistema, sem prejuízo da experiência do usuário em diferentes dispositivos. A seguir são apresentadas as tecnologias, bibliotecas e ferramentas utilizadas no desenvolvimento, com suas respectivas versões.

Em relação à versão anterior deste documento, o conjunto de tecnologias foi revisto para refletir as decisões tomadas durante a implementação. A autenticação, antes delegada ao serviço *Firebase Authentication*, passou a ser realizada pela própria aplicação, o que eliminou a dependência de um serviço externo para a operação básica do sistema e permitiu implementar o fluxo de ativação de conta por código. O acesso ao banco de dados, antes previsto por meio do mapeador objeto-relacional *Prisma*, passou a ser feito com SQL direto por meio do *driver node-postgres*, uma vez que as regras de negócio críticas do sistema residem em gatilhos no próprio banco e a aplicação precisa apenas repassar comandos e interpretar suas respostas.

Frontend (interface web):

- *React 18.3* — biblioteca *JavaScript* para construção da interface de usuário;

- *TypeScript* 5.5 — linguagem tipada baseada em *JavaScript*, para maior segurança no desenvolvimento;
- *Vite* 5.4 — ferramenta de build e empacotamento (*bundler*) do projeto *React*;
- *Tailwind CSS* 3.4 — *framework* de estilização baseado em classes utilitárias, com suporte a tema escuro;
- *React Router* 6 — roteamento das telas e proteção de rotas por perfil de usuário.

Backend (API e lógica de negócio):

- *Node.js* 20 (LTS) — ambiente de execução *JavaScript* para o servidor;
- *NestJS* 10.4 — *framework* para construção de aplicações *backend* modulares em *Node.js*;
- *node-postgres* (pg) 8.12 — *driver* de acesso ao *PostgreSQL*, com consultas parametrizadas e controle de transações;
- *JSON Web Token* (@nestjs/jwt 10.2) — emissão e validação do *token* de sessão, com validade de oito horas;
- *multer* 2 e *sharp* 0.33 — recebimento de imagens das áreas comuns e sua compressão e redimensionamento automáticos no momento do envio.

Banco de dados:

- *PostgreSQL* 16 — banco de dados relacional, com as regras de negócio críticas implementadas por restrições declarativas e gatilhos em *PL/pgSQL*.

Segurança:

- Armazenamento de senhas exclusivamente como resumo criptográfico (*hash*) gerado pelo algoritmo PBKDF2 com SHA-512, 100.000 iterações e valor aleatório (*salt*) individual por senha;
- Limitação de requisições (*rate limiting*) com @nestjs/throttler 6: limite geral por endereço IP e limite específico nas rotas de autenticação, contado por IP e e-mail, para dificultar ataques de tentativa e erro sem bloquear outros moradores da mesma rede;
- *helmet* 8 — definição de cabeçalhos HTTP de segurança nas respostas da API;
- Controle de acesso por perfil em todas as rotas da API, por meio de guardas (*guards*) do *NestJS*;

- Modo de demonstração controlado por variável de ambiente, que habilita os atalhos de acesso usados na apresentação acadêmica e é desativado no ambiente de produção.

Infraestrutura:

- *Docker* e *Docker Compose* — empacotamento do sistema em três contêineres (banco de dados, *backend* e *frontend*), o que permite reproduzir o ambiente completo com um único comando;
- *nginx* — servidor *web* que entrega a interface compilada e encaminha as requisições da API ao *backend*;
- Hospedagem dos contêineres em serviço com plano gratuito compatível com o porte do projeto, como Railway ou Render;
- *GitHub* — versionamento de código, com um *branch* por alteração e revisão por meio de *pull requests*.

Ferramentas auxiliares:

- *Visual Studio Code* e *Antigravity* — ambientes de desenvolvimento;
- *Claude Code* — assistente de programação utilizado no terminal, sob revisão dos autores;
- *Figma* — prototipação inicial da interface;
- *Adobe Illustrator* — criação do logotipo e das ilustrações que compõem a identidade visual do aplicativo;
- *Postman* — testes manuais das rotas da API;
- BRModelo Web, *PlantUML* e *Graphviz* — elaboração dos diagramas UML, do DER e do diagrama lógico.

4.1 BENEFÍCIOS

A implantação do OASIS proporcionará os seguintes benefícios ao condomínio:

- Eliminação das restrições de reserva por apartamento, permitindo que todos os moradores de uma unidade, inclusive os dependentes, agendem os espaços coletivos de forma independente;
- Possibilidade de várias reservas no mesmo dia e, quando a área comportar, no mesmo horário, uma vez que a reserva passa a ser controlada por intervalo de data e hora e pela capacidade acumulada do espaço;

- Centralização e formalização da comunicação condominial em um canal oficial, organizado, com histórico acessível e confirmação de leitura por morador;
- Redução do tempo despendido pela administração no gerenciamento manual de reservas e na resolução de conflitos de agendamento;
- Rastreabilidade completa do empréstimo de chaves e da movimentação de encomendas, com identificação do funcionário responsável por cada operação;
- Visão em tempo real, para a portaria, de quais áreas estão ocupadas, por quem e até que horas;
- Maior transparência na gestão das reservas, permitindo que os moradores acompanhem o status de suas solicitações sem necessidade de contato direto com a administração;
- Interface simplificada, com tema claro e escuro, e disponibilidade em computador, tablet ou smartphone.

4.2 CUSTOS

Os custos do projeto foram estruturados em dois grupos: esforço intelectual (mão de obra) e infraestrutura e serviços de terceiros. Embora o projeto seja desenvolvido em contexto acadêmico, sem remuneração direta aos desenvolvedores, os valores foram calculados com base em referências de mercado a fim de representar o custo real equivalente do desenvolvimento.

A estimativa de esforço foi derivada do calendário letivo do projeto, apresentado no cronograma da seção 5: 40 semanas letivas distribuídas ao longo dos quatro bimestres de 2026 (de 02 de março a 18 de dezembro, descontado o recesso de julho). Considerou-se a dedicação de dois desenvolvedores, com carga individual de 10 horas semanais, compatível com a carga horária da disciplina e com as demais atividades acadêmicas do semestre. A memória de cálculo é apresentada na Tabela 1, obtida pela expressão: $2 \text{ desenvolvedores} \times 10 \text{ horas/semana} \times 40 \text{ semanas} = 800 \text{ horas}$.

Tabela 1 – Memória de cálculo do esforço estimado

Fase do projeto	Semanas	Desenv.	h/semana por desenv.	Total de horas
Levantamento de requisitos e anteprojeto (S01–S10)	10	2	10	200
Modelagem e projeto de software (S11–S20)	10	2	10	200
Implementação do núcleo do sistema (S21–S30)	10	2	10	200
Consolidação, testes e implantação (S31–S40)	10	2	10	200
Total	40	2	10	800

Fonte: Os autores.

O valor de referência por hora foi obtido na plataforma Glassdoor (2026), que aponta a média salarial de um desenvolvedor pleno no Brasil em R\$ 40,00 por hora. Os valores de infraestrutura referem-se a serviços e ferramentas com planos gratuitos compatíveis com o porte do projeto acadêmico, conforme a Tabela 2. Com a substituição do *Firebase* pela autenticação própria, o sistema deixou de depender de serviços pagos por volume de uso.

Tabela 2 – Custos estimados do projeto

Categoria	Item / descrição	Valor (R\$)
Esforço intelectual	Mão de obra — 800 horas a R\$ 40,00/h	32.000,00
	Subtotal — esforço intelectual	32.000,00
Infraestrutura	Hospedagem dos contêineres — Railway ou Render (plano gratuito)	0,00
Infraestrutura	Banco de dados PostgreSQL em contêiner (software livre)	0,00
Infraestrutura	Docker, Docker Compose e nginx (software livre)	0,00
Infraestrutura	Ferramentas de desenvolvimento — VS Code, Figma, GitHub (gratuitos)	0,00
	Subtotal — infraestrutura e terceiros	0,00
	Custo total estimado do projeto	32.000,00

Fonte: Os autores. Referência salarial: Glassdoor (2026).

4.3 RISCOS

Conforme metodologia de análise de riscos proposta por Pressman e Maxim (2016), foram identificados os principais riscos do projeto, classificados em termos de probabilidade e impacto, com suas respectivas estratégias de mitigação. O Quadro 1 apresenta a matriz consolidada de riscos, atualizada com os riscos identificados durante a implementação.

Quadro 1 – Matriz de riscos do projeto

Risco	Probabilidade	Impacto	Estratégia de mitigação
Escopo ampliado além da capacidade da equipe	Média	Alto	Definição clara do escopo e separação entre o escopo do projeto e as funcionalidades de evolução futura.
Dificuldade na futura integração com a API da Intelbras	Média	Alto	Controle de acesso já modelado de forma lógica (reservas, bloqueios e penalidades), de modo que a integração física se limite a consumir essas informações.
Dificuldade de adaptação da cliente ao novo sistema	Baixa	Médio	Treinamento presencial e manual de uso simplificado.
Indisponibilidade dos serviços de hospedagem gratuitos	Baixa	Médio	Sistema empacotado em contêineres, portátil para qualquer provedor que execute Docker.
Conflitos de agenda entre os membros da equipe	Média	Médio	Metodologia ágil com entregas semanais e revisão das alterações por pull request.
Mudança nos requisitos por parte da cliente	Alta	Alto	Documentação formal dos requisitos com validação da cliente e controle de mudanças registrado.
Desistência de um membro da equipe	Baixa	Alto	Distribuição equilibrada de tarefas e documentação contínua do progresso.
Falhas ou defeitos após a implementação	Média	Alto	Testes contínuos, revisão de código e período de homologação antes de cada banca.
Regra de negócio implementada apenas na aplicação	Média	Alto	Regras críticas implementadas no banco de dados por gatilhos, garantindo a integridade independentemente da camada de aplicação.
Divergência do esquema do banco entre os ambientes dos desenvolvedores	Alta	Médio	Scripts do banco como fonte única do esquema e recriação obrigatória do volume de dados sempre que os scripts forem alterados.
Uso indevido dos atalhos de demonstração em produção	Baixa	Alto	Atalhos condicionados ao modo de demonstração, desativado por padrão no ambiente de produção.

Fonte: Os autores, com base em Pressman e Maxim (2016).

4.4 MODELAGEM DO SISTEMA

A modelagem do OASIS adota a abordagem orientada a objetos com notação UML (Unified Modeling Language). São apresentados a seguir o Diagrama de Casos de Uso com suas descrições detalhadas, o Diagrama de Classes e os Diagramas de Sequência. Os diagramas de banco de dados, o Dicionário de Dados e os *scripts* de criação são apresentados na seção 4.5.

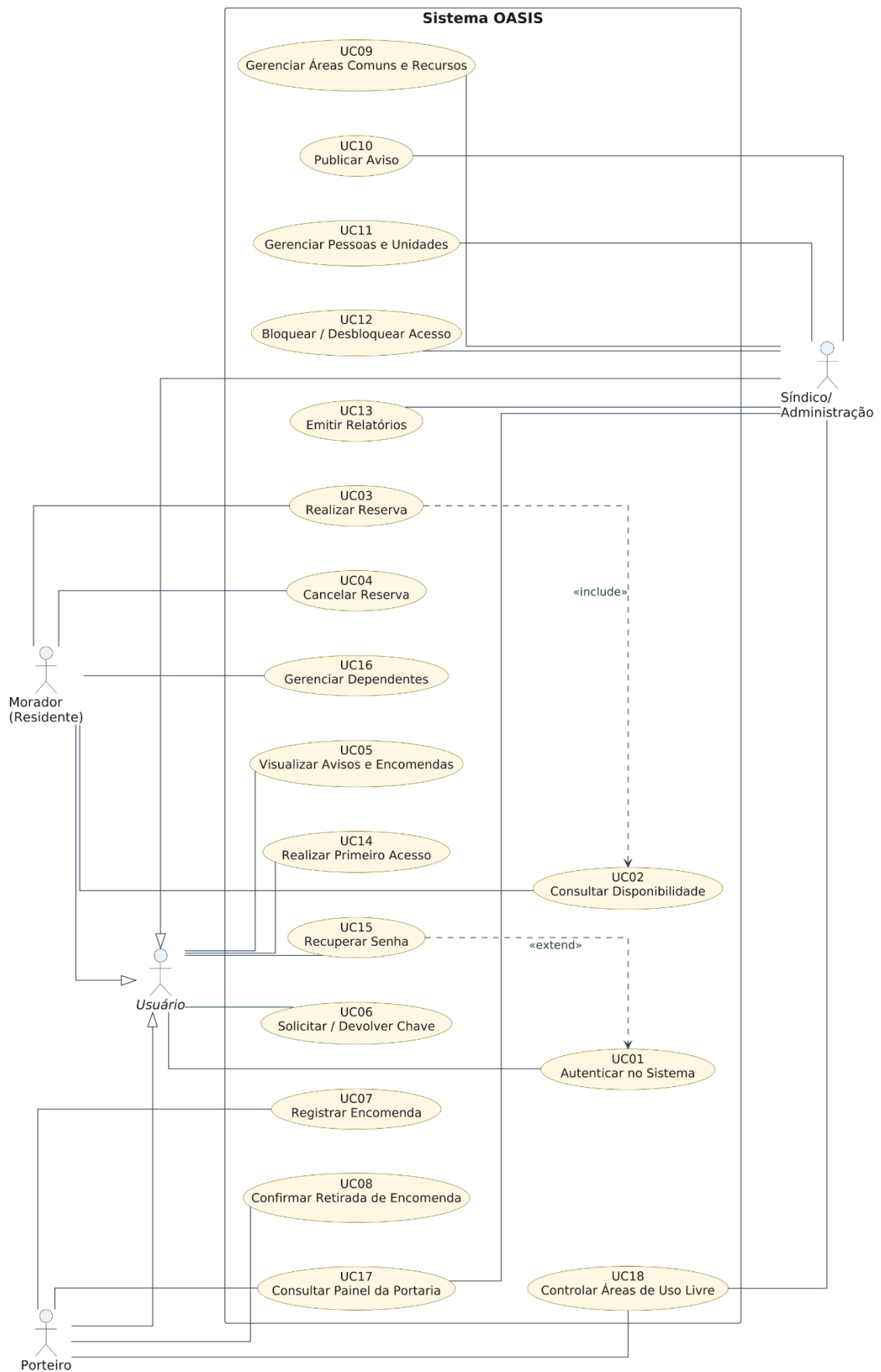
4.4.1 Diagrama de casos de uso

O Diagrama de Casos de Uso, apresentado na Figura 1, representa as interações entre os três perfis de atores humanos — Morador (Residente), Síndico/Administração e Porteiro — e o sistema OASIS. Em relação à versão anterior, o sistema passou de 13 para 18 casos de uso. Foram criados cinco casos de uso: Realizar Primeiro Acesso (UC14), Recuperar Senha (UC15), Gerenciar Dependentes (UC16), Consultar Painel da Portaria (UC17) e Controlar Áreas de Uso Livre (UC18). Outras funcionalidades implementadas foram incorporadas a casos de uso existentes, por representarem novas formas de atingir um objetivo que o ator já possuía: a gestão

de chaves físicas passou a integrar o UC09, a aprovação de dependentes passou a integrar o UC11 e a consulta das próprias encomendas passou a integrar o UC05.

O perfil ADMINISTRADOR, presente no tipo enumerado de perfis do banco de dados, é tratado pela aplicação como sinônimo de SÍNDICO e, por isso, não constitui um ator distinto. Uma mesma pessoa pode acumular mais de um perfil — a síndica, por exemplo, é também moradora — e, nesse caso, participa dos casos de uso de ambos os atores. Como os três perfis compartilham um conjunto de casos de uso, o diagrama emprega a generalização de atores prevista na UML: o ator abstrato *Usuário*, com o nome em itálico, representa qualquer pessoa autenticável e está ligado aos casos de uso comuns a todos os perfis — Autenticar no Sistema (UC01), Visualizar Avisos e Encomendas (UC05), Solicitar / Devolver Chave (UC06), Realizar Primeiro Acesso (UC14) e Recuperar Senha (UC15). Morador, Síndico/Administração e Porteiro especializam o ator *Usuário*, herdando esses casos de uso, e se ligam diretamente apenas aos casos de uso específicos de cada perfil. O ator *Usuário* não corresponde a um quarto perfil do sistema. Os casos de uso são detalhados individualmente nos Quadros 2 a 19, conforme a notação proposta por Fowler (2004).

Figura 1 – Diagrama de casos de uso do software OASIS



Fonte: Os autores.

Quadro 2 – Descrição do UC01: Autenticar no Sistema

Campo	Descrição do caso de uso
Identificador	UC01
Nome	Autenticar no Sistema
Ator(es)	Morador (Residente) / Síndico/Administração / Porteiro
Descrição	Permite a qualquer usuário com conta ativa entrar no sistema informando e-mail e senha. A senha é conferida com o resumo criptográfico armazenado no banco e, em caso de sucesso, o sistema emite um token de sessão com os perfis vigentes da pessoa, válido por oito horas.
Pré-condição	Usuário com conta ativa (status ATIVO) e ao menos um perfil vigente.
Fluxo principal	1. Usuário acessa a tela de login. 2. Informa e-mail e senha. 3. Sistema verifica se o limite de tentativas para o IP e o e-mail informados não foi excedido. 4. Sistema localiza a pessoa pelo e-mail e verifica a situação da conta. 5. Sistema confere a senha com o resumo PBKDF2 armazenado. 6. Sistema recupera os perfis vigentes e as unidades da pessoa. 7. Sistema emite o token de sessão e redireciona o usuário à tela de início, com o conteúdo do perfil de maior privilégio.
Fluxo alternativo	A1 – Credenciais inválidas: sistema exibe mensagem genérica, sem revelar se o e-mail existe. A2 – Conta bloqueada ou sem perfil vigente: sistema informa que a conta está indisponível. A3 – Conta aguardando primeiro acesso: sistema orienta o usuário a ativar a conta pelo código (UC14). A4 – Limite de tentativas excedido: sistema recusa novas tentativas temporariamente, com mensagem em português. A5 – Usuário esqueceu a senha: aciona o UC15 (Recuperar Senha).
Pós-condição	Usuário autenticado, com sessão válida e acesso às telas dos seus perfis.

Fonte: Os autores.

Quadro 3 – Descrição do UC02: Consultar Disponibilidade

Campo	Descrição do caso de uso
Identificador	UC02
Nome	Consultar Disponibilidade
Ator(es)	Morador (Residente)
Descrição	Permite ao morador consultar os intervalos disponíveis de uma área comum em uma data, considerando os horários de funcionamento do dia da semana, as reservas existentes, a capacidade restante, o limite de reservas simultâneas e os bloqueios programados.
Pré-condição	Morador autenticado.
Fluxo principal	1. Morador acessa o módulo de reservas e seleciona uma área comum. 2. Seleciona uma data no calendário mensal. 3. Sistema recupera as janelas de funcionamento do dia da semana. 4. Sistema divide as janelas em intervalos conforme a duração configurada ou, nas áreas de reserva por dia inteiro, apresenta o dia como um único intervalo. 5. Sistema marca cada intervalo como livre, parcialmente ocupado, lotado ou bloqueado. 6. Sistema exibe a grade de horários e as regras da área.
Fluxo alternativo	A1 – Área bloqueada para manutenção: sistema exibe o motivo e o período do bloqueio. A2 – Área sem horário no dia: sistema informa que a área não funciona na data. A3 – Área de uso livre: a área não é oferecida para reserva; seu estado é controlado pela portaria (UC18).
Pós-condição	Disponibilidade exibida ao morador.

Fonte: Os autores.

Quadro 4 – Descrição do UC03: Realizar Reserva

Campo	Descrição do caso de uso
Identificador	UC03
Nome	Realizar Reserva
Ator(es)	Morador (Residente)
Descrição	Permite ao morador reservar uma área comum em intervalo disponível, respeitando as regras configuradas na área. Inclui a consulta de disponibilidade (UC02). A reserva do elevador de serviço para mudanças também é realizada por este caso de uso.
Pré-condição	Morador autenticado, sem penalidade vigente, e intervalo disponível.
Fluxo principal	1. Morador consulta a disponibilidade (UC02, «include»). 2. Seleciona o intervalo e informa o número de pessoas. 3. Morador confirma a reserva. 4. Sistema registra a reserva com status ATIVA, momento em que os gatilhos validam as regras RN01 a RN07 e RN16. 5. Se a área possuir valor de uso, sistema informa que a taxa será lançada no boleto condominial. 6. Sistema exibe a confirmação.
Fluxo alternativo	A1 – Unidade já possui reserva no mesmo horário, ou limite de reservas simultâneas atingido (RN01). A2 – Capacidade do horário excedida, considerando as reservas simultâneas (RN04). A3 – Antecedência desrespeitada ou horário no passado (RN06). A4 – Limite diário, semanal ou mensal da unidade atingido (RN07). A5 – Morador abaixo da idade mínima da área (RN16). A6 – Área ou morador bloqueado (RN02 e RN03).
Pós-condição	Reserva registrada com status ATIVA.

Fonte: Os autores.

Quadro 5 – Descrição do UC04: Cancelar Reserva

Campo	Descrição do caso de uso
Identificador	UC04
Nome	Cancelar Reserva
Ator(es)	Morador (Residente)
Descrição	Permite ao morador cancelar uma reserva ativa dentro do prazo de cancelamento configurado para a área. Reservas cujo horário já terminou são concluídas automaticamente e não podem ser canceladas.
Pré-condição	Morador autenticado; reserva ATIVA dentro do prazo.
Fluxo principal	1. Morador acessa a lista de suas reservas, com filtros por situação. 2. Seleciona a reserva e confirma o cancelamento, informando opcionalmente o motivo. 3. Sistema altera o status para CANCELADA; o gatilho RN08 verifica o prazo e registra a data e a hora do cancelamento. 4. Sistema libera o intervalo para novas reservas.
Fluxo alternativo	A1 – Fora do prazo: sistema informa o limite de horas configurado (RN08). A2 – Reserva já concluída: o sistema não oferece a opção de cancelar (RN08). A3 – Cancelamento administrativo, por manutenção ou por penalidade: não está sujeito ao prazo do morador.
Pós-condição	Reserva cancelada e intervalo liberado.

Fonte: Os autores.

Quadro 6 – Descrição do UC05: Visualizar Avisos e Encomendas

Campo	Descrição do caso de uso
Identificador	UC05
Nome	Visualizar Avisos e Encomendas
Ator(es)	Morador (Residente) / Síndico/Administração / Porteiro
Descrição	Permite ao usuário ler os comunicados a ele destinados no mural, separados entre comunicados da administração e avisos de encomenda, marcar cada aviso como lido e, no caso do morador, consultar as próprias encomendas e sua situação.
Pré-condição	Usuário autenticado.
Fluxo principal	1. Usuário acessa o mural de avisos. 2. Sistema exibe os avisos em ordem cronológica decrescente, separados por aba, com destaque para os fixados e não lidos. 3. Usuário abre um aviso; o gatilho RN14 registra a data e a hora da leitura. 4. O morador acessa a tela de encomendas e consulta as que aguardam retirada e as já retiradas.
Fluxo alternativo	A1 – Sem avisos ou sem encomendas: sistema exibe mensagem informativa.
Pós-condição	Avisos exibidos com leitura registrada; encomendas consultadas.

Fonte: Os autores.

Quadro 7 – Descrição do UC06: Solicitar / Devolver Chave

Campo	Descrição do caso de uso
Identificador	UC06
Nome	Solicitar / Devolver Chave
Ator(es)	Morador (Residente) / Porteiro / Síndico/Administração
Descrição	Permite registrar o empréstimo e a devolução de chaves de áreas comuns, feitos pelo porteiro ou pela administração, a moradores e também a visitantes e prestadores de serviço cadastrados. O morador consulta as chaves que estão em posse de sua família.
Pré-condição	Chave cadastrada com status DISPONIVEL.
Fluxo principal	1. Morador, visitante ou prestador de serviço solicita a chave na portaria. 2. Funcionário localiza a chave e a pessoa e registra o empréstimo, vinculando-a ao perfil do morador ou, no caso de visitante ou prestador, diretamente à pessoa, e, se houver, à reserva. 3. Gatilho RN09 valida a disponibilidade e altera o status para EMPRESTADA. 4. Na devolução, o funcionário registra o retorno; o gatilho RN10 valida a data e libera a chave. 5. O morador visualiza, na tela de início, as chaves em posse de sua família.
Fluxo alternativo	A1 – Chave já emprestada: sistema exibe o responsável e impede novo empréstimo (RN09). A2 – Devolução anterior à retirada ou sem funcionário informado: sistema recusa (RN10).
Pós-condição	Empréstimo ou devolução registrado e status da chave atualizado.

Fonte: Os autores.

Quadro 8 – Descrição do UC07: Registrar Encomenda

Campo	Descrição do caso de uso
Identificador	UC07
Nome	Registrar Encomenda
Ator(es)	Porteiro
Descrição	Permite ao porteiro registrar o recebimento de uma encomenda destinada a um morador. O sistema gera automaticamente um aviso individual no mural do destinatário.
Pré-condição	Porteiro autenticado; destinatário cadastrado.
Fluxo principal	1. Porteiro busca o destinatário por nome, CPF, bloco ou apartamento. 2. Informa a descrição e o tamanho da encomenda. 3. Sistema registra a encomenda com status AGUARDANDO_RETIRADA e o perfil do porteiro responsável. 4. Gatilho RN11 gera um aviso individual para o destinatário, exibido na aba de encomendas do mural.
Fluxo alternativo	A1 – Morador não encontrado: sistema permite nova busca.
Pós-condição	Encomenda registrada e morador avisado no mural.

Fonte: Os autores.

Quadro 9 – Descrição do UC08: Confirmar Retirada de Encomenda

Campo	Descrição do caso de uso
Identificador	UC08
Nome	Confirmar Retirada de Encomenda
Ator(es)	Porteiro
Descrição	Permite ao porteiro alterar a situação de uma encomenda, confirmando a retirada com data, hora e responsável, ou registrando a devolução ao remetente.
Pré-condição	Encomenda com status AGUARDANDO_RETIRADA.
Fluxo principal	1. Porteiro localiza a encomenda e abre a tela de alteração de status. 2. Informa quem retirou (próprio, terceiro ou porteiro) e, se terceiro, o nome do retirante. 3. Gatilho RN12 valida o preenchimento e atualiza o status para RETIRADA.
Fluxo alternativo	A1 – Responsável não informado, ou nome do terceiro ausente: sistema recusa (RN12). A2 – Devolução ao remetente: status DEVOLVIDA.
Pós-condição	Encomenda com retirada registrada.

Fonte: Os autores.

Quadro 10 – Descrição do UC09: Gerenciar Áreas Comuns e Recursos

Campo	Descrição do caso de uso
Identificador	UC09
Nome	Gerenciar Áreas Comuns e Recursos
Ator(es)	Síndico/Administração
Descrição	Permite à administração manter as áreas comuns e seus recursos: horários de funcionamento por dia da semana, utensílios, imagem ilustrativa e chaves físicas. Permite configurar as regras de reserva de cada área e definir se a área exige reserva ou é de uso livre.
Pré-condição	Administração autenticada.
Fluxo principal	1. Administração acessa a gestão de áreas comuns e seleciona a operação. 2. Informa nome, descrição, capacidade, idade mínima, tipo de acesso e tipo de uso. 3. Configura as regras: duração do intervalo ou reserva por dia inteiro, antecedência mínima e máxima, prazo de cancelamento, tipo e quantidade do limite por unidade e quantidade de reservas simultâneas. 4. Cadastra horários, utensílios e chaves físicas da área. 5. Envia a imagem da área, que é redimensionada e comprimida automaticamente.
Fluxo alternativo	A1 – Nome duplicado: sistema recusa. A2 – Imagem em formato inválido ou acima do limite: sistema informa o motivo antes do envio. A3 – Exclusão de área com reservas, chaves ou bloqueios associados: sistema recusa e oferece a inativação, preservando o histórico. A4 – Inativação: a área deixa de ser oferecida aos moradores e pode ser reativada a qualquer momento.
Pós-condição	Area comum e seus recursos atualizados.

Fonte: Os autores.

Quadro 11 – Descrição do UC10: Publicar Aviso

Campo	Descrição do caso de uso
Identificador	UC10
Nome	Publicar Aviso
Ator(es)	Síndico/Administração
Descrição	Permite à administração publicar e remover comunicados no mural, com distribuição automática a todos os perfis ativos.
Pré-condição	Administração autenticada.
Fluxo principal	1. Administração cria o aviso informando título, conteúdo e, opcionalmente, expiração e fixação no topo. 2. Publica o aviso com escopo MURAL. 3. Gatilho RN13 gera um registro de destinatário para cada perfil ativo.
Fluxo alternativo	A1 – Remoção do aviso: os registros de destinatários são excluídos em cascata.
Pós-condição	Aviso publicado e distribuído.

Fonte: Os autores.

Quadro 12 – Descrição do UC11: Gerenciar Pessoas e Unidades

Campo	Descrição do caso de uso
Identificador	UC11
Nome	Gerenciar Pessoas e Unidades
Ator(es)	Síndico/Administração
Descrição	Permite à administração manter blocos, unidades, vagas, pessoas e perfis, aprovar os dependentes cadastrados pelos titulares e gerar os códigos de primeiro acesso. Uma pessoa pode receber mais de um perfil. Permite ainda cadastrar visitantes e prestadores de serviço, que ficam registrados no sistema sem acesso a ele.
Pré-condição	Administração autenticada.
Fluxo principal	1. Administração cadastra a pessoa com dados pessoais, unidade, tipo de vínculo e perfil. 2. Sistema cria a conta com status AGUARDANDO_PRIMEIRO_ACESSO e gera o código de ativação, entregue ao morador. 3. Administração consulta a lista de aprovações pendentes e aprova ou rejeita, com motivo, os dependentes cadastrados pelos titulares. 4. Na aprovação, sistema gera o código de primeiro acesso do dependente. 5. Para visitante ou prestador de serviço, administração informa os dados pessoais, a unidade vinculada, se houver, e, no caso do prestador, a descrição do serviço; o sistema não cria perfil nem código de acesso. 6. Administração pode editar, inativar, reativar ou excluir pessoas.
Fluxo alternativo	A1 – E-mail ou CPF duplicado: sistema recusa. A2 – Unidade já possui titular ativo: sistema impede um segundo titular. A3 – Tentativa de remover o único administrador do sistema: sistema recusa. A4 – Encerramento de vínculo: informada a data de fim, preservando o histórico.
Pós-condição	Cadastro atualizado e, quando aplicável, código de primeiro acesso gerado.

Fonte: Os autores.

Quadro 13 – Descrição do UC12: Bloquear / Desbloquear Acesso

Campo	Descrição do caso de uso
Identificador	UC12
Nome	Bloquear / Desbloquear Acesso
Ator(es)	Síndico/Administração
Descrição	Permite registrar dois tipos de restrição: o bloqueio de uma área comum, que a torna indisponível a todos em um período (manutenção, limpeza, evento ou obra); e a penalidade a um morador, que o impede de reservar uma área ou todas elas.
Pré-condição	Administração autenticada.
Fluxo principal	1. Administração escolhe o tipo de restrição e informa período e motivo. 2. Sistema registra a restrição e o perfil responsável. 3. Na penalidade, o gatilho RN17 cancela automaticamente as reservas ativas do morador no período e na área atingidos. 4. Administração pode encerrar a restrição antes do prazo.
Fluxo alternativo	A1 – Bloqueio de área sobre reservas existentes: sistema alerta a administração.
Pós-condição	Restrição registrada e reservas afetadas canceladas.

Fonte: Os autores.

Quadro 14 – Descrição do UC13: Emitir Relatórios

Campo	Descrição do caso de uso
Identificador	UC13
Nome	Emitir Relatórios
Ator(es)	Síndico/Administração
Descrição	Permite à administração consultar indicadores consolidados do condomínio: reservas por período, áreas mais utilizadas, encomendas pendentes, moradores ativos e chaves em uso, com gráficos e filtros.
Pré-condição	Administração autenticada.
Fluxo principal	1. Administração acessa o painel administrativo. 2. Seleciona o período e os filtros desejados. 3. Sistema consolida os dados e exibe indicadores e gráficos.
Fluxo alternativo	A1 – Sem dados no período: sistema informa a ausência de registros.
Pós-condição	Indicadores exibidos.

Fonte: Os autores.

Quadro 15 – Descrição do UC14: Realizar Primeiro Acesso

Campo	Descrição do caso de uso
Identificador	UC14
Nome	Realizar Primeiro Acesso
Ator(es)	Morador (Residente) / Síndico/Administração / Porteiro
Descrição	Permite à pessoa cadastrada ativar sua conta com o código gerado pela administração, definindo a própria senha. Substitui o convite de definição de senha que era enviado pelo Firebase na versão anterior.
Pré-condição	Pessoa cadastrada com status AGUARDANDO_PRIMEIRO_ACESSO e código DISPONIVEL dentro da validade.
Fluxo principal	1. Usuário acessa a aba de primeiro acesso da tela de login. 2. Informa o código e o e-mail. 3. Sistema valida o código e exibe o nome e a unidade da pessoa para confirmação. 4. Usuário define e confirma a nova senha. 5. Sistema grava o resumo da senha, marca a conta como ATIVO e o código como UTILIZADO. 6. Sistema inicia a sessão do usuário.
Fluxo alternativo	A1 – Código inexistente, expirado, já utilizado ou de outra pessoa: sistema recusa. A2 – Excesso de tentativas: limitação de requisições aplicada.
Pós-condição	Conta ativada e sessão iniciada.

Fonte: Os autores.

Quadro 16 – Descrição do UC15: Recuperar Senha

Campo	Descrição do caso de uso
Identificador	UC15
Nome	Recuperar Senha
Ator(es)	Morador (Residente) / Síndico/Administração / Porteiro
Descrição	Estende o UC01 («extend»), permitindo ao usuário redefinir a senha por meio de um código de recuperação com validade de duas horas. O envio do código por e-mail é um trabalho futuro; nesta versão o código é obtido junto à administração ou exibido em modo de demonstração.
Pré-condição	Conta ativa.
Fluxo principal	1. Usuário aciona a opção "esqueci minha senha" e informa o e-mail. 2. Sistema cancela códigos de recuperação anteriores e gera um novo, válido por duas horas. 3. Usuário informa o código, o e-mail e a nova senha. 4. Sistema valida o código, grava o resumo da nova senha e marca o código como UTILIZADO.
Fluxo alternativo	A1 – E-mail não cadastrado: sistema responde de forma genérica, sem revelar se a conta existe. A2 – Código expirado ou inválido: sistema recusa.
Pós-condição	Senha redefinida.

Fonte: Os autores.

Quadro 17 – Descrição do UC16: Gerenciar Dependentes

Campo	Descrição do caso de uso
Identificador	UC16
Nome	Gerenciar Dependentes
Ator(es)	Morador (Residente)
Descrição	Permite ao morador titular (proprietário ou inquilino) cadastrar familiares vinculados à sua unidade. O cadastro fica pendente até a aprovação da administração (UC11).
Pré-condição	Morador titular ativo da unidade.
Fluxo principal	1. Titular acessa a tela Família. 2. Informa nome, CPF, data de nascimento, e-mail, celular e grau de parentesco. 3. Sistema registra o vínculo como DEPENDENTE, com status de aprovação PENDENTE; o gatilho RN15 confirma que o responsável é titular ativo da mesma unidade. 4. Após a aprovação, o titular visualiza o código de primeiro acesso para entregar ao dependente.
Fluxo alternativo	A1 – Responsável que não é titular ativo: sistema recusa (RN15). A2 – Cadastro rejeitado: sistema exibe o motivo informado pela administração.
Pós-condição	Dependente cadastrado, aguardando aprovação ou aprovado.

Fonte: Os autores.

Quadro 18 – Descrição do UC17: Consultar Painel da Portaria

Campo	Descrição do caso de uso
Identificador	UC17
Nome	Consultar Painel da Portaria
Ator(es)	Porteiro / Síndico/Administração
Descrição	Permite acompanhar em tempo real quais áreas estão ocupadas, por quem e até que horas; consultar a agenda de reservas de um dia; e buscar um morador para ver sua atividade atual, suas próximas reservas e seu histórico de uso.
Pré-condição	Porteiro ou administração autenticados.
Fluxo principal	1. Usuário acessa o painel da portaria, tela inicial do porteiro. 2. Sistema exibe as áreas ocupadas no momento, com tempo restante, atualizando a cada 60 segundos. 3. Sistema exibe a agenda do dia, com a situação de cada reserva. 4. Usuário busca um morador por nome, CPF, bloco ou apartamento. 5. Sistema exibe a atividade atual, as próximas reservas e o histórico dos últimos 90 dias.
Fluxo alternativo	A1 – Reserva com horário encerrado e não concluída: sinalizada como em atraso. A2 – Nenhuma área ocupada: mensagem informativa. A3 – Nenhuma pessoa encontrada na busca: mensagem informativa, permitindo nova busca. A4 – Usuário aciona Ver cadastro completo: sistema exibe a lista cadastral de pessoas em modo somente leitura, sem opções de edição.
Pós-condição	Informações exibidas; o histórico é limitado a 90 dias por se tratar de dado pessoal.

Fonte: Os autores.

Quadro 19 – Descrição do UC18: Controlar Áreas de Uso Livre

Campo	Descrição do caso de uso
Identificador	UC18
Nome	Controlar Areas de Uso Livre
Ator(es)	Porteiro / Síndico/Administração
Descrição	Permite à portaria registrar o estado das áreas que dispensam reserva, marcando-as como livres ou em uso, com observação, horário e identificação do funcionário.
Pré-condição	Área configurada para uso sem reserva.
Fluxo principal	1. Porteiro acessa a tela de áreas livres. 2. Seleciona a área e marca como LIVRE ou EM_USO, com observação opcional. 3. Sistema registra o estado, o horário da atualização e o funcionário.
Fluxo alternativo	A1 – Área que exige reserva: não aparece nesta tela.
Pós-condição	Estado da área atualizado.

Fonte: Os autores.

4.4.2 Diagrama de classes

O Diagrama de Classes representa a estrutura estática do sistema OASIS seguindo a notação UML proposta por Fowler (2004). O diagrama foi organizado de acordo com o padrão de análise *Boundary-Control-Entity* (BCE), que separa as classes em três estereótipos conforme sua responsabilidade, favorecendo a coesão e o baixo acoplamento entre a interface, a lógica de negócio e os dados. Para manter a legibilidade na página, o diagrama é apresentado em duas visões complementares: a visão de arquitetura, na Figura 2, com as classes de fronteira e de controle e suas dependências, na qual o pacote de entidades aparece recolhido; e o modelo de entidades, dividido nas Figuras 3 e 4, com os atributos e as associações das classes de entidade.

As classes de fronteira («*boundary*») representam as telas por meio das quais os atores interagem com o sistema; são 15 classes, uma para cada tela do Quadro 45: TelaLogin, TelaInicio, TelaMural, TelaReserva, TelaMinhasReservas, TelaMinhasEncomendas, TelaFamilia, TelaPainelPortaria, TelaAreasLivres, TelaEncomendasPortaria, TelaChaves, TelaPainelAdmin, TelaAreasComuns, TelaPessoas e TelaPublicarAviso. A TelaPainelPortaria depende da TelaPessoas porque incorpora a lista cadastral em modo somente leitura.

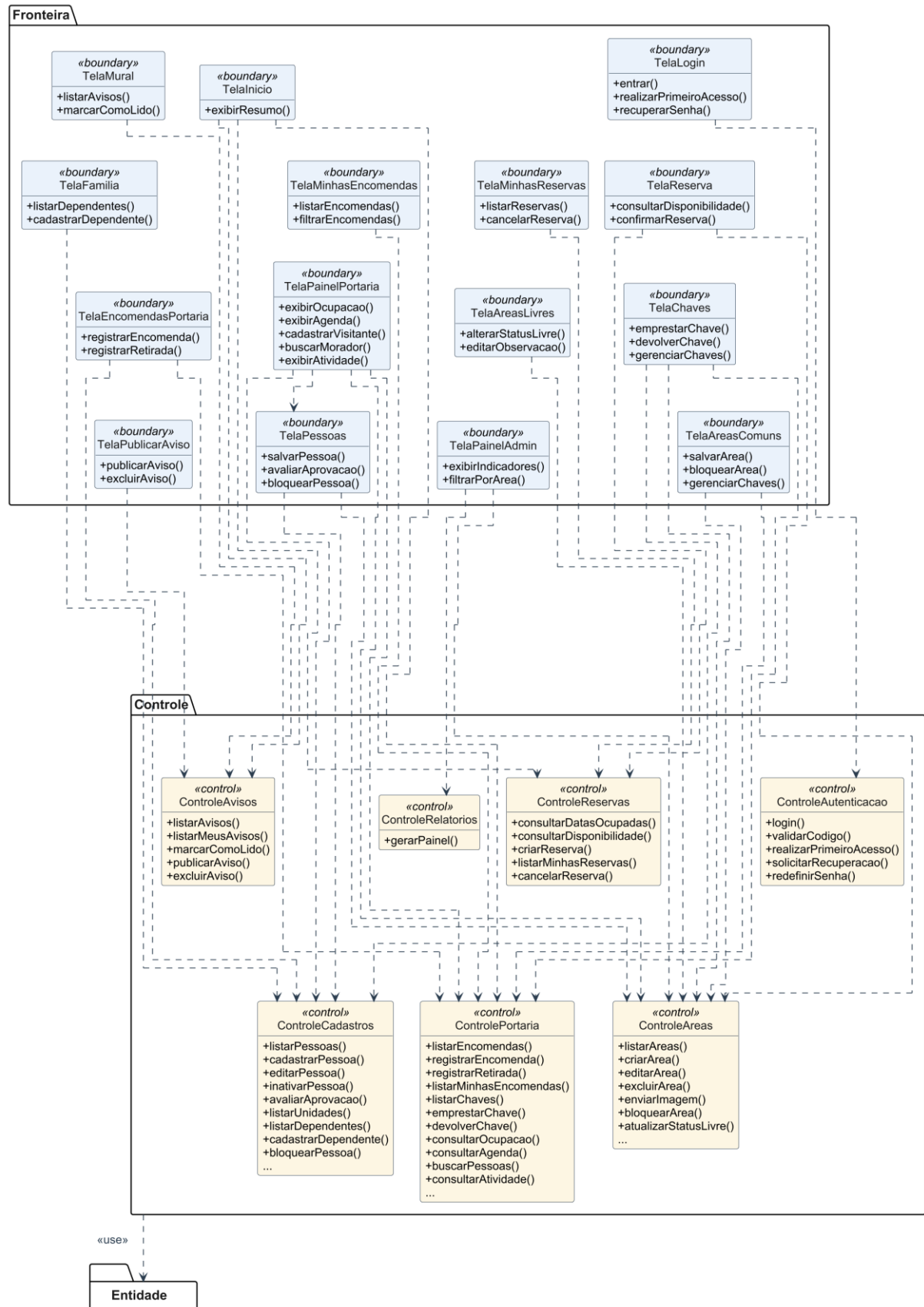
As classes de controle («*control*») correspondem aos módulos da API implementados em *NestJS* e coordenam a execução dos casos de uso: ControleAutenticacao, ControleCadastros, ControleAreas, ControleReservas, ControlePortaria, ControleAvisos e ControleRelatorios. Elas são protegidas pelos guardas de autenticação, de perfil e de limitação de tentativas.

As classes de entidade («*entity*») correspondem às dezoito tabelas do modelo relacional descrito na seção 4.5. Em relação à versão anterior, entrou a entidade CODIGO_PRIMEIRO_ACESSO, responsável pelos códigos de ativação de conta e de recuperação de senha; a entidade PESSOA passou a guardar o resumo da senha, a situação da conta, o papel da pessoa no condomínio — que distingue moradores e funcionários de visitantes e prestadores de serviço — e o serviço prestado; a entidade PESSOA_UNIDADE passou a registrar o titular responsável, o grau de parentesco e a situação de aprovação dos dependentes; e a entidade ENTREGA_CHAVE passou a admitir como solicitante uma pessoa sem perfil, para o empréstimo a visitantes e

prestadores. A classe PERFIL continua representando o usuário efetivo do sistema e é referenciada por todas as ações rastreáveis.

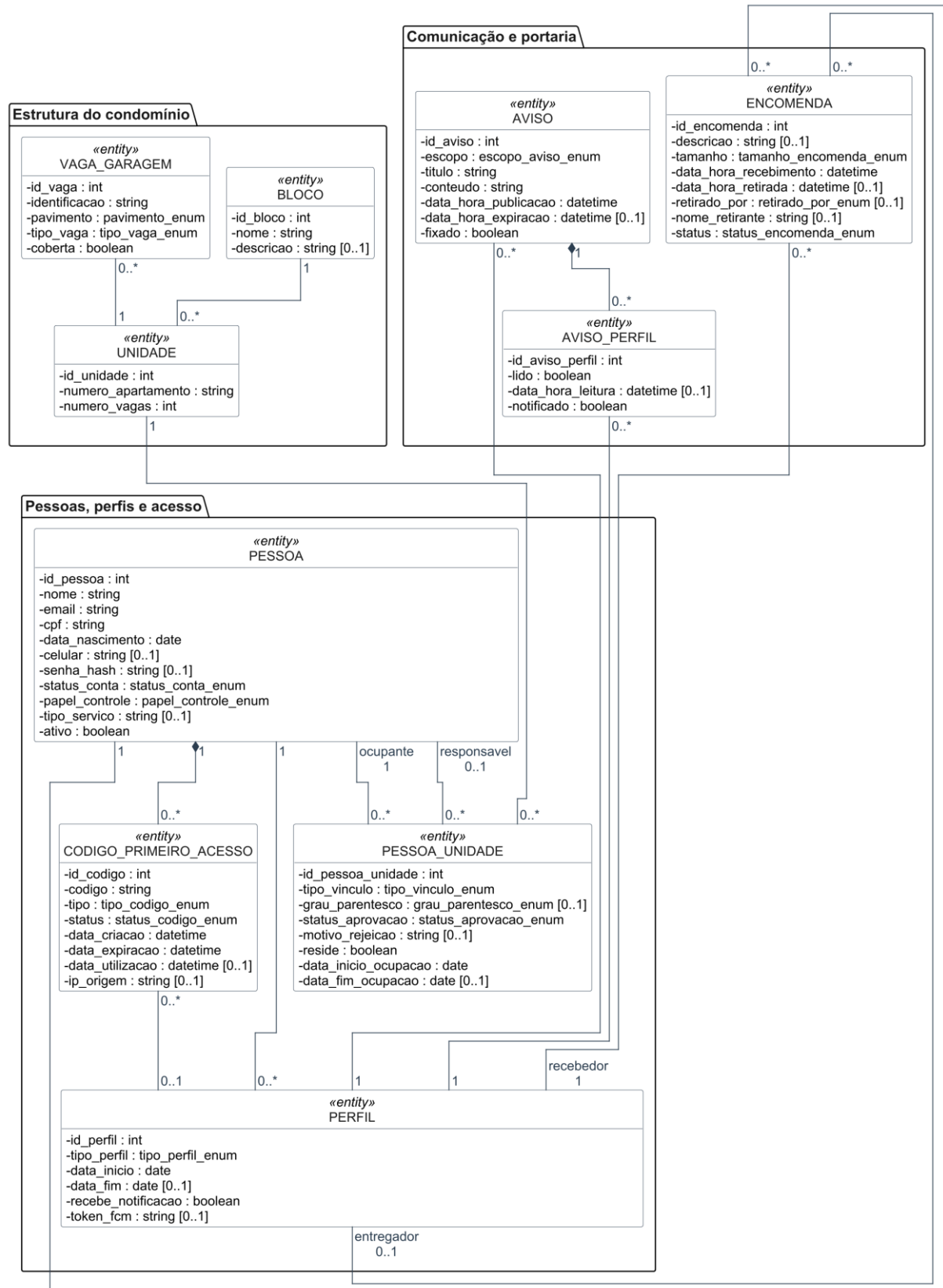
No modelo de entidades, cada tabela do banco corresponde a uma classe com todos os seus atributos, com visibilidade privada e tipo; os atributos opcionais recebem a multiplicidade [0..1]. As chaves estrangeiras não aparecem como atributos, mas como associações, uma para cada chave estrangeira, com multiplicidade nas duas pontas e nome de papel quando há mais de uma associação entre as mesmas classes. A composição, representada pelo losango preenchido, é usada apenas quando a parte não existe sem o todo: horários e utensílios de uma área comum, entregas de um aviso e códigos de primeiro acesso de uma pessoa. As entidades estão agrupadas em cinco pacotes: estrutura do condomínio, pessoas, perfis e acesso, e comunicação e portaria, na Figura 3; áreas comuns e recursos, e uso das áreas, na Figura 4. Nesta segunda figura, PERFIL e PESSOA aparecem apenas como referência, com os atributos suprimidos, para representar as associações entre as duas partes do modelo.

Figura 2 – Diagrama de classes do software OASIS: visão de arquitetura (fronteira e controle)



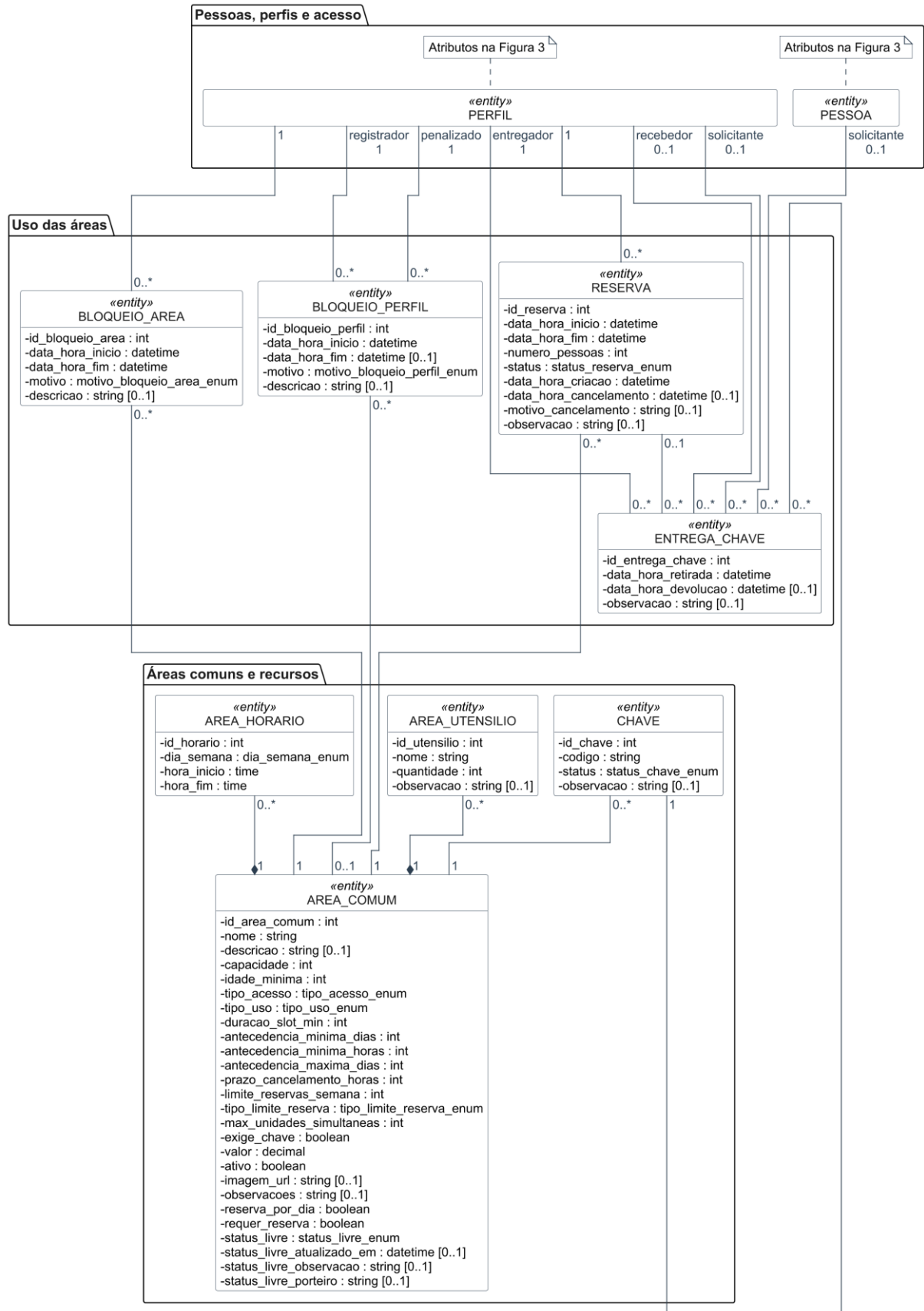
Fonte: Os autores.

Figura 3 – Diagrama de classes do software OASIS: modelo de entidades (cadastro e comunicação)



Fonte: Os autores.

Figura 4 – Diagrama de classes do software OASIS: modelo de entidades (áreas comuns e uso)



Fonte: Os autores.

4.4.3 Diagramas de sequência

Os Diagramas de Sequência, apresentados nas Figuras 5 a 9, ilustram a interação entre os objetos do sistema ao longo do tempo para os fluxos de maior relevância. As mensagens seguem a organização do padrão *Boundary-Control-Entity*, percorrendo a sequência tela → controlador → banco de dados, com os nomes de classes e operações do diagrama de classes da Figura 2, e as notas junto ao banco indicam em que ponto os gatilhos da seção 4.5.6 são disparados.

A Figura 5 apresenta a autenticação (UC01). O diagrama foi refeito em relação à versão anterior: o serviço externo de autenticação deixou de existir, e a sequência passa a mostrar a verificação do limite de tentativas pelo guarda de autenticação, a consulta da pessoa e da situação da conta, a conferência da senha com o resumo PBKDF2 e a emissão do *token* de sessão com os perfis vigentes.

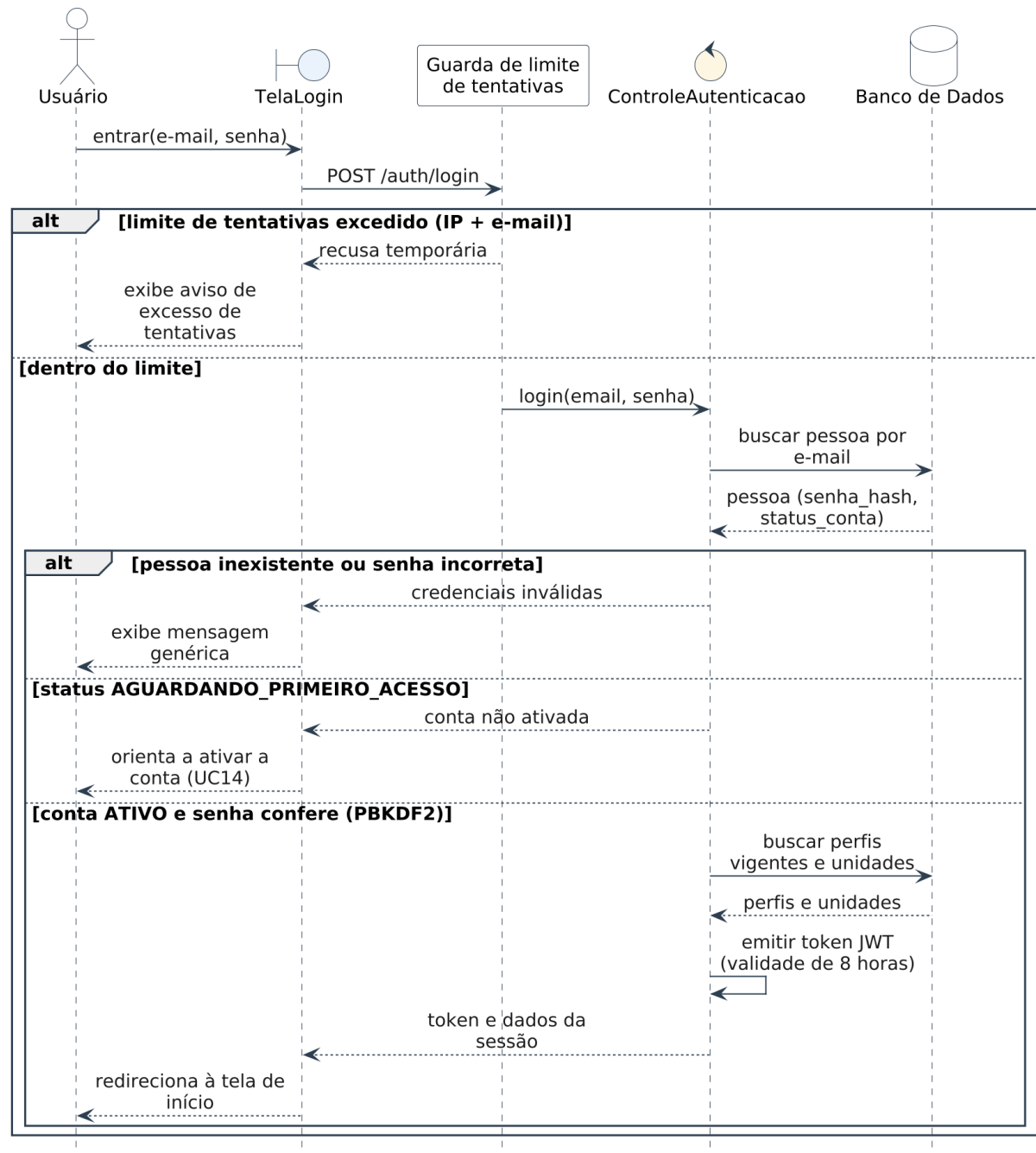
A Figura 6 apresenta a realização de reserva (UC03). A interação com a plataforma Intelbras foi removida, por constituir trabalho futuro, e a nota de gatilhos passou a listar as regras RN01 a RN07 e RN16, incluindo as reservas simultâneas, a capacidade acumulada, os limites por período e a idade mínima.

A Figura 7 apresenta o empréstimo e a devolução de chave (UC06), agora realizados pelo porteiro ou pela administração, com validação pelos gatilhos RN09 e RN10.

A Figura 8 apresenta o registro e a retirada de encomenda (UC07 e UC08). O envio de notificação por serviço externo foi removido; o aviso ao morador passa a ser gerado exclusivamente pelo gatilho RN11 no mural. A localização do destinatário é feita sobre a lista de pessoas fornecida pelo controle de cadastros, filtrada na própria tela.

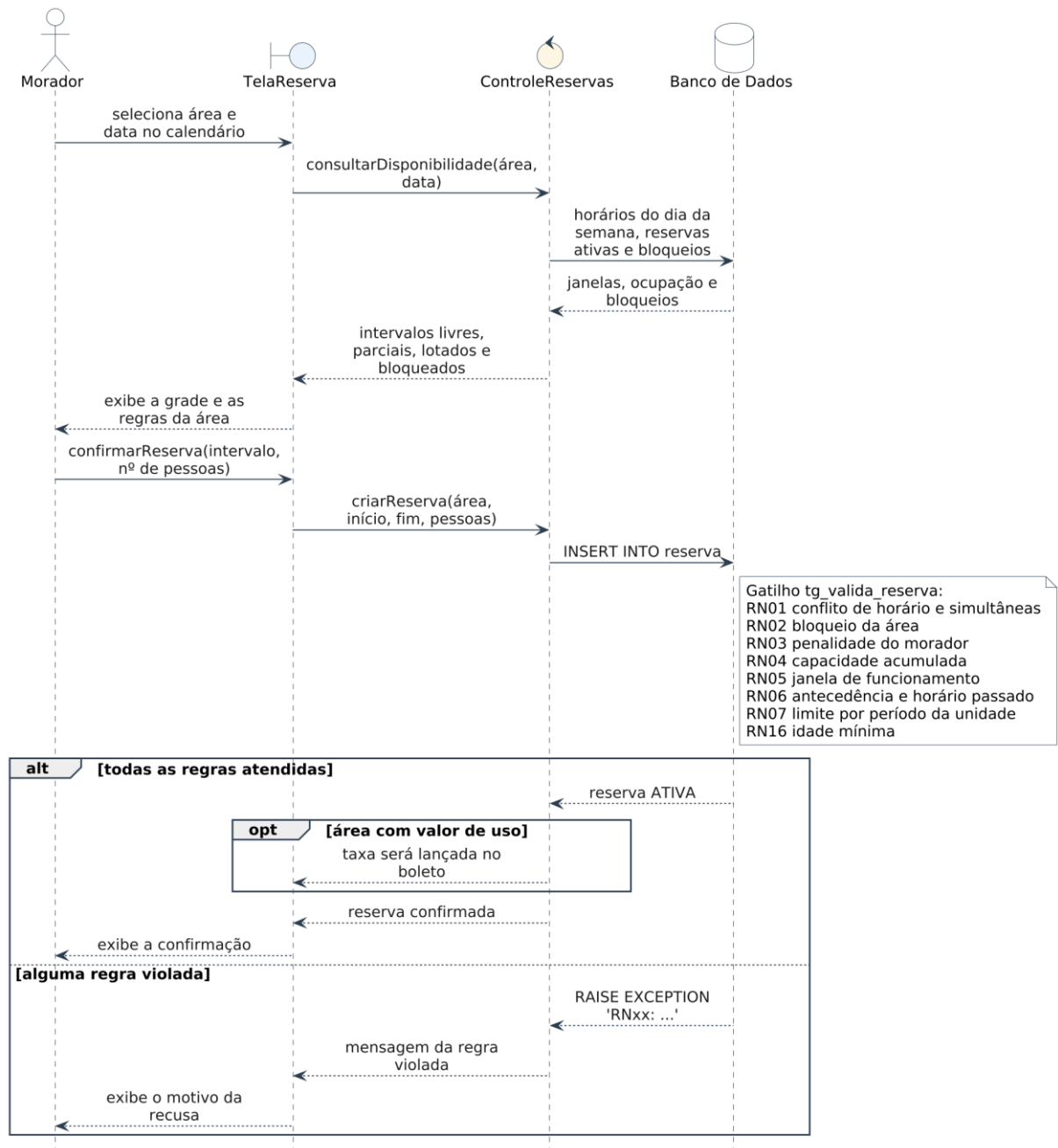
A Figura 9, criada nesta versão, apresenta o primeiro acesso por código (UC14): a validação do código e do e-mail, a exibição dos dados da pessoa para confirmação, a definição da senha e a ativação da conta, com a marcação do código como utilizado na mesma transação.

Figura 5 – Diagrama de sequência: Autenticar no Sistema (UC01)



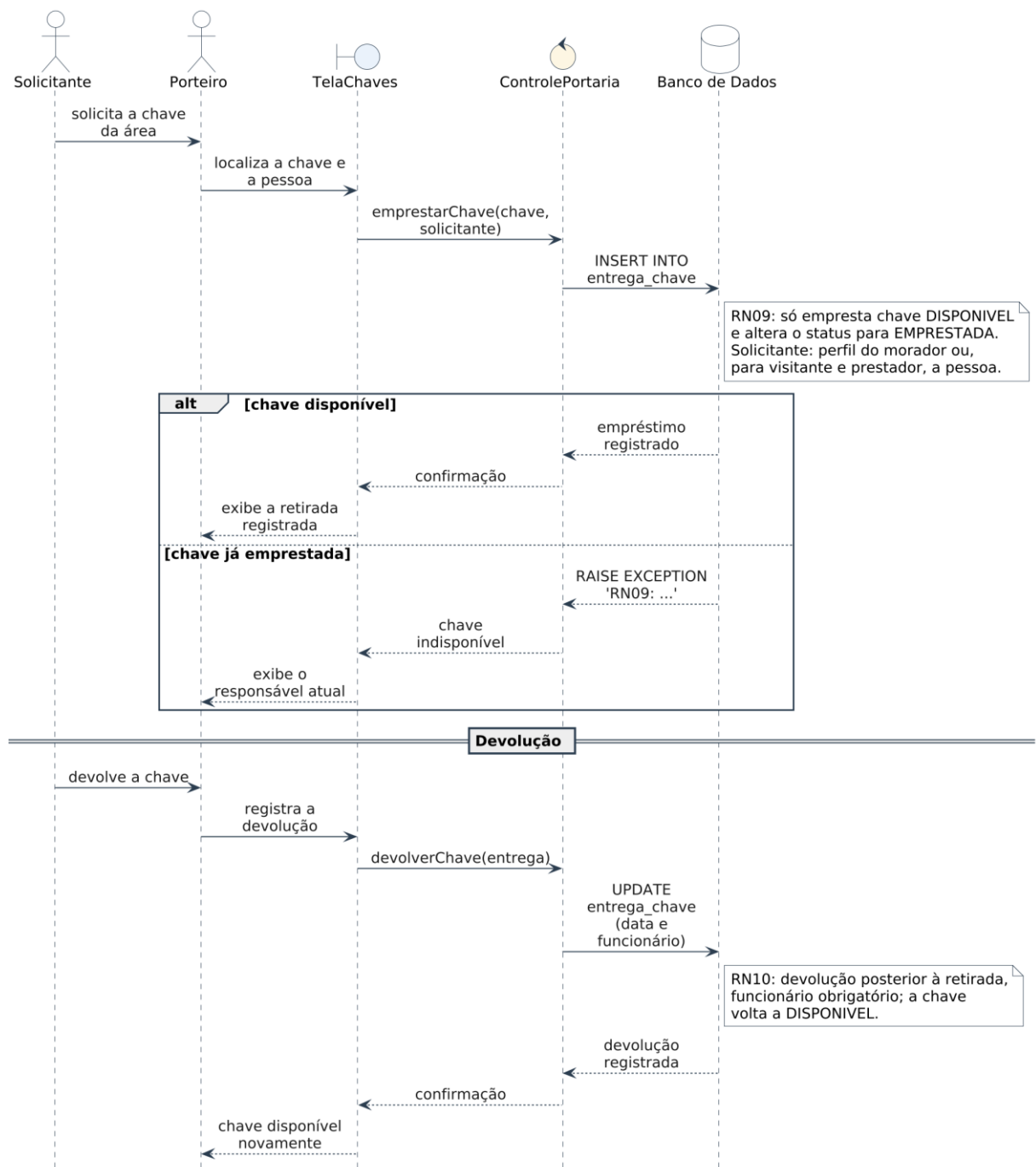
Fonte: Os autores.

Figura 6 – Diagrama de sequência: Realizar Reserva (UC03)



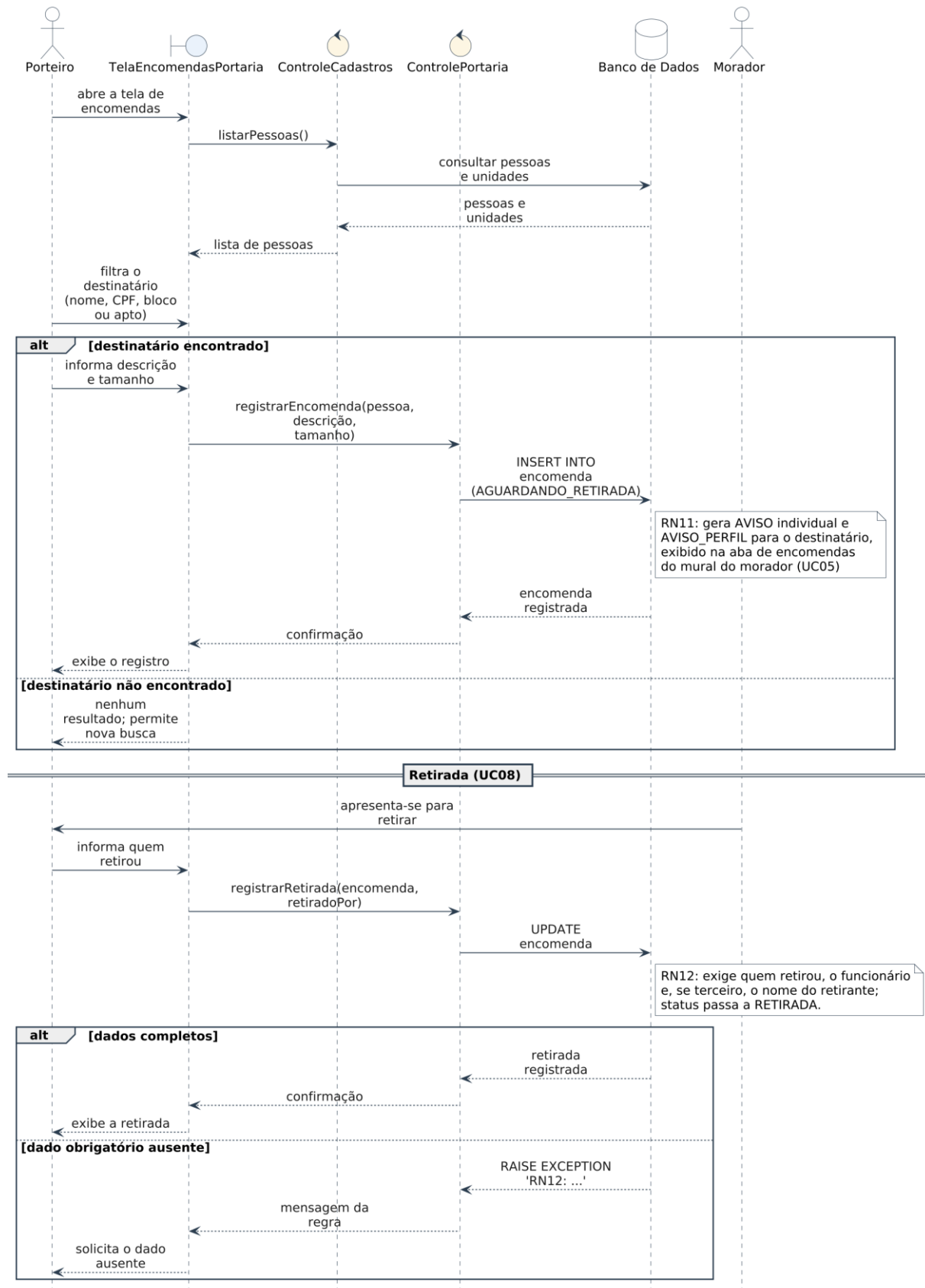
Fonte: Os autores.

Figura 7 – Diagrama de sequência: Solicitar / Devolver Chave (UC06)



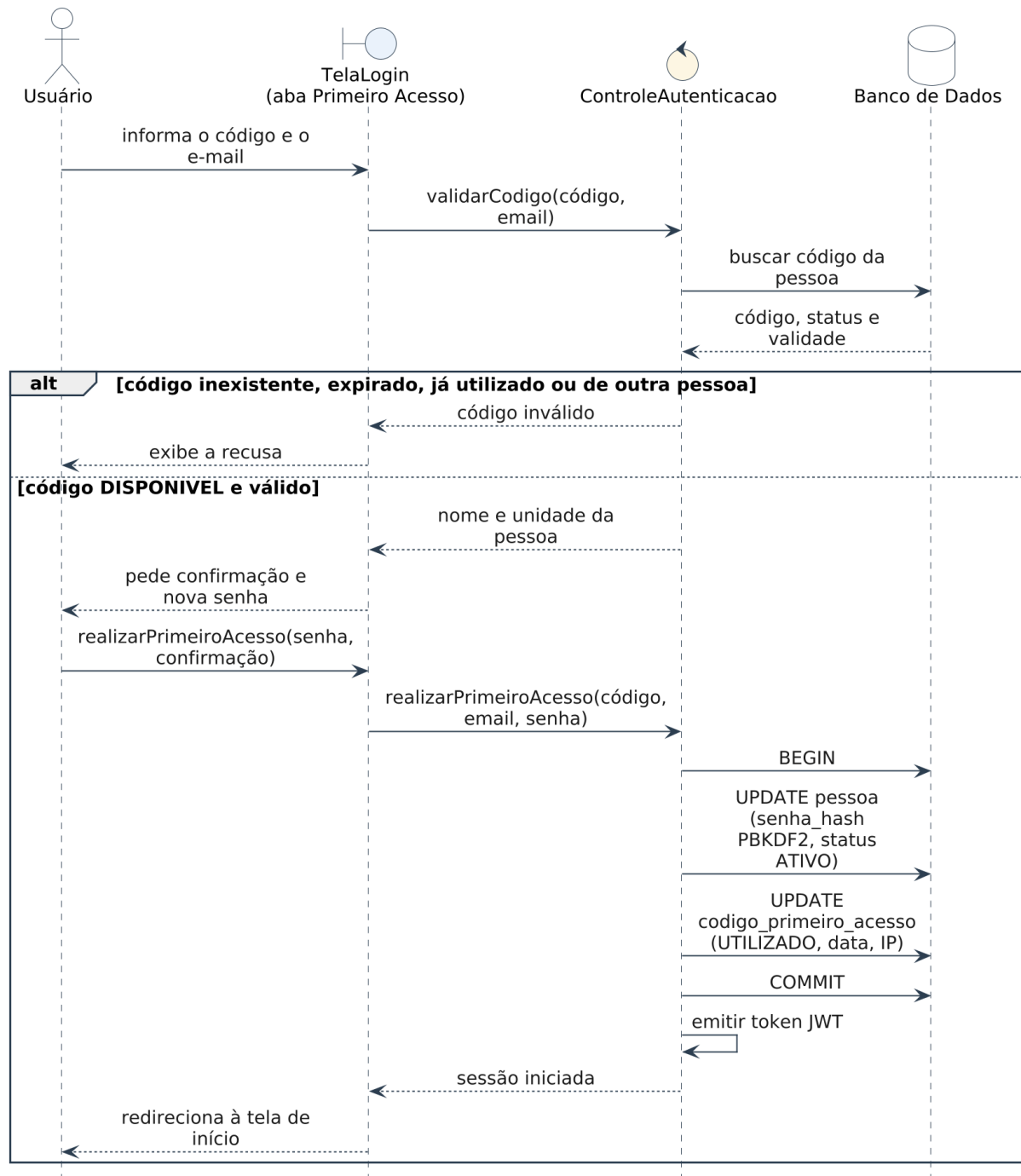
Fonte: Os autores.

Figura 8 – Diagrama de sequência: Registrar Encomenda e Confirmar Retirada (UC07 e UC08)



Fonte: Os autores.

Figura 9 – Diagrama de sequência: Realizar Primeiro Acesso (UC14)



Fonte: Os autores.

4.5 MODELAGEM DO BANCO DE DADOS

Esta seção apresenta a modelagem do banco de dados relacional do sistema OASIS: o Diagrama Entidade-Relacionamento (DER) conceitual, o Diagrama Lógico, os domínios enumerados, o Dicionário de Dados, os *scripts* de criação em *PostgreSQL* e as regras de negócio implementadas por gatilhos.

Em relação à versão anterior deste documento, o modelo passou de 17 para 18 tabelas, com a criação de CODIGO_PRIMEIRO_ACESSO, e de 15 para 23 tipos enumerados. As tabelas PESSOA, PESSOA_UNIDADE, AREA_COMUM e ENTREGA_CHAVE receberam novos atributos para suportar a autenticação própria, a gestão de dependentes, o cadastro de visitantes e prestadores de serviço e as novas regras de reserva. O conjunto de regras de negócio implementadas por gatilhos passou de 14 para 17.

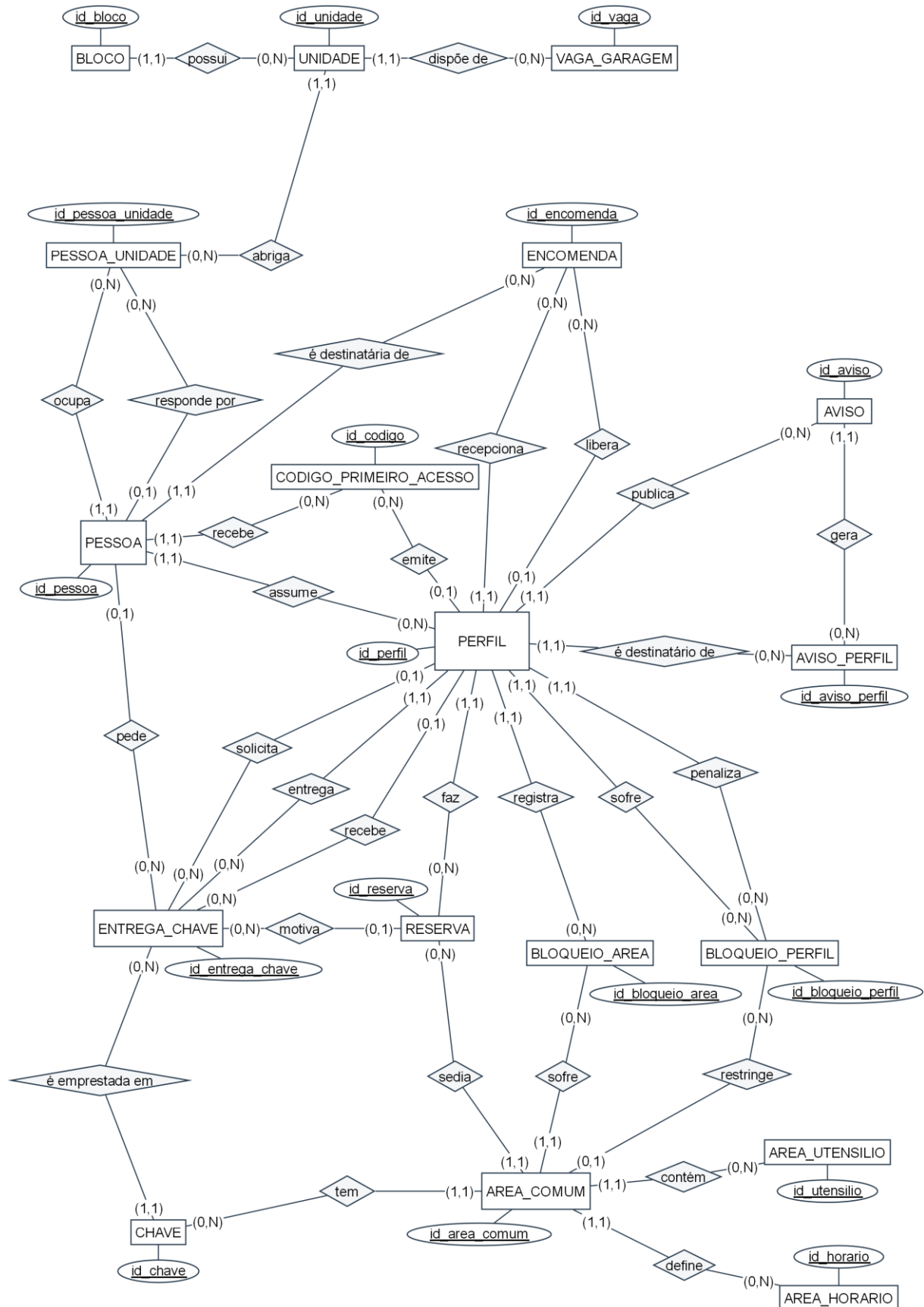
O modelo documentado nesta seção corresponde à forma consolidada do banco de dados, na qual os *scripts* de banco/ são a fonte única do esquema: todas as estruturas são criadas pelos *scripts*, sem alterações realizadas pela aplicação em tempo de execução; os atributos de domínio fechado utilizam tipos enumerados; e não há atributos redundantes. A consolidação do código-fonte a este modelo está prevista no cronograma da seção 5.

4.5.1 Diagrama entidade-relacionamento

O DER, apresentado na Figura 10, representa o modelo conceitual do banco de dados do OASIS na notação de Chen. No modelo conceitual não são representadas chaves estrangeiras, que surgem no mapeamento para o modelo relacional. Os atributos identificadores são apresentados sublinhados, os losangos representam os relacionamentos e os pares entre parênteses indicam as cardinalidades mínima e máxima. Para manter a legibilidade, o DER apresenta apenas o atributo identificador de cada entidade; os demais atributos constam do Diagrama Lógico e do Dicionário de Dados. Cada chave estrangeira do modelo relacional corresponde a um relacionamento, nomeado por um verbo lido da entidade referenciada para a entidade relacionada, totalizando 30 relacionamentos entre as 18 entidades.

Nesta versão foram acrescentados a entidade CODIGO_PRIMEIRO_ACESSO, com dois relacionamentos: recebe, com PESSOA (1,1 para 0,n), e emite, com PERFIL (0,1 para 0,n), que identifica o administrador que gerou o código; o relacionamento responde por, que liga cada vínculo de dependente ao titular responsável pela unidade; e o relacionamento pede, entre PESSOA e ENTREGA_CHAVE (0,1 para 0,n), que permite registrar o empréstimo de chaves a visitantes e prestadores de serviço, que não possuem perfil.

Figura 10 – Diagrama entidade-relacionamento do software OASIS

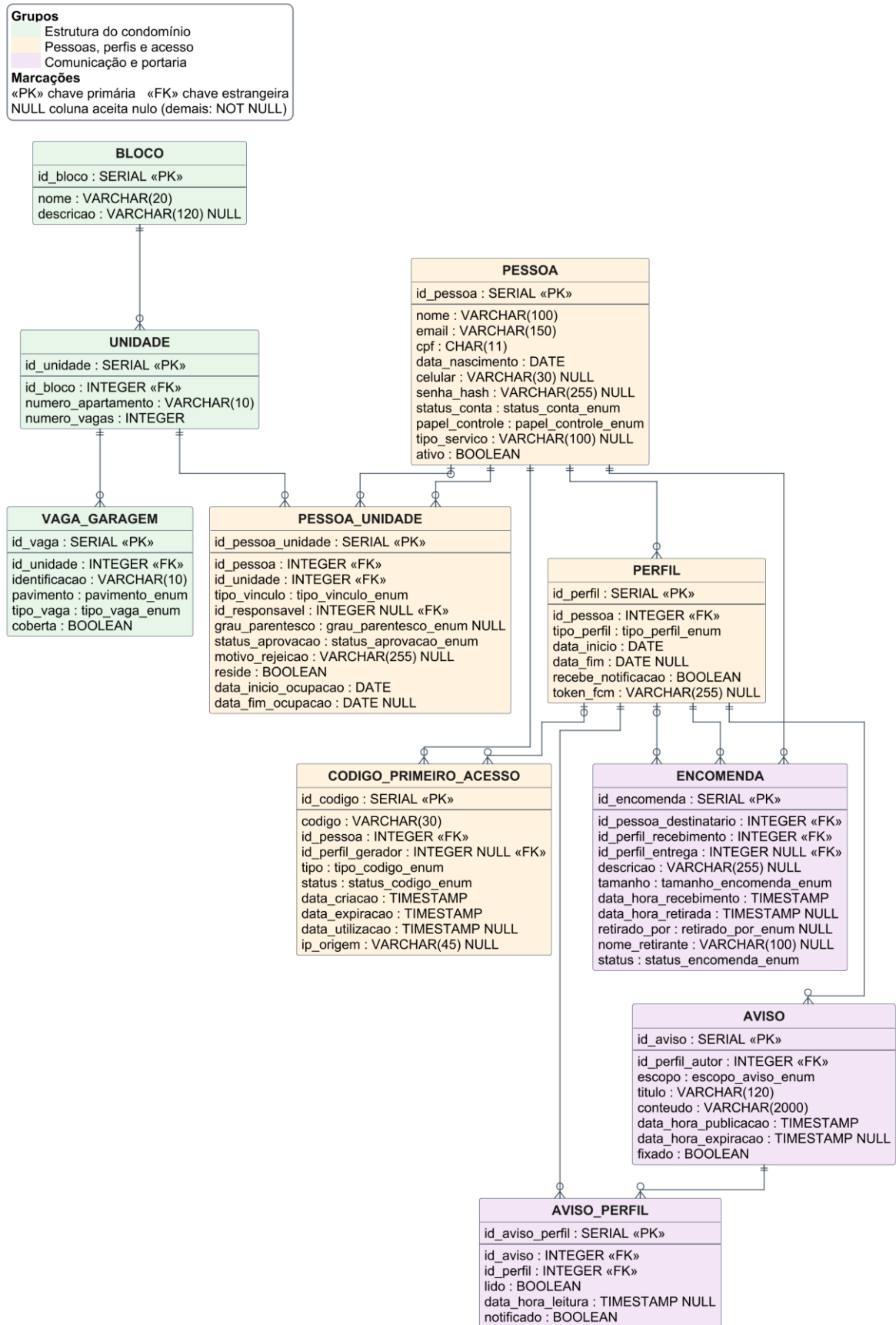


Fonte: Os autores.

4.5.2 Diagrama lógico

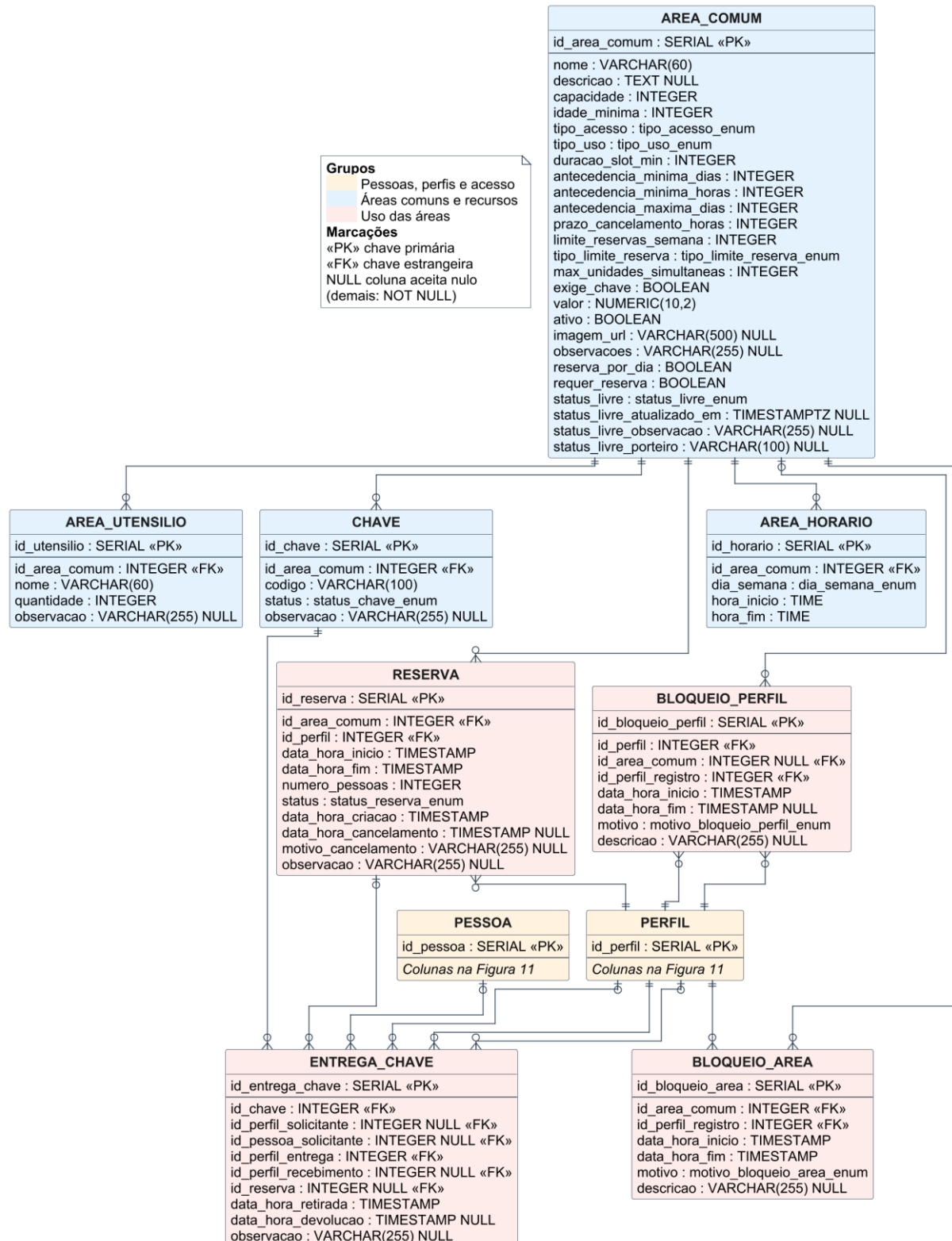
O Diagrama Lógico, apresentado nas Figuras 11 e 12, representa o mapeamento do modelo conceitual para o modelo relacional, com a identificação das chaves primárias, das chaves estrangeiras, dos tipos de dados e dos tamanhos de campo. As cores agrupam as tabelas por área funcional, conforme a legenda do próprio diagrama, e o símbolo em pé de galinha indica o lado de cardinalidade máxima n de cada relacionamento. A tabela CODIGO_PRIMEIRO_ACESSO integra o grupo de pessoas, perfis e vínculos. As colunas que aceitam valor nulo são marcadas com NULL; as demais são obrigatórias. Assim como no modelo de entidades, o diagrama foi dividido em duas figuras: a Figura 11 reúne as tabelas de estrutura, pessoas e comunicação, e a Figura 12 reúne as tabelas de áreas comuns e de uso das áreas, com PERFIL e PESSOA apresentadas apenas pela chave primária, para indicar os relacionamentos entre as duas partes.

Figura 11 – Diagrama lógico do software OASIS: estrutura, pessoas e comunicação



Fonte: Os autores.

Figura 12 – Diagrama lógico do software OASIS: áreas comuns e uso das áreas



Fonte: Os autores.

4.5.3 Domínios e tipos enumerados

Os atributos cujo domínio é um conjunto fechado e estável de valores são implementados como tipos enumerados nativos do *PostgreSQL*, o que garante a validação do domínio no próprio catálogo do banco. O Quadro 20 apresenta os 23 tipos definidos. Oito deles foram criados nesta versão: três para as funcionalidades novas (grau de parentesco, situação de aprovação e situação do código) e cinco para substituir atributos antes definidos como texto (situação da conta, papel da pessoa, tipo de limite de reserva, estado da área livre e tipo de código). O tipo de vínculo com a unidade recebeu os valores VISITANTE e PRESTADOR_SERVICO.

O valor ADMINISTRADOR de tipo_perfil_enum é tratado pela aplicação como sinônimo de SINDICO: ambos concedem o perfil de administração, e a equivalência é resolvida nos guardas de autorização do *backend*.

Quadro 20 – Tipos enumerados definidos no banco de dados

Tipo enumerado	Valores válidos	Utilizado em
tipo_perfil_enum	MORADOR, ADMINISTRADOR, SINDICO, PORTEIRO	PERFIL.tipo_perfil
tipo_vinculo_enum	PROPRIETARIO, INQUILINO, DEPENDENTE, VISITANTE, PRESTADOR_SERVICO	PESSOA_UNIDADE.tipo_vinculo
pavimento_enum	TERREO, SUBSOLO_1, SUBSOLO_2	VAGA_GARAGEM.pavimento
tipo_vaga_enum	SIMPLES, DUPLA	VAGA_GARAGEM.tipo_vaga
tipo_acesso_enum	LIVRE, BIOMETRIA, FACIAL, CHAVE	AREA_COMUM.tipo_acesso
tipo_uso_enum	COMUM, EXCLUSIVO, RESERVAVEL	AREA_COMUM.tipo_uso
dia_semana_enum	DOMINGO, SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO	AREA_HORARIO.dia_semana
status_reserva_enum	ATIVA, CANCELADA, CONCLUIDA	RESERVA.status
status_chave_enum	DISPONIVEL, EMPRESTADA, EXTRAVIADA	CHAVE.status
motivo_bloqueio_area_enum	MANUTENCAO, LIMPEZA, EVENTO, OBRA	BLOQUEIO_AREA.motivo
motivo_bloqueio_perfil_enum	INFRACAO, INADIMPLENCIA, SOLICITACAO, OUTRO	BLOQUEIO_PERFIL.motivo
tamanho_encomenda_enum	PEQUENO, MEDIO, GRANDE	ENCOMENDA.tamanho
status_encomenda_enum	AGUARDANDO_RETIRADA, RETIRADA, DEVOLVIDA	ENCOMENDA.status
retirado_por_enum	PROPRIO, TERCEIRO, PORTEIRO	ENCOMENDA.retirado_por
escopo_aviso_enum	MURAL, INDIVIDUAL	AVISO.escopo
grau_parentesco_enum	CONJUGE, FILHO, PAI_MAE, OUTRO	PESSOA_UNIDADE.grau_parentesco
status_aprovacao_enum	PENDENTE, APROVADO, REJEITADO	PESSOA_UNIDADE.status_aprovacao
status_codigo_enum	DISPONIVEL, USADO, EXPIRADO, CANCELADO	CODIGO_PRIMEIRO_ACESSO.status
status_conta_enum	AGUARDANDO_PRIMEIRO_ACESSO, ATIVO, BLOQUEADO	PESSOA.status_conta
tipo_limite_reserva_enum	DIARIO, SEMANAL, MENSAL	AREA_COMUM.tipo_limite_reserva
status_livre_enum	LIVRE, EM_USO	AREA_COMUM.status_livre
tipo_codigo_enum	PRIMEIRO_ACESSO, RECUPERACAO	CODIGO_PRIMEIRO_ACESSO.tipo
papel_controle_enum	MORADOR, SINDICO, PORTEIRO, VISITANTE, PRESTADOR_SERVICO	PESSOA.papel_controle

Fonte: Os autores.

4.5.4 Dicionário de dados

O Dicionário de Dados descreve todas as tabelas e atributos do banco de dados, apresentando para cada campo o tipo, o tamanho, a restrição de nulidade, o tipo de chave (PK, FK ou UK, esta última para unicidade), o domínio, a descrição e observações pertinentes. As tabelas são apresentadas em ordem alfabética nos Quadros 21 a 38.

O Quadro 21 apresenta o dicionário de dados da tabela AREA_COMUM, que armazena as áreas comuns e as regras de reserva configuráveis pela administração. Nesta versão recebeu os atributos de idade mínima, limite por período, reservas simultâneas, reserva por dia inteiro, imagem ilustrativa e o controle das áreas de uso livre.

Quadro 21 – Dicionário de dados: AREA_COMUM

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_area_comum	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da área comum	Gerado automaticamente
nome	VARCHAR	60	NÃO	UK	Texto	Nome da área comum	Não pode haver nomes repetidos
descricao	TEXT	-	SIM	-	Texto	Descrição detalhada da área	Opcional; sem limite de tamanho
capacidade	INTEGER	-	NÃO	-	Inteiro > 0	Capacidade máxima de pessoas	Limita numero pessoas da reserva
idade_minima	INTEGER	-	NÃO	-	Inteiro >= 0	Idade mínima para reservar a área	Padrão: 0; validado pelo RN16
tipo_acesso	ENUM	-	NÃO	-	tipo_acesso_enum	Forma de controle de acesso a área	-
tipo_uso	ENUM	-	NÃO	-	tipo_uso_enum	Modo de utilização da área	-
duracao_slot_min	INTEGER	-	NÃO	-	1 a 1440	Duração em minutos do intervalo reservável	Padrão: 60
antecedencia_minima_dias	INTEGER	-	NÃO	-	Inteiro >= 0	Antecedência mínima exigida para reservar	Padrão: 0
antecedencia_minima_horas	INTEGER	-	NÃO	-	Inteiro >= 0	Complemento em horas da antecedência mínima	Padrão: 0
antecedencia_maxima_dias	INTEGER	-	NÃO	-	Inteiro >= mínima	Antecedência máxima permitida	Padrão: 30
prazo_cancelamento_horas	INTEGER	-	NÃO	-	Inteiro >= 0	Prazo limite para cancelar a reserva	Padrão: 24
limite_reservas_semana	INTEGER	-	NÃO	-	Inteiro > 0	Quantidade máxima de reservas da unidade no período	Período definido em tipo_limite_reserva; RN07
tipo_limite_reserva	ENUM	-	NÃO	-	tipo_limite_reserva_enum	Período de contagem do limite por unidade	Padrão: SEMANAL
max_unidades_simultaneas	INTEGER	-	NÃO	-	Inteiro >= 1	Unidades que podem reservar o mesmo horário	Padrão: 1; validado pelo RN01
exige_chave	BOOLEAN	-	NÃO	-	true, false	Indica se o uso requer retirada de chave	Padrão: false
valor	NUMERIC	10,2	NÃO	-	Decimal >= 0	Valor cobrado pelo uso da área	Padrão: 0,00
ativo	BOOLEAN	-	NÃO	-	true, false	Indica se a área está em operação	Padrão: true
imagem_url	VARCHAR	500	SIM	-	Texto	Caminho da imagem ilustrativa da área	Imagem convertida para WebP no envio
observacoes	VARCHAR	255	SIM	-	Texto	Observações adicionais	Opcional
reserva_por_dia	BOOLEAN	-	NÃO	-	true, false	Indica reserva do dia inteiro em vez de intervalos	Padrão: false
requer_reserva	BOOLEAN	-	NÃO	-	true, false	Indica se a área exige reserva ou é de uso livre	Padrão: true
status_livre	ENUM	-	NÃO	-	status_livre_enum	Estado de ocupação das áreas de uso livre	Padrão: LIVRE; controlado pela portaria
status_livre_atualizado_em	TIMESTAMPTZ	-	SIM	-	Data e hora	Momento da última atualização do estado	Preenchido pela aplicação
status_livre_observacao	VARCHAR	255	SIM	-	Texto	Observação sobre o estado da área livre	Opcional

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
status_livre_porteiro	VARCHAR	100	SIM	-	Texto	Nome do funcionário que atualizou o estado	Opcional

Fonte: Os autores.

O Quadro 22 apresenta o dicionário de dados da tabela AREA_HORARIO, que define as janelas de funcionamento das áreas comuns por dia da semana.

Quadro 22 – Dicionário de dados: AREA_HORARIO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_horario	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do horário	Gerado automaticamente
id_area_comum	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Área comum a qual o horário pertence	AREA_COMUM; exclusão em cascata
dia_semana	ENUM	-	NÃO	UK	dia_semana_enum	Dia da semana da janela de funcionamento	Único com área e hora_inicio
hora_inicio	TIME	-	NÃO	UK	Hora	Horário de abertura da área no dia	-
hora_fim	TIME	-	NÃO	-	Hora	Horário de fechamento da área no dia	Deve ser maior que hora_inicio

Fonte: Os autores.

O Quadro 23 apresenta o dicionário de dados da tabela AREA_UTENSILIO, que registra os utensílios disponibilizados em cada área comum.

Quadro 23 – Dicionário de dados: AREA_UTENSILIO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_utensilio	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do utensílio	Gerado automaticamente
id_area_comum	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Área comum a qual o utensílio pertence	AREA_COMUM; exclusão em cascata
nome	VARCHAR	60	NÃO	UK	Texto	Nome do utensílio	Único dentro da área comum
quantidade	INTEGER	-	NAO	-	Inteiro > 0	Quantidade disponível	Padrão: 1
observacao	VARCHAR	255	SIM	-	Texto	Observações sobre o utensílio	Opcional

Fonte: Os autores.

O Quadro 24 apresenta o dicionário de dados da tabela AVISO, que armazena os comunicados do mural. O atributo escopo distingue os avisos da administração, destinados a todos os perfis, dos avisos individuais gerados automaticamente, como o de chegada de encomenda.

Quadro 24 – Dicionário de dados: AVISO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_aviso	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do aviso	Gerado automaticamente
id_perfil_autor	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil responsável pela publicação	PERFIL
escopo	ENUM	-	NÃO	-	escopo_aviso_enum	Alcance do aviso: mural ou individual	Padrão: MURAL
titulo	VARCHAR	120	NAO	-	Texto	Título do aviso	-
conteudo	VARCHAR	2000	NAO	-	Texto	Conteúdo do aviso	-
data_hora_publicacao	TIMESTAMP	-	NÃO	-	Data e hora	Momento da publicação	Padrão: data e hora atuais
data_hora_expiracao	TIMESTAMP	-	SIM	-	Data e hora	Momento em que o aviso deixa o mural	Nulo se não expira
fixado	BOOLEAN	-	NÃO	-	true, false	Indica destaque no topo do mural	Padrão: false

Fonte: Os autores.

O Quadro 25 apresenta o dicionário de dados da tabela AVISO_PERFIL, que registra individualmente a entrega e a leitura de cada aviso por perfil.

Quadro 25 – Dicionário de dados: AVISO_PERFIL

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_aviso_perfil	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do destinatário	Gerado automaticamente
id_aviso	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Aviso destinado ao perfil	AVISO; exclusão em cascata
id_perfil	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Perfil destinatário do aviso	PERFIL; único com id aviso
lido	BOOLEAN	-	NÃO	-	true, false	Indica se o aviso já foi visualizado	Padrão: false
data_hora_leitura	TIMESTAMP	-	SIM	-	Data e hora	Momento da leitura do aviso	Preenchido por gatilho
notificado	BOOLEAN	-	NÃO	-	true, false	Indica envio da notificação push	Padrão: false

Fonte: Os autores.

O Quadro 26 apresenta o dicionário de dados da tabela BLOCO, que representa a divisão física do condomínio.

Quadro 26 – Dicionário de dados: BLOCO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_bloco	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do bloco	Gerado automaticamente
nome	VARCHAR	20	NÃO	UK	Texto	Identificação do bloco	Exemplo: A, B, Torre Norte
descricao	VARCHAR	120	SIM	-	Texto	Descrição complementar do bloco	Opcional

Fonte: Os autores.

O Quadro 27 apresenta o dicionário de dados da tabela BLOQUEIO_AREA, que registra os períodos em que uma área comum fica indisponível a todos os moradores.

Quadro 27 – Dicionário de dados: BLOQUEIO_AREA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_bloqueio_area	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do bloqueio	Gerado automaticamente
id_area_comum	INTEGER	-	NÃO	FK	Inteiro > 0	Área comum interditada	AREA_COMUM
id_perfil_registro	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil que registrou o bloqueio	PERFIL
data_hora_inicio	TIMESTAMP	-	NÃO	-	Data e hora	Início do período de bloqueio	Permite bloqueio de meio período
data_hora_fim	TIMESTAMP	-	NÃO	-	Data e hora	Término do período de bloqueio	Deve ser maior que o início
motivo	ENUM	-	NÃO	-	motivo_bloqueio_area_enum	Motivo da interdição da área	-
descricao	VARCHAR	255	SIM	-	Texto	Detalhamento do motivo	Opcional

Fonte: Os autores.

O Quadro 28 apresenta o dicionário de dados da tabela BLOQUEIO_PERFIL, que registra as penalidades que restringem um morador específico em uma área ou em todas elas. Quando o campo id_area_comum é nulo, a restrição alcança todas as áreas.

Quadro 28 – Dicionário de dados: BLOQUEIO_PERFIL

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_bloqueio_perfil	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do bloqueio	Gerado automaticamente
id_perfil	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil que sofre a restrição	PERFIL
id_area_comum	INTEGER	-	SIM	FK	Inteiro > 0	Área comum alcançada pela restrição	Nulo indica todas as áreas
id_perfil_registro	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil que registrou o bloqueio	PERFIL
data_hora_inicio	TIMESTAMP	-	NÃO	-	Data e hora	Início da restrição	Padrão: data e hora atuais
data_hora_fim	TIMESTAMP	-	SIM	-	Data e hora	Término da restrição	Nulo indica prazo indeterminado
motivo	ENUM	-	NÃO	-	motivo_bloqueio_perfil_enum	Motivo da restrição ao morador	-
descricao	VARCHAR	255	SIM	-	Texto	Detalhamento do motivo	Opcional

Fonte: Os autores.

O Quadro 29 apresenta o dicionário de dados da tabela CHAVE, que armazena as chaves físicas das áreas comuns e sua situação.

Quadro 29 – Dicionário de dados: CHAVE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_chave	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da chave	Gerado automaticamente
id_area_comum	INTEGER	-	NÃO	FK	Inteiro > 0	Área comum a qual a chave pertence	AREA_COMUM
codigo	VARCHAR	100	NÃO	UK	Texto	Código físico gravado na chave	Exemplo: CH-SALAO-01
status	ENUM	-	NÃO	-	status_chave_enum	Situação atual da chave	Padrão: DISPONIVEL; atualizado por gatilho
observacao	VARCHAR	255	SIM	-	Texto	Observações sobre a chave	Opcional

Fonte: Os autores.

O Quadro 30 apresenta o dicionário de dados da tabela CODIGO_PRIMEIRO_ACESSO, que foi criada nesta versão em substituição ao convite de senha do *Firebase*. Ela armazena os códigos de ativação de conta, gerados pela administração, e os códigos de recuperação de senha, com validade, situação e registro de utilização.

Quadro 30 – Dicionário de dados: CODIGO_PRIMEIRO_ACESSO

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_codigo	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do código	Gerado automaticamente
codigo	VARCHAR	30	NÃO	UK	Texto	Código de ativação ou de recuperação	Único no sistema
id_pessoa	INTEGER	-	NÃO	FK	Inteiro > 0	Pessoa a quem o código pertence	PESSOA; exclusão em cascata
id_perfil_gerador	INTEGER	-	SIM	FK	Inteiro > 0	Perfil da administração que gerou o código	Nulo na recuperação de senha
tipo	ENUM	-	NÃO	-	tipo_codigo_enum	Finalidade do código	Padrão: PRIMEIRO_ACESSO
status	ENUM	-	NÃO	-	status_codigo_enum	Situação do código	Padrão: DISPONIVEL
data_criacao	TIMESTAMP	-	NÃO	-	Data e hora	Momento da geração do código	Padrão: data e hora atuais
data_expiracao	TIMESTAMP	-	NÃO	-	Data e hora	Fim da validade do código	Padrão: 7 dias; recuperação: 2 horas
data_utilizacao	TIMESTAMP	-	SIM	-	Data e hora	Momento em que o código foi utilizado	Nulo enquanto não utilizado
ip_origem	VARCHAR	45	SIM	-	Texto	Endereço IP de onde o código foi utilizado	Registro para auditoria

Fonte: Os autores.

O Quadro 31 apresenta o dicionário de dados da tabela ENCOMENDA, que registra o recebimento e a retirada de encomendas na portaria, com os funcionários responsáveis.

Quadro 31 – Dicionário de dados: ENCOMENDA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_encomenda	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da encomenda	Gerado automaticamente
id_pessoa_destinatario	INTEGER	-	NÃO	FK	Inteiro > 0	Pessoa destinatária da encomenda	PESSOA
id_perfil_recebimento	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil do porteiro que recebeu	PERFIL
id_perfil_entrega	INTEGER	-	SIM	FK	Inteiro > 0	Perfil do porteiro que entregou	Nulo até a retirada
descricao	VARCHAR	255	SIM	-	Texto	Descrição da encomenda	Opcional
tamanho	ENUM	-	NÃO	-	tamanho_encomenda_enum	Porte da encomenda	-
data_hora_recebimento	TIMESTAMP	-	NÃO	-	Data e hora	Momento do recebimento na portaria	Padrão: data e hora atuais
data_hora_retirada	TIMESTAMP	-	SIM	-	Data e hora	Momento da retirada pelo morador	Nulo até a retirada
retirado_por	ENUM	-	SIM	-	retirado_por_enum	Vínculo de quem retirou a encomenda	Obrigatório ao registrar retirada
nome_retirante	VARCHAR	100	SIM	-	Texto	Nome de quem retirou a encomenda	Obrigatório quando TERCEIRO
status	ENUM	-	NÃO	-	status_encomenda_enum	Situação atual da encomenda	Padrão: AGUARDANDO_RETIRADA

Fonte: Os autores.

O Quadro 32 apresenta o dicionário de dados da tabela ENTREGA_CHAVE, que registra o empréstimo e a devolução das chaves, mantendo o histórico completo das operações. O solicitante é o perfil de um morador ou, no caso de visitantes e prestadores de serviço, a própria pessoa, sendo obrigatório um dos dois.

Quadro 32 – Dicionário de dados: ENTREGA_CHAVE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_entrega_chave	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da entrega	Gerado automaticamente
id_chave	INTEGER	-	NAO	FK	Inteiro > 0	Chave emprestada	CHAVE
id_perfil_solicitante	INTEGER	-	SIM	FK	Inteiro > 0	Perfil do morador que retirou a chave	PERFIL; nulo quando o solicitante é visitante ou prestador
id_pessoa_solicitante	INTEGER	-	SIM	FK	Inteiro > 0	Visitante ou prestador que retirou a chave	PESSOA; um dos dois solicitantes é obrigatório
id_perfil_entrega	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil do porteiro que entregou	PERFIL
id_perfil_recebimento	INTEGER	-	SIM	FK	Inteiro > 0	Perfil do porteiro que recebeu de volta	Nulo até a devolução
id_reserva	INTEGER	-	SIM	FK	Inteiro > 0	Reserva que originou o empréstimo	Nulo se não houver reserva
data_hora_retirada	TIMESTAMP	-	NÃO	-	Data e hora	Momento da retirada da chave	Padrão: data e hora atuais
data_hora_devolucao	TIMESTAMP	-	SIM	-	Data e hora	Momento da devolução da chave	Nulo enquanto não devolvida
observacao	VARCHAR	255	SIM	-	Texto	Observações sobre a entrega	Opcional

Fonte: Os autores.

O Quadro 33 apresenta o dicionário de dados da tabela PERFIL, que define o papel de cada pessoa no condomínio e representa o usuário efetivo do sistema. Uma pessoa pode ter mais de um perfil vigente.

Quadro 33 – Dicionário de dados: PERFIL

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_perfil	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do perfil	Gerado automaticamente
id_pessoa	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Pessoa associada ao perfil	PESSOA
tipo_perfil	ENUM	-	NÃO	UK	tipo_perfil_enum	Papel exercido no condomínio	-
data_inicio	DATE	-	NÃO	UK	Data	Data em que a pessoa assumiu a função	Padrão: data atual
data_fim	DATE	-	SIM	-	Data	Data em que a pessoa deixou a função	Nulo enquanto o perfil está vigente
recebe_notificacao	BOOLEAN	-	NÃO	-	true, false	Indica se o perfil recebe notificações	Padrão: true
token_fcm	VARCHAR	255	SIM	-	Texto	Token do dispositivo para push	Fornecido pelo Firebase

Fonte: Os autores.

O Quadro 34 apresenta o dicionário de dados da tabela PESSOA, que armazena os dados das pessoas cadastradas — moradores, funcionários, visitantes e prestadores de serviço. Nesta versão deixou de guardar o identificador do *Firebase* e passou a guardar o resumo criptográfico da senha, a situação da conta, o papel da pessoa no condomínio e o serviço prestado.

Quadro 34 – Dicionário de dados: PESSOA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_pessoa	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da pessoa	Gerado automaticamente
nome	VARCHAR	100	NÃO	-	Texto	Nome completo da pessoa	-
email	VARCHAR	150	NÃO	UK	Texto no formato de e-mail	Endereço de e-mail usado no login	-
cpf	CHAR	11	NÃO	UK	Onze dígitos numéricos	CPF da pessoa, somente números	Validado por expressão regular
data_nascimento	DATE	-	NÃO	-	Data anterior a atual	Data de nascimento da pessoa	-
celular	VARCHAR	30	SIM	-	Texto	Telefone celular de contato	Opcional
senha_hash	VARCHAR	255	SIM	-	Texto	Resumo criptográfico da senha (PBKDF2-SHA512 com salt)	Nulo até o primeiro acesso
status_conta	ENUM	-	NÃO	-	status_conta_enum	Situação da conta de acesso	Padrão: ATIVO; cadastro pela administração inicia em AGUARDANDO PRIMEIRO ACESSO
papel_controle	ENUM	-	NÃO	-	papel_controle_enum	Papel principal da pessoa no condomínio	Padrão: MORADOR; VISITANTE e PRESTADOR SERVIÇO não acessam o sistema
tipo_servico	VARCHAR	100	SIM	-	Texto	Descrição do serviço prestado	Apenas para prestadores de serviço
ativo	BOOLEAN	-	NÃO	-	true, false	Indica se o cadastro está ativo	Padrão: true

Fonte: Os autores.

O Quadro 35 apresenta o dicionário de dados da tabela PESSOA_UNIDADE, que associa pessoas a unidades e mantém o histórico dos vínculos. Nesta versão passou a registrar o titular responsável, o grau de parentesco e a situação de aprovação dos dependentes.

Quadro 35 – Dicionário de dados: PESSOA_UNIDADE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_pessoa_unidade	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único do vínculo	Gerado automaticamente
id_pessoa	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Pessoa vinculada a unidade	PESSOA
id_unidade	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Unidade vinculada a pessoa	UNIDADE
tipo_vinculo	ENUM	-	NÃO	-	tipo_vinculo_enum	Natureza do vínculo da pessoa	-
id_responsavel	INTEGER	-	SIM	FK	Inteiro > 0	Titular responsável pelo dependente	PESSOA; nulo para titulares; validado pelo RN15
grau_parentesco	ENUM	-	SIM	-	grau_parentesco_enum	Parentesco do dependente com o titular	Nulo para titulares
status_aprovacao	ENUM	-	NÃO	-	status_aprovacao_enum	Situação da aprovação do vínculo	Dependentes iniciam como PENDENTE
motivo_rejeicao	VARCHAR	255	SIM	-	Texto	Justificativa da rejeição do cadastro	Preenchido apenas quando REJEITADO
reside	BOOLEAN	-	NÃO	-	true, false	Indica se a pessoa reside na unidade	Padrão: true
data_inicio_ocupacao	DATE	-	NÃO	UK	Data	Início do vínculo com a unidade	Padrão: data atual
data_fim_ocupacao	DATE	-	SIM	-	Data	Término do vínculo com a unidade	Nulo enquanto o vínculo vigora

Fonte: Os autores.

O Quadro 36 apresenta o dicionário de dados da tabela RESERVA, que armazena as reservas de áreas comuns, delimitadas por intervalo de data e hora.

Quadro 36 – Dicionário de dados: RESERVA

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_reserva	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da reserva	Gerado automaticamente
id_area_comum	INTEGER	-	NÃO	FK	Inteiro > 0	Área comum reservada	AREA_COMUM
id_perfil	INTEGER	-	NÃO	FK	Inteiro > 0	Perfil do morador solicitante	PERFIL
data_hora_inicio	TIMESTAMP	-	NÃO	-	Data e hora	Início do período reservado	-
data_hora_fim	TIMESTAMP	-	NÃO	-	Data e hora	Término do período reservado	Mesma data do início e maior que ele
numero_pessoas	INTEGER	-	NÃO	-	Inteiro > 0	Quantidade de pessoas na reserva	Não pode exceder a capacidade
status	ENUM	-	NÃO	-	status_reserva_enum	Situação atual da reserva	Padrão: ATIVA
data_hora_criacao	TIMESTAMP	-	NÃO	-	Data e hora	Momento do registro da reserva	Padrão: data e hora atuais
data_hora_cancelamento	TIMESTAMP	-	SIM	-	Data e hora	Momento do cancelamento	Preenchido por gatilho
motivo_cancelamento	VARCHAR	255	SIM	-	Texto	Justificativa do cancelamento	Opcional
observacao	VARCHAR	255	SIM	-	Texto	Observações da reserva	Opcional

Fonte: Os autores.

O Quadro 37 apresenta o dicionário de dados da tabela UNIDADE, que armazena os apartamentos do condomínio.

Quadro 37 – Dicionário de dados: UNIDADE

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_unidade	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da unidade	Gerado automaticamente
id_bloco	INTEGER	-	NÃO	FK, UK	Inteiro > 0	Bloco ao qual a unidade pertence	BLOCO
numero_apartamento	VARCHAR	10	NÃO	UK	Texto	Número ou identificação do apartamento	Único dentro do bloco
numero_vagas	INTEGER	-	NÃO	-	Inteiro >= 0	Quantidade de vagas da unidade	Padrão: 0

Fonte: Os autores.

O Quadro 38 apresenta o dicionário de dados da tabela VAGA_GARAGEM, que armazena as vagas de estacionamento vinculadas às unidades.

Quadro 38 – Dicionário de dados: VAGA_GARAGEM

Campo	Tipo	Tam.	Nulo	Chave	Domínio	Descrição	Observação
id_vaga	SERIAL	-	NÃO	PK	Inteiro > 0	Identificador único da vaga	Gerado automaticamente
id_unidade	INTEGER	-	NÃO	FK	Inteiro > 0	Unidade proprietária da vaga	UNIDADE
identificacao	VARCHAR	10	NÃO	UK	Texto	Número de identificação da vaga	Exemplo: G-014
pavimento	ENUM	-	NÃO	-	pavimento_enum	Pavimento onde fica a vaga	-
tipo_vaga	ENUM	-	NÃO	-	tipo_vaga_enum	Dimensão da vaga de garagem	-
coberta	BOOLEAN	-	NÃO	-	true, false	Indica se a vaga é coberta	Padrão: false

Fonte: Os autores.

4.5.5 Scripts do banco de dados

Os *scripts* de criação do banco foram desenvolvidos para o *PostgreSQL* 16 e seguem integralmente o Dicionário de Dados. Eles são completos e reprodutíveis a partir de uma instalação limpa do SGBD e constituem a fonte única do esquema: não dependem de nenhuma alteração posterior feita pela aplicação. Os *scripts* estão divididos em quatro arquivos, executados em ordem: criação do banco e dos tipos enumerados (Quadro 39), criação das tabelas e restrições (Quadro 40), índices e carga inicial (Quadro 41) e gatilhos (Quadro 44, apresentado na seção 4.5.6).

Os *scripts* foram executados em uma instalação limpa do *PostgreSQL* 16, resultando nas 18 tabelas, nos 23 tipos enumerados e nos 12 gatilhos descritos neste documento.

Quadro 39 – Script de criação do banco de dados e dos tipos enumerados (PostgreSQL 16)

```
-- =====
-- OASIS - Sistema de gestao de reservas e controle de acesso
-- Script de criacao do banco de dados - PostgreSQL 16
-- Parte 1 de 3: criacao do banco e dos tipos enumerados
-- =====

DROP DATABASE IF EXISTS oasis;

-- A codificacao e o locale sao herdados do cluster, o que torna o script
-- portavel entre Windows e Linux (os nomes de locale diferem entre eles).
CREATE DATABASE oasis;

COMMENT ON DATABASE oasis IS 'Base de dados do sistema OASIS - UEPG 2026';

\c oasis

-- -----
-- Tipos enumerados (dominios fechados)
-- -----
CREATE TYPE tipo_perfil_enum AS ENUM ('MORADOR', 'ADMINISTRADOR', 'SINDICO', 'PORTEIRO');
CREATE TYPE tipo_vinculo_enum AS ENUM ('PROPRIETARIO', 'INQUILINO', 'DEPENDENTE', 'VISITANTE', 'PRESTADOR_SERVICO');
CREATE TYPE pavimento_enum AS ENUM ('TERREO', 'SUBSOLO_1', 'SUBSOLO_2');
CREATE TYPE tipo_vaga_enum AS ENUM ('SIMPLES', 'DUPLA');
CREATE TYPE tipo_acesso_enum AS ENUM ('LIVRE', 'BIOMETRIA', 'FACIAL', 'CHAVE');
CREATE TYPE tipo_uso_enum AS ENUM ('COMUM', 'EXCLUSIVO', 'RESERVAVEL');
CREATE TYPE dia_semana_enum AS ENUM ('DOMINGO', 'SEGUNDA', 'TERCA', 'QUARTA', 'QUINTA', 'SEXTA', 'SABADO');

CREATE TYPE status_reserva_enum AS ENUM ('ATIVA', 'CANCELADA', 'CONCLUIDA');
CREATE TYPE status_chave_enum AS ENUM ('DISPONIVEL', 'EMPRESTADA', 'EXTRAVIADA');
CREATE TYPE motivo_bloqueio_area_enum AS ENUM ('MANUTENCAO', 'LIMPEZA', 'EVENTO', 'OBRA');
CREATE TYPE motivo_bloqueio_perfil_enum AS ENUM ('INFRACAO', 'INADIMPLENCIA', 'SOLICITACAO', 'OUTRO');
CREATE TYPE tamanho_encomenda_enum AS ENUM ('PEQUENO', 'MEDIO', 'GRANDE');
CREATE TYPE status_encomenda_enum AS ENUM ('AGUARDANDO_RETIRADA', 'RETIRADA', 'DEVOLVIDA');
CREATE TYPE retirado_por_enum AS ENUM ('PROPRIO', 'TERCEIRO', 'PORTEIRO');
CREATE TYPE escopo_aviso_enum AS ENUM ('MURAL', 'INDIVIDUAL');
CREATE TYPE grau_pareteesco_enum AS ENUM ('CONJUGE', 'FILHO', 'PAI_MAE', 'OUTRO');
CREATE TYPE status_aprovacao_enum AS ENUM ('PENDENTE', 'APROVADO', 'REJEITADO');
CREATE TYPE status_codigo_enum AS ENUM ('DISPONIVEL', 'USADO', 'EXPIRADO', 'CANCELADO');

-- Tipos criados na consolidacao do schema (antes eram VARCHAR com CHECK)
CREATE TYPE status_conta_enum AS ENUM ('AGUARDANDO_PRIMEIRO_ACESSO', 'ATIVO', 'BLOQUEADO');
CREATE TYPE tipo_limite_reserva_enum AS ENUM ('DIARIO', 'SEMANAL', 'MENSAL');
CREATE TYPE status_livre_enum AS ENUM ('LIVRE', 'EM_USO');
CREATE TYPE tipo_codigo_enum AS ENUM ('PRIMEIRO_ACESSO', 'RECUPERACAO');
CREATE TYPE papel_contrôle_enum AS ENUM ('MORADOR', 'SINDICO', 'PORTEIRO', 'VISITANTE', 'PRESTADOR_SERVICO');

-- Observacao: em tipo_perfil_enum, o valor ADMINISTRADOR e tratado pela
-- aplicacao como sinonimo de SINDICO (equivalencia resolvida em
-- backend/src/auth/guards.ts). Ambos concedem o perfil de administracao.
```

Fonte: Os autores. PostgreSQL 16.

Quadro 40 – Script de criação das tabelas e restrições de integridade (PostgreSQL 16)

```
-- =====
-- Parte 2 de 3: criacao das tabelas
-- As tabelas sao criadas na ordem de dependencia das chaves estrangeiras
-- =====
```

```

CREATE TABLE bloco (
    id_bloco SERIAL PRIMARY KEY,
    nome VARCHAR(20) NOT NULL UNIQUE,
    descricao VARCHAR(120)
);

CREATE TABLE unidade (
    id_unidade SERIAL PRIMARY KEY,
    id_bloco INTEGER NOT NULL,
    numero_apartamento VARCHAR(10) NOT NULL,
    numero_vagas INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT fk_unidade_bloco FOREIGN KEY (id_bloco) REFERENCES bloco (id_bloco),
    CONSTRAINT uk_unidade_ap UNIQUE (id_bloco, numero_apartamento),
    CONSTRAINT ck_unidade_vagas CHECK (numero_vagas >= 0)
);

CREATE TABLE vaga_garagem (
    id_vaga SERIAL PRIMARY KEY,
    id_unidade INTEGER NOT NULL,
    identificacao VARCHAR(10) NOT NULL UNIQUE,
    pavimento pavimento_enum NOT NULL,
    tipo_vaga tipo_vaga_enum NOT NULL,
    coberta BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_vaga_unidade FOREIGN KEY (id_unidade) REFERENCES unidade (id_unidade)
);

CREATE TABLE pessoa (
    id_pessoa SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    cpf CHAR(11) NOT NULL UNIQUE,
    data_nascimento DATE NOT NULL,
    celular VARCHAR(30),
    senha_hash VARCHAR(255),
    status_conta status_conta_enum NOT NULL DEFAULT 'ATIVO',
    papel_controle papel_controle_enum NOT NULL DEFAULT 'MORADOR',
    tipo_servico VARCHAR(100),
    ativo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT ck_pessoa_cpf CHECK (cpf ~ '^[0-9]{11}$'),
    CONSTRAINT ck_pessoa_email CHECK (email LIKE '%@%.%'),
    CONSTRAINT ck_pessoa_nascimento CHECK (data_nascimento < CURRENT_DATE)
);

CREATE TABLE pessoa_unidade (
    id_pessoa_unidade SERIAL PRIMARY KEY,
    id_pessoa INTEGER NOT NULL,
    id_unidade INTEGER NOT NULL,
    tipo_vinculo tipo_vinculo_enum NOT NULL,
    id_responsavel INTEGER,
    grau_parentesco grau_parentesco_enum,
    status_aprovacao status_aprovacao_enum NOT NULL DEFAULT 'APROVADO',
    motivo_rejeicao VARCHAR(255),
    reside BOOLEAN NOT NULL DEFAULT TRUE,
    data_inicio_ocupacao DATE NOT NULL DEFAULT CURRENT_DATE,
    data_fim_ocupacao DATE,
    CONSTRAINT fk_pu_pessoa FOREIGN KEY (id_pessoa) REFERENCES pessoa (id_pessoa),
    CONSTRAINT fk_pu_unidade FOREIGN KEY (id_unidade) REFERENCES unidade (id_unidade),
    CONSTRAINT fk_pu_responsavel FOREIGN KEY (id_responsavel) REFERENCES pessoa (id_pessoa) ON DELETE RESTRICT,
    CONSTRAINT uk_pu_vinculo UNIQUE (id_pessoa, id_unidade, data_inicio_ocupacao),
    CONSTRAINT ck_pu_periodo CHECK (data_fim_ocupacao IS NULL OR data_fim_ocupacao >= data_inicio_ocupacao),
    CONSTRAINT ck_pu_dependente CHECK (
        (tipo_vinculo = 'DEPENDENTE' AND id_responsavel IS NOT NULL AND id_responsavel <> id_pessoa)
        OR
        (tipo_vinculo IN ('PROPRIETARIO', 'INQUILINO') AND id_responsavel IS NULL)
    )
);

CREATE UNIQUE INDEX uk_unidade_proprietario_ativo
ON pessoa_unidade (id_unidade)
WHERE tipo_vinculo = 'PROPRIETARIO' AND data_fim_ocupacao IS NULL;

CREATE UNIQUE INDEX uk_unidade_inquilino_ativo
ON pessoa_unidade (id_unidade)
WHERE tipo_vinculo = 'INQUILINO' AND data_fim_ocupacao IS NULL;

CREATE INDEX idx_pu_responsavel ON pessoa_unidade (id_responsavel);

CREATE TABLE perfil (
    id_perfil SERIAL PRIMARY KEY,
    id_pessoa INTEGER NOT NULL,
    tipo_perfil tipo_perfil_enum NOT NULL,
    data_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
    data_fim DATE,
    recebe_notificacao BOOLEAN NOT NULL DEFAULT TRUE,
    token_fcm VARCHAR(255),
    CONSTRAINT fk_perfil_pessoa FOREIGN KEY (id_pessoa) REFERENCES pessoa (id_pessoa),
    CONSTRAINT uk_perfil UNIQUE (id_pessoa, tipo_perfil, data_inicio),
    CONSTRAINT ck_perfil_data CHECK (data_fim IS NULL OR data_fim >= data_inicio)
);

CREATE TABLE area_comum (
    id_area_comum SERIAL PRIMARY KEY,
    nome VARCHAR(60) NOT NULL UNIQUE,
    descricao TEXT,
    capacidade INTEGER NOT NULL,
    idade_minima INTEGER NOT NULL DEFAULT 0,
    tipo_acesso tipo_acesso_enum NOT NULL,
    tipo_uso tipo_uso_enum NOT NULL,
    duracao_slot_min INTEGER NOT NULL DEFAULT 60,
    antecedencia_minima_dias INTEGER NOT NULL DEFAULT 0,
    antecedencia_minima_horas INTEGER NOT NULL DEFAULT 0,
    antecedencia_maxima_dias INTEGER NOT NULL DEFAULT 30,
    prazo_cancelamento_horas INTEGER NOT NULL DEFAULT 24,
    limite_reservas_semana INTEGER NOT NULL DEFAULT 2,
    tipo_limite_reserva tipo_limite_reserva_enum NOT NULL DEFAULT 'SEMANAL',
    max_unidades_simultaneas INTEGER NOT NULL DEFAULT 1,
    exige_chave BOOLEAN NOT NULL DEFAULT FALSE,

```

```

        valor                NUMERIC(10,2)        NOT NULL DEFAULT 0,
        ativo                BOOLEAN              NOT NULL DEFAULT TRUE,
        imagem_url           VARCHAR(500),
        observacoes          VARCHAR(255),
        reserva_por_dia      BOOLEAN              NOT NULL DEFAULT FALSE,
        requer_reserva       BOOLEAN              NOT NULL DEFAULT TRUE,
        status_livre          status_livre_enum    NOT NULL DEFAULT 'LIVRE',
        status_livre_atualizado_em timestamp       DEFAULT CURRENT_TIMESTAMP,
        status_livre_observacao VARCHAR(255),
        status_livre_porteiro VARCHAR(100),
        CONSTRAINT ck_area_capacidade CHECK (capacidade > 0),
        CONSTRAINT ck_area_idade CHECK (idade_minima >= 0),
        CONSTRAINT ck_area_slot CHECK (duracao_slot_min BETWEEN 1 AND 1440),
        CONSTRAINT ck_area_antec CHECK (antecedencia_maxima_dias >= antecedencia_minima_dias
        AND antecedencia_minima_dias >= 0),
        CONSTRAINT ck_area_prazo CHECK (prazo_cancelamento_horas >= 0),
        CONSTRAINT ck_area_limite CHECK (limite_reservas_semana > 0),
        CONSTRAINT ck_area_max_simult CHECK (max_unidades_simultaneas >= 1),
        CONSTRAINT ck_area_valor CHECK (valor >= 0)
    );

CREATE TABLE area_horario (
    id_horario SERIAL PRIMARY KEY,
    id_area_comum INTEGER NOT NULL,
    dia_semana dia_semana_enum NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fim TIME NOT NULL,
    CONSTRAINT fk_horario_area FOREIGN KEY (id_area_comum)
    REFERENCES area_comum (id_area_comum) ON DELETE CASCADE,
    CONSTRAINT uk_horario UNIQUE (id_area_comum, dia_semana, hora_inicio),
    CONSTRAINT ck_horario CHECK (hora_fim > hora_inicio)
);

CREATE TABLE area_utilisio (
    id_utilisio SERIAL PRIMARY KEY,
    id_area_comum INTEGER NOT NULL,
    nome VARCHAR(60) NOT NULL,
    quantidade INTEGER NOT NULL DEFAULT 1,
    observacao VARCHAR(255),
    CONSTRAINT fk_utilisio_area FOREIGN KEY (id_area_comum)
    REFERENCES area_comum (id_area_comum) ON DELETE CASCADE,
    CONSTRAINT uk_utilisio UNIQUE (id_area_comum, nome),
    CONSTRAINT ck_utilisio_qtd CHECK (quantidade > 0)
);

CREATE TABLE chave (
    id_chave SERIAL PRIMARY KEY,
    id_area_comum INTEGER NOT NULL,
    codigo VARCHAR(100) NOT NULL UNIQUE,
    status status_chave_enum NOT NULL DEFAULT 'DISPONIVEL',
    observacao VARCHAR(255),
    CONSTRAINT fk_chave_area FOREIGN KEY (id_area_comum) REFERENCES area_comum (id_area_comum)
);

CREATE TABLE reserva (
    id_reserva SERIAL PRIMARY KEY,
    id_area_comum INTEGER NOT NULL,
    id_perfil INTEGER NOT NULL,
    data_hora_inicio TIMESTAMP NOT NULL,
    data_hora_fim TIMESTAMP NOT NULL,
    numero_pessoas INTEGER NOT NULL,
    status status_reserva_enum NOT NULL DEFAULT 'ATIVA',
    data_hora_criacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_hora_cancelamento TIMESTAMP,
    motivo_cancelamento VARCHAR(255),
    observacao VARCHAR(255),
    CONSTRAINT fk_reserva_area FOREIGN KEY (id_area_comum) REFERENCES area_comum (id_area_comum),
    CONSTRAINT fk_reserva_perfil FOREIGN KEY (id_perfil) REFERENCES perfil (id_perfil),
    CONSTRAINT ck_reserva_periodo CHECK (data_hora_fim > data_hora_inicio),
    CONSTRAINT ck_reserva_mesmodia CHECK (data_hora_fim::date = data_hora_inicio::date),
    CONSTRAINT ck_reserva_pessoas CHECK (numero_pessoas > 0),
    CONSTRAINT ck_reserva_cancel CHECK ((status = 'CANCELADA' AND data_hora_cancelamento IS NOT NULL)
    OR (status <> 'CANCELADA' AND data_hora_cancelamento IS NULL))
);

CREATE TABLE entrega_chave (
    id_entrega_chave SERIAL PRIMARY KEY,
    id_chave INTEGER NOT NULL,
    id_perfil_solicitante INTEGER,
    id_pessoa_solicitante INTEGER,
    id_perfil_entrega INTEGER NOT NULL,
    id_perfil_recebimento INTEGER,
    id_reserva INTEGER,
    data_hora_retirada TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_hora_devolucao TIMESTAMP,
    observacao VARCHAR(255),
    CONSTRAINT fk_ec_chave FOREIGN KEY (id_chave) REFERENCES chave (id_chave) ON
    DELETE CASCADE,
    CONSTRAINT fk_ec_solic FOREIGN KEY (id_perfil_solicitante) REFERENCES perfil (id_perfil),
    CONSTRAINT fk_ec_pessoa FOREIGN KEY (id_pessoa_solicitante) REFERENCES pessoa (id_pessoa),
    CONSTRAINT ck_ec_solicitante CHECK (id_perfil_solicitante IS NOT NULL OR id_pessoa_solicitante
    IS NOT NULL),
    CONSTRAINT fk_ec_entrega FOREIGN KEY (id_perfil_entrega) REFERENCES perfil (id_perfil),
    CONSTRAINT fk_ec_receb FOREIGN KEY (id_perfil_recebimento) REFERENCES perfil (id_perfil),
    CONSTRAINT fk_ec_reserva FOREIGN KEY (id_reserva) REFERENCES reserva (id_reserva),
    CONSTRAINT ck_ec_devolucao CHECK (data_hora_devolucao IS NULL
    OR data_hora_devolucao >= data_hora_retirada)
);

CREATE TABLE bloqueio_area (
    id_bloqueio_area SERIAL PRIMARY KEY,
    id_area_comum INTEGER NOT NULL,
    id_perfil_registro INTEGER NOT NULL,
    data_hora_inicio TIMESTAMP NOT NULL,
    data_hora_fim TIMESTAMP NOT NULL,
    motivo motivo_bloqueio_area_enum NOT NULL,
    descricao VARCHAR(255),
    CONSTRAINT fk_ba_area FOREIGN KEY (id_area_comum) REFERENCES area_comum (id_area_comum),
    CONSTRAINT fk_ba_perfil FOREIGN KEY (id_perfil_registro) REFERENCES perfil (id_perfil),
    CONSTRAINT ck_ba_periodo CHECK (data_hora_fim > data_hora_inicio)
);

```

```

);

CREATE TABLE bloqueio_perfil (
    id_bloqueio_perfil SERIAL PRIMARY KEY,
    id_perfil INTEGER NOT NULL,
    id_area_comum INTEGER,
    id_perfil_registro INTEGER NOT NULL,
    data_hora_inicio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_hora_fim TIMESTAMP,
    motivo VARCHAR(255),
    descricao VARCHAR(255),
    CONSTRAINT fk_bp_perfil FOREIGN KEY (id_perfil) REFERENCES perfil (id_perfil),
    CONSTRAINT fk_bp_area FOREIGN KEY (id_area_comum) REFERENCES area_comum (id_area_comum),
    CONSTRAINT fk_bp_registro FOREIGN KEY (id_perfil_registro) REFERENCES perfil (id_perfil),
    CONSTRAINT ck_bp_periodo CHECK (data_hora_fim IS NULL OR data_hora_fim > data_hora_inicio)
);

CREATE TABLE encomenda (
    id_encomenda SERIAL PRIMARY KEY,
    id_pessoa_destinatario INTEGER NOT NULL,
    id_perfil_recebimento INTEGER NOT NULL,
    id_perfil_entrega INTEGER,
    descricao VARCHAR(255),
    tamanho tamanho_encomenda_enum NOT NULL,
    data_hora_recebimento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_hora_retirada TIMESTAMP,
    retirado_por retirado_por_enum,
    nome_retirante VARCHAR(100),
    status status_encomenda_enum NOT NULL DEFAULT 'AGUARDANDO RETIRADA',
    CONSTRAINT fk_enc_pessoa FOREIGN KEY (id_pessoa_destinatario) REFERENCES pessoa (id_pessoa),
    CONSTRAINT fk_enc_receb FOREIGN KEY (id_perfil_recebimento) REFERENCES perfil (id_perfil),
    CONSTRAINT fk_enc_entrega FOREIGN KEY (id_perfil_entrega) REFERENCES perfil (id_perfil),
    CONSTRAINT ck_enc_retirada CHECK ((data_hora_retirada IS NULL AND retirado_por IS NULL)
    OR (data_hora_retirada IS NOT NULL AND retirado_por IS NOT NULL)),
    CONSTRAINT ck_enc_terceiro CHECK (retirado_por <> 'TERCEIRO' OR nome_retirante IS NOT NULL),
    CONSTRAINT ck_enc_ordem CHECK (data_hora_retirada IS NULL
    OR data_hora_retirada >= data_hora_recebimento)
);

CREATE TABLE aviso (
    id_aviso SERIAL PRIMARY KEY,
    id_perfil_autor INTEGER NOT NULL,
    escopo escopo_aviso_enum NOT NULL DEFAULT 'MURAL',
    titulo VARCHAR(120) NOT NULL,
    conteudo VARCHAR(2000) NOT NULL,
    data_hora_publicacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_hora_expiracao TIMESTAMP,
    fixado BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_aviso_autor FOREIGN KEY (id_perfil_autor) REFERENCES perfil (id_perfil),
    CONSTRAINT ck_aviso_exp CHECK (data_hora_expiracao IS NULL
    OR data_hora_expiracao > data_hora_publicacao)
);

CREATE TABLE aviso_perfil (
    id_aviso_perfil SERIAL PRIMARY KEY,
    id_aviso INTEGER NOT NULL,
    id_perfil INTEGER NOT NULL,
    lido BOOLEAN NOT NULL DEFAULT FALSE,
    data_hora_leitura TIMESTAMP,
    notificado BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_ap_aviso FOREIGN KEY (id_aviso) REFERENCES aviso (id_aviso) ON DELETE CASCADE,
    CONSTRAINT fk_ap_perfil FOREIGN KEY (id_perfil) REFERENCES perfil (id_perfil),
    CONSTRAINT uk_ap UNIQUE (id_aviso, id_perfil),
    CONSTRAINT ck_ap_leitura CHECK ((lido = FALSE AND data_hora_leitura IS NULL)
    OR (lido = TRUE AND data_hora_leitura IS NOT NULL))
);

CREATE TABLE codigo_primeiro_acesso (
    id_codigo SERIAL PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    id_pessoa INTEGER NOT NULL,
    id_perfil_gerador INTEGER,
    tipo tipo_codigo_enum NOT NULL DEFAULT 'PRIMEIRO_ACESSO',
    status status_codigo_enum NOT NULL DEFAULT 'DISPONIVEL',
    data_criacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_expiracao TIMESTAMP NOT NULL DEFAULT (CURRENT_TIMESTAMP + INTERVAL '7
    days'),
    data_utilizacao TIMESTAMP,
    ip_origem VARCHAR(45),
    CONSTRAINT fk_cpa_pessoa FOREIGN KEY (id_pessoa) REFERENCES pessoa (id_pessoa) ON
    DELETE CASCADE,
    CONSTRAINT fk_cpa_gerador FOREIGN KEY (id_perfil_gerador) REFERENCES perfil (id_perfil)
);

CREATE INDEX idx_cpa_codigo ON codigo_primeiro_acesso (codigo) WHERE status = 'DISPONIVEL';

```

Fonte: Os autores. PostgreSQL 16.

Quadro 41 – Script de criação dos índices e da carga inicial de dados (PostgreSQL 16)

```

-- =====
-- Parte 3 de 3: índices de apoio e carga inicial de dados
-- =====

-- -----
-- Índices sobre as colunas mais utilizadas nas consultas e nos gatilhos
-- -----

CREATE INDEX idx_reserva_area_periodo ON reserva (id_area_comum, data_hora_inicio, status);
CREATE INDEX idx_reserva_perfil ON reserva (id_perfil, status);
CREATE INDEX idx_bloqueio_area_per ON bloqueio_area (id_area_comum, data_hora_inicio, data_hora_fim);
CREATE INDEX idx_bloqueio_perfil_per ON bloqueio_perfil (id_perfil, data_hora_inicio);
CREATE INDEX idx_pu_pessoa ON pessoa_unidade (id_pessoa, data_fim_ocupacao);
CREATE INDEX idx_perfil_pessoa ON perfil (id_pessoa, data_fim);
CREATE INDEX idx_encomenda_status ON encomenda (status, data_hora_recebimento);

```

```

CREATE INDEX idx_entrega_chave_aberta ON entrega_chave (id_chave, data_hora_devolucao);
CREATE INDEX idx_aviso_perfil_lido ON aviso_perfil (id_perfil, lido);

-- -----
-- Carga inicial: estrutura do condominio
-- -----
INSERT INTO bloco (nome, descricao) VALUES
('A', 'Bloco A');

DO $$
DECLARE
    f INT;
    a INT;
BEGIN
    FOR f IN 5..23 LOOP
        FOR a IN 1..4 LOOP
            INSERT INTO unidade (id_bloco, numero_apartamento, numero_vagas)
            VALUES (1, (f * 10) + a, 1);
        END LOOP;
    END LOOP;
END $$;

INSERT INTO vaga_garagem (id_unidade, identificacao, pavimento, tipo_vaga, coberta) VALUES
(1, 'G-014', 'SUBSOLO_1', 'SIMPLES', TRUE),
(1, 'G-015', 'SUBSOLO_1', 'SIMPLES', TRUE),
(2, 'G-032', 'TERREO', 'SIMPLES', FALSE),
(3, 'G-047', 'SUBSOLO_2', 'DUPLA', TRUE);

-- -----
-- Carga inicial: pessoas, perfis e vinculos
-- -----
INSERT INTO pessoa (nome, email, cpf, data_nascimento, celular) VALUES
('Carlos Silva', 'carlos.silva@teste.com',
'11122233344', '1985-04-12', '42999110001'),
('Maria Santos', 'maria.santos@teste.com',
'22233344455', '1990-09-30', '42999110002'),
('Joao Oliveira', 'joao.oliveira@teste.com',
'33344455566', '1998-01-25', '42999110003'),
('Ana Paula Souza', 'ana.souza@teste.com',
'44455566677', '1979-06-08', '42999110004'),
('Roberto Lima', 'roberto.lima@teste.com',
'55566677788', '1982-11-17', '42999110005'),
('Helena Braga', 'helenabraga@teste.com',
'66677788899', '1995-03-03', '42999110006');

INSERT INTO perfil (id_pessoa, tipo_perfil, data_inicio) VALUES
(1, 'MORADOR', '2024-02-01'),
(2, 'MORADOR', '2024-05-10'),
(3, 'MORADOR', '2025-08-01'),
(4, 'SINDICO', '2026-01-01'),
(4, 'MORADOR', '2020-03-15'),
(5, 'PORTEIRO', '2023-07-01'),
(6, 'MORADOR', '2025-01-20');

INSERT INTO pessoa_unidade
(id_pessoa, id_unidade, tipo_vinculo, id_responsavel, grau_parentesco, status_aprovacao, reside,
data_inicio_ocupacao) VALUES
(1, 1, 'PROPRIETARIO', NULL, NULL, 'APROVADO', TRUE, '2024-02-01'),
(3, 1, 'DEPENDENTE', 1, 'FILHO', 'APROVADO', TRUE, '2025-08-01'),
(2, 2, 'INQUILINO', NULL, NULL, 'APROVADO', TRUE, '2024-05-10'),
(4, 5, 'PROPRIETARIO', NULL, NULL, 'APROVADO', TRUE, '2020-03-15'),
(6, 3, 'INQUILINO', NULL, NULL, 'APROVADO', TRUE, '2025-01-20');

-- -----
-- Carga inicial: codigos de primeiro acesso para testes
-- -----
INSERT INTO codigo_primeiro_acesso (codigo, id_pessoa, id_perfil_gerador, status) VALUES
('OASIS-7489', 1, 4, 'DISPONIVEL'),
('OASIS-1234', 2, 4, 'DISPONIVEL'),
('OASIS-5678', 3, 4, 'DISPONIVEL');

-- -----
-- Carga inicial: areas comuns, horarios, utensilios e chaves
-- -----
-- colunas: nome, descricao, capacidade, idade_minima, tipo_acesso, tipo_uso, duracao_slot_min,
-- antecedencia_minima_dias, antecedencia_maxima_dias,
-- prazo_cancelamento_horas, limite_reservas_semana, exige_chave, valor
INSERT INTO area_comum (nome, descricao, capacidade, idade_minima, tipo_acesso, tipo_uso,
duracao_slot_min, antecedencia_minima_dias, antecedencia_maxima_dias,
prazo_cancelamento_horas, limite_reservas_semana, exige_chave, valor) VALUES
('Academia', 'Sala de musculacao e esteiras', 10, 16, 'BIOMETRIA', 'RESERVAVEL',
60, 1, 15, 24, 2, FALSE, 0.00),
('Piscina', 'Piscina adulto e infantil', 30, 0, 'BIOMETRIA', 'RESERVAVEL',
120, 2, 30, 24, 2, FALSE, 0.00),
('Salao de Festas', 'Salao com cozinha de apoio', 50, 18, 'CHAVE', 'RESERVAVEL',
360, 7, 90, 72, 1, TRUE, 150.00),
('Churrasqueira', 'Area gourmet coberta', 15, 18, 'CHAVE', 'RESERVAVEL',
240, 3, 60, 48, 1, TRUE, 60.00),
('Elevador de Servico', 'Uso para mudancas', 4, 18, 'LIVRE', 'RESERVAVEL',
120, 0, 30, 6, 1, FALSE, 0.00);

INSERT INTO area_horario (id_area_comum, dia_semana, hora_inicio, hora_fim) VALUES
(1, 'SEGUNDA', '06:00', '22:00'), (1, 'TERCA', '06:00', '22:00'),
(1, 'QUARTA', '06:00', '22:00'), (1, 'QUINTA', '06:00', '22:00'),
(1, 'SEXTA', '06:00', '22:00'), (1, 'SABADO', '08:00', '18:00'),
(1, 'DOMINGO', '08:00', '14:00'),
(2, 'SABADO', '09:00', '20:00'), (2, 'DOMINGO', '09:00', '20:00'),
(3, 'SEXTA', '12:00', '23:00'), (3, 'SABADO', '10:00', '23:00'),
(3, 'DOMINGO', '10:00', '22:00'),
(4, 'SABADO', '10:00', '22:00'), (4, 'DOMINGO', '10:00', '22:00'),
(5, 'SEGUNDA', '08:00', '17:00'), (5, 'TERCA', '08:00', '17:00'),
(5, 'QUARTA', '08:00', '17:00'), (5, 'QUINTA', '08:00', '17:00'),
(5, 'SEXTA', '08:00', '17:00');

INSERT INTO area_utilidade (id_area_comum, nome, quantidade) VALUES
(3, 'Mesa de plastico', 10), (3, 'Cadeira de plastico', 50),
(3, 'Jogo de talheres', 4), (4, 'Grelha grande', 2), (4, 'Espeto', 12);

```

```
INSERT INTO chave (id_area_comum, codigo) VALUES
(3, 'CH-SALAO-01'), (4, 'CH-CHURRAS-01'), (4, 'CH-CHURRAS-02');
```

Fonte: Os autores. PostgreSQL 16.

4.5.6 Regras de negócio e gatilhos

Além das restrições declarativas, o OASIS implementa regras de negócio dinâmicas por meio de gatilhos em *PL/pgSQL*. Eles garantem a consistência dos dados independentemente da camada de aplicação: ainda que uma operação seja executada diretamente no banco, as regras permanecem válidas. O Quadro 42 descreve as 17 regras implementadas por gatilhos. Em relação à versão anterior, as regras RN01, RN04, RN06, RN07 e RN08 tiveram o comportamento ampliado, e foram criadas as regras RN15, RN16 e RN17.

Quadro 42 – Regras de negócio implementadas por gatilhos

ID	Regra de negócio	Descrição	Evento (gatilho)
RN01	Controlar conflito de horário	Impede que a mesma unidade tenha duas reservas ativas sobrepostas na mesma área e limita a quantidade de unidades que reservam o mesmo horário ao valor de <code>max_unidades_simultaneas</code> .	RESERVA (BEFORE INSERT OR UPDATE)
RN02	Impedir reserva em área bloqueada	Verifica se há bloqueio da área que se sobreponha ao intervalo solicitado.	RESERVA (BEFORE INSERT OR UPDATE)
RN03	Impedir reserva de perfil penalizado	Verifica se o morador possui penalidade vigente para a área solicitada ou para todas as áreas.	RESERVA (BEFORE INSERT OR UPDATE)
RN04	Validar capacidade	Verifica se a soma das pessoas das reservas simultâneas, somada à nova reserva, não excede a capacidade da área.	RESERVA (BEFORE INSERT OR UPDATE)
RN05	Validar janela de funcionamento	Verifica se o intervalo está contido em uma janela de funcionamento do dia da semana.	RESERVA (BEFORE INSERT OR UPDATE)
RN06	Validar antecedência	Impede reserva no passado e verifica a antecedência mínima, em dias e horas, e a máxima, em dias.	RESERVA (BEFORE INSERT OR UPDATE)
RN07	Limitar reservas por unidade	Conta as reservas ativas da unidade na área no período configurado (diário, semanal ou mensal) e impede a inserção ao atingir o limite.	RESERVA (BEFORE INSERT OR UPDATE)
RN08	Controlar cancelamento	Impede o cancelamento de reserva concluída, valida o prazo de cancelamento — exceto nos cancelamentos por manutenção, administrativos ou por penalidade — e registra a data e a hora do cancelamento.	RESERVA (BEFORE UPDATE)
RN09	Controlar empréstimo de chave	Impede o empréstimo de chave que não esteja DISPONIVEL e altera seu status para EMPRESTADA.	ENTREGA_CHAVE (BEFORE e AFTER INSERT)
RN10	Controlar devolução de chave	Valida que a devolução é posterior à retirada e que o funcionário foi informado, liberando a chave.	ENTREGA_CHAVE (BEFORE e AFTER UPDATE)
RN11	Avisar chegada de encomenda	Gera um aviso individual e o registro de destinatário para o morador destinatário.	ENCOMENDA (AFTER INSERT)
RN12	Controlar retirada de encomenda	Exige quem retirou, o funcionário que entregou e, no caso de terceiro, o nome do retirante, atualizando o status para RETIRADA.	ENCOMENDA (BEFORE UPDATE)
RN13	Distribuir avisos do mural	Gera um registro de destinatário para cada perfil ativo que recebe notificações.	AVISO (AFTER INSERT)
RN14	Registrar leitura do aviso	Preenche a data e a hora da leitura ao marcar o aviso como lido.	AVISO_PERFIL (BEFORE UPDATE)
RN15	Validar vínculo de dependente	Exige que o responsável por um dependente seja titular ativo (proprietário ou inquilino) da mesma unidade.	PESSOA_UNIDADE (BEFORE INSERT OR UPDATE)
RN16	Validar idade mínima	Impede a reserva por morador com idade inferior à mínima configurada na área.	RESERVA (BEFORE INSERT OR UPDATE)
RN17	Aplicar penalidade às reservas	Ao registrar uma penalidade, cancela automaticamente as reservas ativas do morador no período e na área atingidos.	BLOQUEIO_PERFIL (AFTER INSERT)

Fonte: Os autores.

Algumas regras não dependem apenas dos dados e, por isso, são implementadas na camada de aplicação. Elas estão descritas no Quadro 43.

Quadro 43 – Regras de negócio implementadas na aplicação

Regra	Descrição	Onde
Unicidade de titular	Cada unidade possui no máximo um titular ativo; dependentes se vinculam a esse titular.	Módulo de cadastros
Proteção do administrador único	O sistema impede inativar, excluir ou retirar o perfil do único administrador ativo.	Módulo de cadastros
Aprovação de dependente	O dependente só recebe código de primeiro acesso após aprovação da administração.	Módulo de cadastros
Pessoas sem acesso ao sistema	Visitantes e prestadores de serviço não recebem perfil nem código de acesso e não podem acumular outro papel.	Módulo de cadastros
Validade dos códigos	Código de ativação válido por 7 dias; código de recuperação válido por 2 horas, com cancelamento dos anteriores.	Módulo de autenticação
Limitação de tentativas	Tentativas de autenticação limitadas por IP e e-mail; demais rotas limitadas por IP.	Guardas da API
Conclusão automática	Reservas ativas cujo horário já se encerrou passam para CONCLUIDA.	Módulo de reservas
Exclusão condicional de área	Área só é excluída se não possuir reservas, chaves ou bloqueios; caso contrário, apenas inativada.	Módulo de áreas
Validação de imagem	Imagem das áreas limitada em formato e tamanho, redimensionada e convertida para WebP.	Módulo de áreas
Privacidade do histórico	O histórico de uso consultado pela portaria é limitado aos últimos 90 dias.	Módulo de portaria

Fonte: Os autores.

O Quadro 44 apresenta a codificação completa dos 12 gatilhos em *PL/pgSQL*.

Quadro 44 – Implementação dos gatilhos em PL/pgSQL

```
-- =====
-- OASIS - Gatilhos (triggers) em PL/pgSQL - PostgreSQL 16
-- =====

-- -----
-- RN01 a RN07: validacao completa da reserva
-- -----
CREATE OR REPLACE FUNCTION fn_valida_reserva() RETURNS TRIGGER AS $$
DECLARE
    v_area          area_comum%ROWTYPE;
    v_dia           dia_semana_enum;
    v_dias_antec    INTEGER;
    v_qtd_periodo   INTEGER;
    v_qtd_simultaneas INTEGER;
    v_total_pessoas_horario INTEGER;
    v_nasc          DATE;
BEGIN
    -- na alteracao de status (cancelamento) as validacoes nao se aplicam
    IF TG_OP = 'UPDATE' AND NEW.status <> 'ATIVA' THEN
        RETURN NEW;
    END IF;

    SELECT * INTO v_area FROM area_comum WHERE id_area_comum = NEW.id_area_comum;

    IF NOT v_area.ativo THEN
        RAISE EXCEPTION 'Area comum inativa: reservas nao sao permitidas.';
    END IF;

    -- RN01-A: a mesma unidade nao pode reservar o mesmo horario mais de uma vez
    IF EXISTS (
        SELECT 1
        FROM reserva r
        JOIN perfil pf ON pf.id_perfil = r.id_perfil
        JOIN pessoa_unidade pu ON pu.id_pessoa = pf.id_pessoa AND pu.data_fim_ocupacao IS NULL
        WHERE r.id_area_comum = NEW.id_area_comum
        AND r.status IN ('ATIVA', 'CONCLUIDA')
        AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
        AND (NEW.data_hora_inicio, NEW.data_hora_fim)
        OVERLAPS (r.data_hora_inicio, r.data_hora_fim)
        AND pu.id_unidade IN (
            SELECT pu2.id_unidade
            FROM pessoa_unidade pu2
            JOIN perfil pf2 ON pf2.id_pessoa = pu2.id_pessoa
            WHERE pf2.id_perfil = NEW.id_perfil
            AND pu2.data_fim_ocupacao IS NULL)
    ) THEN
        RAISE EXCEPTION 'RN01: sua unidade ja possui uma reserva ativa para este mesmo horario.';
    END IF;

    -- RN01-B: limite de unidades/reservas simultaneas no mesmo horario
    SELECT COUNT(*) INTO v_qtd_simultaneas
```

```

FROM reserva r
WHERE r.id_area_comum = NEW.id_area_comum
AND r.status IN ('ATIVA', 'CONCLUIDA')
AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
AND (NEW.data_hora_inicio, NEW.data_hora_fim)
OVERLAPS (r.data_hora_inicio, r.data_hora_fim);

IF v_qtd_simultaneas >= COALESCE(v_area.max_unidades_simultaneas, 1) THEN
RAISE EXCEPTION 'RN01: o limite de reservas simultaneas para este horario ja foi atingido (%
vaga(s)).',
COALESCE(v_area.max_unidades_simultaneas, 1);
END IF;

-- RN02: area interditada no periodo
IF EXISTS (
SELECT 1 FROM bloqueio_area b
WHERE b.id_area_comum = NEW.id_area_comum
AND (NEW.data_hora_inicio, NEW.data_hora_fim)
OVERLAPS (b.data_hora_inicio, b.data_hora_fim)
) THEN
RAISE EXCEPTION 'RN02: area bloqueada no periodo solicitado.';
END IF;

-- RN03: morador com acesso bloqueado para esta area comum ou para todas as areas
IF EXISTS (
SELECT 1 FROM bloqueio_perfil bp
JOIN perfil pf_bloq ON pf_bloq.id_perfil = bp.id_perfil
JOIN perfil pf_res ON pf_res.id_perfil = NEW.id_perfil
WHERE pf_bloq.id_pessoa = pf_res.id_pessoa
AND (bp.id_area_comum IS NULL OR bp.id_area_comum = NEW.id_area_comum)
AND NEW.data_hora_inicio >= bp.data_hora_inicio
AND (bp.data_hora_fim IS NULL OR NEW.data_hora_inicio <= bp.data_hora_fim)
) THEN
RAISE EXCEPTION 'RN03: morador com acesso bloqueado para esta area comum.';
END IF;

-- RN04: capacidade da area comum e soma total de pessoas no mesmo horario
SELECT COALESCE(SUM(r.numero_pessoas), 0) INTO v_total_pessoas_horario
FROM reserva r
WHERE r.id_area_comum = NEW.id_area_comum
AND r.status IN ('ATIVA', 'CONCLUIDA')
AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
AND (NEW.data_hora_inicio, NEW.data_hora_fim)
OVERLAPS (r.data_hora_inicio, r.data_hora_fim);

IF (v_total_pessoas_horario + NEW.numero_pessoas) > v_area.capacidade THEN
IF v_total_pessoas_horario > 0 THEN
RAISE EXCEPTION 'RN04: a capacidade maxima de pessoas (%) para este horario foi excedida
(ja ha % pessoa(s) agendada(s), restando apenas % vaga(s)).',
v_area.capacidade, v_total_pessoas_horario, GREATEST(0,
v_area.capacidade - v_total_pessoas_horario);
ELSE
RAISE EXCEPTION 'RN04: numero de pessoas (%) excede a capacidade da area (%)',
NEW.numero_pessoas, v_area.capacidade;
END IF;
END IF;

-- RN16: restricao de idade minima da area comum
IF COALESCE(v_area.idade_minima, 0) > 0 THEN
SELECT p.data_nascimento INTO v_nasc
FROM perfil pf
JOIN pessoa p ON p.id_pessoa = pf.id_pessoa
WHERE pf.id_perfil = NEW.id_perfil;

IF v_nasc IS NOT NULL AND EXTRACT(YEAR FROM age(NEW.data_hora_inicio::date, v_nasc)) <
v_area.idade_minima THEN
RAISE EXCEPTION 'RN16: a area exige idade minima de % anos para realizacao de
reservas.',
v_area.idade_minima;
END IF;
END IF;

-- RN05: janela de funcionamento do dia da semana
v_dia := (ARRAY['DOMINGO', 'SEGUNDA', 'TERCA', 'QUARTA', 'QUINTA', 'SEXTA', 'SABADO']
)[EXTRACT(DOW FROM NEW.data_hora_inicio)::INTEGER + 1)::dia_semana_enum;

IF NOT EXISTS (
SELECT 1 FROM area_horario h
WHERE h.id_area_comum = NEW.id_area_comum
AND h.dia_semana = v_dia
AND NEW.data_hora_inicio::time >= h.hora_inicio
AND (
NEW.data_hora_fim::time <= h.hora_fim
OR (h.hora_fim >= '23:59:00'::time AND NEW.data_hora_fim::time <= '23:59:59'::time)
)
) THEN
RAISE EXCEPTION 'RN05: horario fora da janela de funcionamento da area em %', v_dia;
END IF;

-- RN06: antecedencia minima e maxima e bloqueio de horario no passado
IF NEW.data_hora_inicio <= CURRENT_TIMESTAMP THEN
RAISE EXCEPTION 'RN06: nao e permitido realizar reserva para horario no passado.';
END IF;

IF NEW.data_hora_inicio < (CURRENT_TIMESTAMP + (COALESCE(v_area.antecedencia_minima_horas,
v_area.antecedencia_minima_dias * 24) || ' hours')::INTERVAL) THEN
RAISE EXCEPTION 'RN06: a reserva exige antecedencia minima de % hora(s).',
COALESCE(v_area.antecedencia_minima_horas, v_area.antecedencia_minima_dias *
24);
END IF;

v_dias_antec := NEW.data_hora_inicio::date - CURRENT_DATE;

IF v_dias_antec > v_area.antecedencia_maxima_dias THEN
RAISE EXCEPTION 'RN06: a reserva so pode ser feita com ate % dia(s) de antecedencia.',
v_area.antecedencia_maxima_dias;

```

```

END IF;

-- RN07: limite de reservas por unidade (DIARIO, SEMANAL ou MENSAL)
SELECT COUNT(DISTINCT r.id_reserva) INTO v_qtd_periodo
FROM reserva r
JOIN perfil pf ON pf.id_perfil = r.id_perfil
JOIN pessoa_unidade pu ON pu.id_pessoa = pf.id_pessoa
AND pu.data_fim_ocupacao IS NULL
WHERE r.id_area_comum = NEW.id_area_comum
AND r.status IN ('ATIVA', 'CONCLUIDA')
AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
AND (
    COALESCE(v_area.tipo_limite_reserva, 'SEMANAL') = 'DIARIO'
    AND r.data_hora_inicio::date = NEW.data_hora_inicio::date)
OR COALESCE(v_area.tipo_limite_reserva, 'SEMANAL') = 'SEMANAL'
    AND date_trunc('week', r.data_hora_inicio) = date_trunc('week', NEW.data_hora_inicio))
OR COALESCE(v_area.tipo_limite_reserva, 'SEMANAL') = 'MENSAL'
    AND date_trunc('month', r.data_hora_inicio) = date_trunc('month', NEW.data_hora_inicio))
)
AND pu.id_unidade IN (
    SELECT pu2.id_unidade
    FROM pessoa_unidade pu2
    JOIN perfil pf2 ON pf2.id_pessoa = pu2.id_pessoa
    WHERE pf2.id_perfil = NEW.id_perfil
    AND pu2.data_fim_ocupacao IS NULL);

IF v_qtd_periodo >= v_area.limite_reservas_semana THEN
    IF COALESCE(v_area.tipo_limite_reserva, 'SEMANAL') = 'DIARIO' THEN
        RAISE EXCEPTION 'RN07: a unidade atingiu o limite de % reserva(s) diaria(s) nesta
            area.',
            v_area.limite_reservas_semana;
    ELSIF COALESCE(v_area.tipo_limite_reserva, 'SEMANAL') = 'MENSAL' THEN
        RAISE EXCEPTION 'RN07: a unidade atingiu o limite de % reserva(s) mensais nesta area.',
            v_area.limite_reservas_semana;
    ELSE
        RAISE EXCEPTION 'RN07: a unidade atingiu o limite de % reserva(s) semanal(is) nesta
            area.',
            v_area.limite_reservas_semana;
    END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_reserva
BEFORE INSERT OR UPDATE ON reserva
FOR EACH ROW EXECUTE FUNCTION fn_valida_reserva();

-- -----
-- RN08: prazo de cancelamento e registro automatico da data
-- -----
CREATE OR REPLACE FUNCTION fn_cancela_reserva() RETURNS TRIGGER AS $$
DECLARE
    v_prazo INTEGER;
BEGIN
    IF NEW.status = 'CANCELADA' AND OLD.status <> 'CANCELADA' THEN
        IF OLD.status = 'CONCLUIDA' THEN
            RAISE EXCEPTION 'RN08: nao e permitido cancelar uma reserva ja concluida.';
        END IF;

        SELECT prazo_cancelamento horas INTO v_prazo
        FROM area_comum WHERE id_area_comum = NEW.id_area_comum;

        IF (NEW.motivo_cancelamento IS NULL OR (NEW.motivo_cancelamento NOT LIKE 'MANUTENCAO%' AND
            NEW.motivo_cancelamento NOT LIKE 'ADMINISTRATIVO%' AND NEW.motivo_cancelamento NOT LIKE
            'PENALIDADE%')) THEN
            IF CURRENT_TIMESTAMP > (OLD.data_hora_inicio - (v_prazo || ' hours')::INTERVAL) THEN
                RAISE EXCEPTION 'RN08: cancelamento permitido somente ate % hora(s) antes do
                    inicio.',
                    v_prazo;
            END IF;
        END IF;

        NEW.data_hora_cancelamento := COALESCE(NEW.data_hora_cancelamento, CURRENT_TIMESTAMP);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_cancela_reserva
BEFORE UPDATE ON reserva
FOR EACH ROW EXECUTE FUNCTION fn_cancela_reserva();

-- -----
-- RN09: emprestimo de chave - validacao e mudanca de status
-- -----
CREATE OR REPLACE FUNCTION fn_valida_emprestimo_chave() RETURNS TRIGGER AS $$
DECLARE
    v_status status_chave_enum;
BEGIN
    SELECT status INTO v_status FROM chave WHERE id_chave = NEW.id_chave;

    IF v_status <> 'DISPONIVEL' THEN
        RAISE EXCEPTION 'RN09: a chave nao esta disponivel (situacao atual: %).', v_status;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_emprestimo_chave
BEFORE INSERT ON entrega_chave
FOR EACH ROW EXECUTE FUNCTION fn_valida_emprestimo_chave();

CREATE OR REPLACE FUNCTION fn_aplica_emprestimo_chave() RETURNS TRIGGER AS $$
BEGIN

```

```

        UPDATE chave SET status = 'EMPRESTADA' WHERE id_chave = NEW.id_chave;
        RETURN NEW;
    END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_aplica_emprestimo_chave
    AFTER INSERT ON entrega_chave
    FOR EACH ROW EXECUTE FUNCTION fn_aplica_emprestimo_chave();

-----
-- RN10: devolucao de chave - validacao e liberacao
-----
CREATE OR REPLACE FUNCTION fn_valida_devolucao_chave() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_hora_devolucao IS NOT NULL THEN
        IF NEW.data_hora_devolucao < NEW.data_hora_retirada THEN
            RAISE EXCEPTION 'RN10: a devolucao nao pode ser anterior a retirada.';
        END IF;
        IF NEW.id_perfil_recebimento IS NULL THEN
            RAISE EXCEPTION 'RN10: informe o funcionario que recebeu a chave.';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_devolucao_chave
    BEFORE UPDATE ON entrega_chave
    FOR EACH ROW EXECUTE FUNCTION fn_valida_devolucao_chave();

CREATE OR REPLACE FUNCTION fn_aplica_devolucao_chave() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_hora_devolucao IS NOT NULL AND OLD.data_hora_devolucao IS NULL THEN
        UPDATE chave SET status = 'DISPONIVEL' WHERE id_chave = NEW.id_chave;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_aplica_devolucao_chave
    AFTER UPDATE ON entrega_chave
    FOR EACH ROW EXECUTE FUNCTION fn_aplica_devolucao_chave();

-----
-- RN11: aviso individual de chegada de encomenda
-----
CREATE OR REPLACE FUNCTION fn_notifica_encomenda() RETURNS TRIGGER AS $$
DECLARE
    v_perfil_dest INTEGER;
    v_aviso        INTEGER;
BEGIN
    SELECT p.id_perfil INTO v_perfil_dest
    FROM perfil p
    WHERE p.id_pessoa = NEW.id_pessoa_destinatario
    AND p.tipo_perfil = 'MORADOR'
    AND p.data_fim IS NULL
    ORDER BY p.data_inicio DESC
    LIMIT 1;

    IF v_perfil_dest IS NOT NULL THEN
        INSERT INTO aviso (id_perfil_autor, escopo, titulo, conteudo)
        VALUES (NEW.id_perfil_recebimento, 'INDIVIDUAL', 'Nova encomenda na portaria',
            'Voce recebeu uma encomenda de porte ' || NEW.tamanho ||
            '. Retire na portaria apresentando um documento.')
        RETURNING id_aviso INTO v_aviso;

        INSERT INTO aviso_perfil (id_aviso, id_perfil) VALUES (v_aviso, v_perfil_dest);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_notifica_encomenda
    AFTER INSERT ON encomenda
    FOR EACH ROW EXECUTE FUNCTION fn_notifica_encomenda();

-----
-- RN12: retirada de encomenda
-----
CREATE OR REPLACE FUNCTION fn_retirada_encomenda() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_hora_retirada IS NOT NULL AND OLD.data_hora_retirada IS NULL THEN
        IF NEW.retirado_por IS NULL THEN
            RAISE EXCEPTION 'RN12: informe quem retirou a encomenda.';
        END IF;
        IF NEW.retirado_por = 'TERCEIRO' AND NEW.nome_retirante IS NULL THEN
            RAISE EXCEPTION 'RN12: informe o nome do terceiro que retirou a encomenda.';
        END IF;
        IF NEW.id_perfil_entrega IS NULL THEN
            RAISE EXCEPTION 'RN12: informe o funcionario que entregou a encomenda.';
        END IF;
        NEW.status := 'RETIRADA';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_retirada_encomenda
    BEFORE UPDATE ON encomenda
    FOR EACH ROW EXECUTE FUNCTION fn_retirada_encomenda();

-----
-- RN13: distribuicao automatica dos avisos do mural
-----
CREATE OR REPLACE FUNCTION fn_distribui_aviso() RETURNS TRIGGER AS $$
BEGIN

```

```

        IF NEW.escopo = 'MURAL' THEN
            INSERT INTO aviso_perfil (id aviso, id_perfil)
            SELECT NEW.id_aviso, p.id_perfil
            FROM perfil_p
            WHERE p.data_fim IS NULL
            AND p.recebe_notificacao = TRUE
            AND p.id_perfil <> NEW.id_perfil_autor;
        END IF;
        RETURN NEW;
    END;
    $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_distribui_aviso
    AFTER INSERT ON aviso
    FOR EACH ROW EXECUTE FUNCTION fn_distribui_aviso();

-----
-- RN14: registro da data e hora de leitura do aviso
-----
CREATE OR REPLACE FUNCTION fn_leitura_aviso() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.lido = TRUE AND OLD.lido = FALSE THEN
        NEW.data_hora_leitura := CURRENT_TIMESTAMP;
    ELSIF NEW.lido = FALSE THEN
        NEW.data_hora_leitura := NULL;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_leitura_aviso
    BEFORE UPDATE ON aviso_perfil
    FOR EACH ROW EXECUTE FUNCTION fn_leitura_aviso();

-----
-- RN15: validacao de dependente (responsavel deve ser titular ativo da mesma unidade)
-----
CREATE OR REPLACE FUNCTION fn_valida_dependente() RETURNS TRIGGER AS $$
BEGIN
    -- Se o vinculo esta sendo encerrado ou ja esta inativo, nao exige titular ativo
    IF NEW.data_fim_ocupacao IS NOT NULL THEN
        RETURN NEW;
    END IF;

    IF NEW.tipo_vinculo = 'DEPENDENTE' THEN
        IF NOT EXISTS (
            SELECT 1 FROM pessoa_unidade pu
            WHERE pu.id_pessoa = NEW.id_responsavel
            AND pu.id_unidade = NEW.id_unidade
            AND pu.data_fim_ocupacao IS NULL
            AND pu.tipo_vinculo IN ('PROPRIETARIO', 'INQUILINO')
        ) THEN
            RAISE EXCEPTION 'RN15: o responsavel informado deve ser o titular ativo (proprietario ou
            inquilino) da mesma unidade.';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_valida_dependente
    BEFORE INSERT OR UPDATE ON pessoa_unidade
    FOR EACH ROW EXECUTE FUNCTION fn_valida_dependente();

-----
-- RN17: cancelamento automatico de reservas ativas ao aplicar penalidade
-----
CREATE OR REPLACE FUNCTION fn_bloqueio_perfil_cancela_reservas() RETURNS TRIGGER AS $$
BEGIN
    UPDATE reserva r
    SET
        status = 'CANCELADA',
        data_hora_cancelamento = CURRENT_TIMESTAMP,
        motivo_cancelamento = 'PENALIDADE: ' || COALESCE(NEW.motivo::text, 'Bloqueio disciplinar
        aplicado pelo sindico.')
    FROM perfil pf_res, perfil pf_bloq
    WHERE r.id_perfil = pf_res.id_perfil
    AND pf_bloq.id_perfil = NEW.id_perfil
    AND pf_res.id_pessoa = pf_bloq.id_pessoa
    AND r.status = 'ATIVA'
    AND (NEW.id_area_comum IS NULL OR r.id_area_comum = NEW.id_area_comum)
    AND (r.data_hora_fim > NEW.data_hora_inicio AND (NEW.data_hora_fim IS NULL OR
    r.data_hora_inicio < NEW.data_hora_fim));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tg_bloqueio_perfil_cancela_reservas ON bloqueio_perfil;
CREATE TRIGGER tg_bloqueio_perfil_cancela_reservas
    AFTER INSERT ON bloqueio_perfil
    FOR EACH ROW EXECUTE FUNCTION fn_bloqueio_perfil_cancela_reservas();

```

Fonte: Os autores. PostgreSQL 16 / PL/pgSQL.

4.6 INTERFACE DO SISTEMA

Nas versões anteriores deste documento, esta seção apresentava protótipos elaborados no *Figma*. Com o sistema implementado, as figuras passam a ser capturas

de tela do sistema em funcionamento. A interface segue uma identidade visual própria, cujo logotipo e ilustrações foram criados pelos autores no *Adobe Illustrator*. O sistema possui 15 telas, relacionadas no Quadro 45 com a rota de acesso, o perfil que as utiliza e os casos de uso que atendem.

Quadro 45 – Telas do sistema

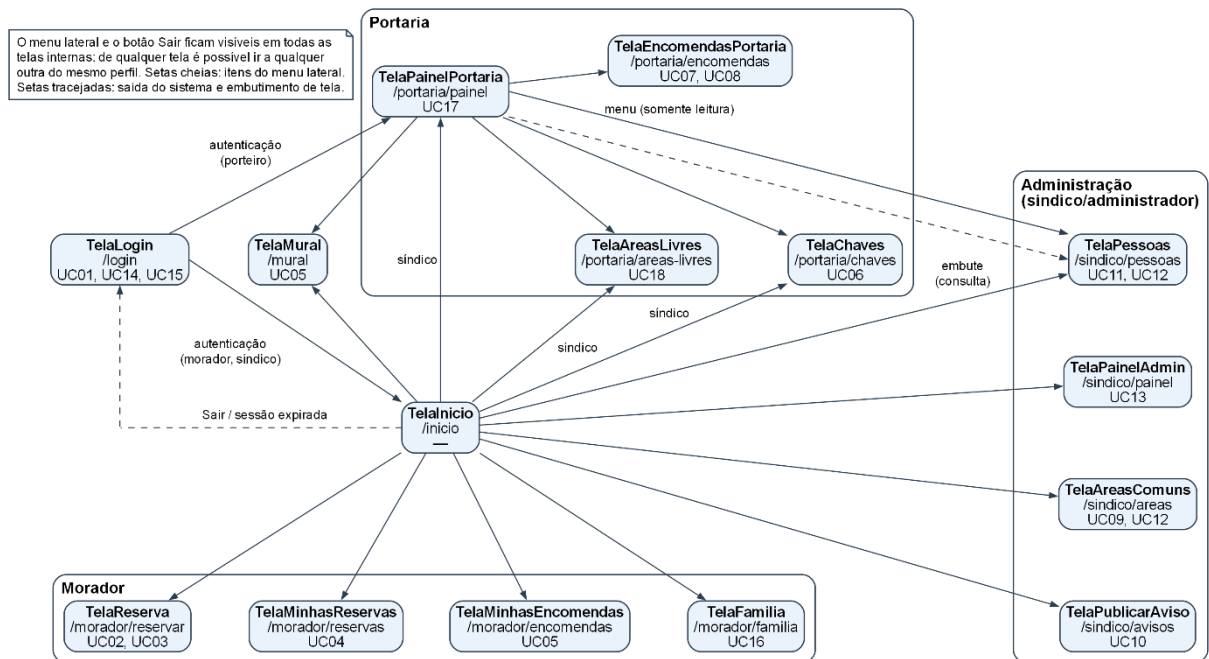
Tela	Rota	Perfil	Casos de uso
Login	/login	Todos	UC01, UC14, UC15
Início	/inicio	Todos	—
Nova reserva	/morador/reservar	Morador	UC02, UC03
Minhas reservas	/morador/reservas	Morador	UC04
Minhas encomendas	/morador/encomendas	Morador	UC05
Família	/morador/familia	Morador	UC16
Mural de avisos	/mural	Todos	UC05
Painel da portaria	/portaria/painel	Porteiro, Síndico	UC17
Áreas de uso livre	/portaria/areas-livres	Porteiro, Síndico	UC18
Encomendas da portaria	/portaria/encomendas	Porteiro	UC07, UC08
Chaves	/portaria/chaves	Porteiro, Síndico	UC06
Painel administrativo	/sindico/painel	Síndico	UC13
Áreas comuns	/sindico/areas	Síndico	UC09, UC12
Pessoas e unidades	/sindico/pessoas	Síndico; Porteiro (consulta)	UC11, UC12
Publicar aviso	/sindico/avisos	Síndico	UC10

Fonte: Os autores.

4.6.1 Mapa de navegação

A navegação parte sempre da tela de *login*. Após a autenticação, o morador e o síndico chegam à tela de início, que reúne saudação, cartões de resumo e atalhos conforme o perfil; o porteiro é levado diretamente ao painel da portaria, que é a sua tela principal. O porteiro também acessa a tela de pessoas e unidades, em modo somente leitura, pelo menu ou pelo painel da portaria. O menu lateral fixo permanece visível nas telas internas e agrupa as funcionalidades por perfil, de modo que uma pessoa com mais de um perfil vê as seções de todos eles sem duplicação. A Figura 13 apresenta o mapa completo de navegação, com o caso de uso atendido por cada tela.

Figura 13 – Mapa de navegação do sistema OASIS por perfil de usuário



Fonte: Os autores.

As seguintes convenções de navegação são adotadas em todas as telas:

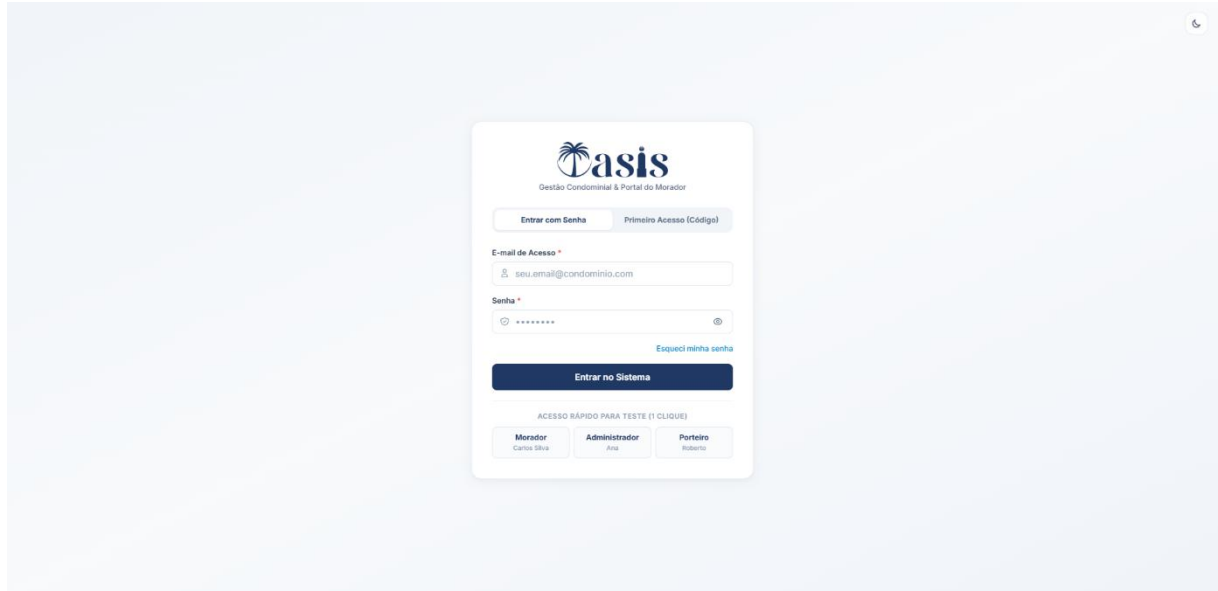
- Menu lateral fixo com os módulos do perfil autenticado, destacando o módulo em uso; o logotipo do cabeçalho e o item Início levam à tela inicial;
- Identificação do usuário e de seus perfis no canto superior direito, com alternância entre tema claro e escuro;
- Ações destrutivas ou irreversíveis exigem confirmação em janela modal, que explica a consequência da ação;
- Mensagens de sucesso e de erro exibidas em notificações flutuantes (*toasts*); os erros originados nos gatilhos do banco exibem o texto da regra correspondente;
- Janelas modais com cabeçalho e rodapé fixos, de modo que os *botões* de ação permaneçam visíveis em qualquer tamanho de tela;
- Layout responsivo, com o menu lateral recolhido em telas de dispositivos móveis.

4.6.2 Telas comuns aos perfis

A tela de *login*, apresentada na Figura 14, possui duas abas: entrada com senha e primeiro acesso por código, além da opção de recuperação de senha. Em

modo de demonstração, ela exibe atalhos de acesso aos perfis de teste, utilizados na apresentação acadêmica.

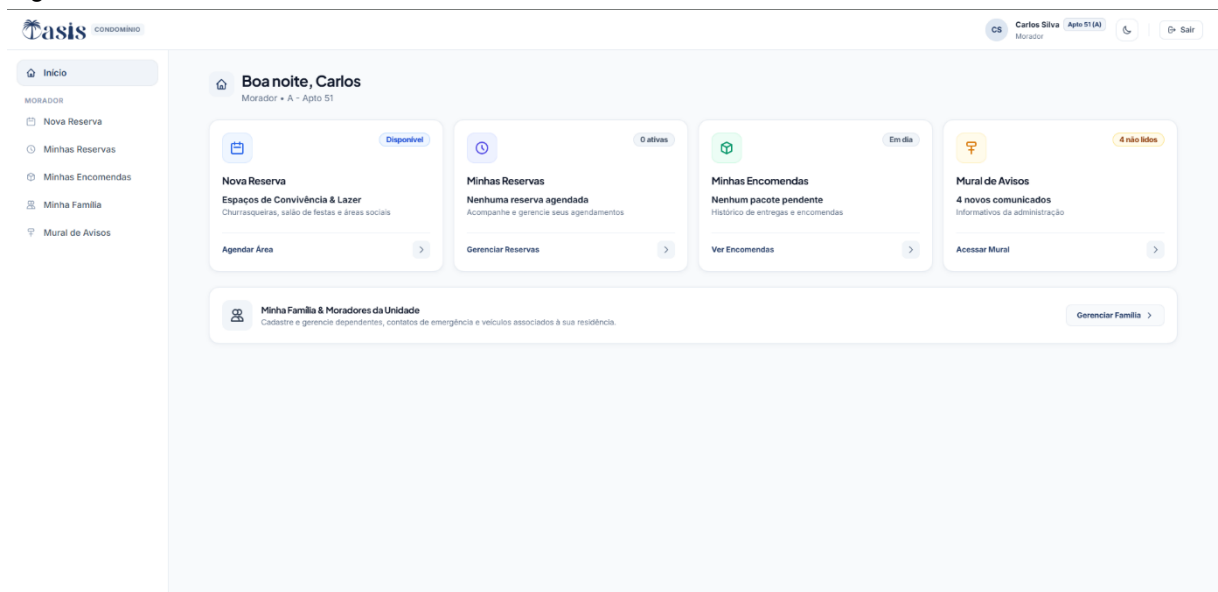
Figura 14 – Tela de login



Fonte: Os autores. Captura de tela do sistema em funcionamento.

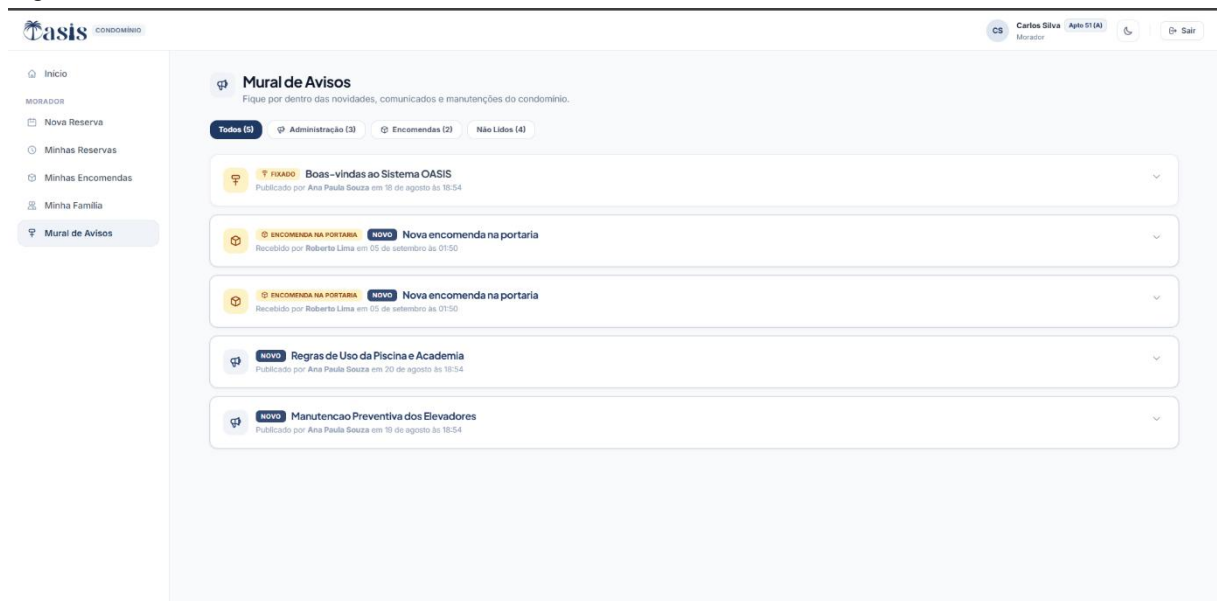
A tela de início, apresentada na Figura 15, cumprimenta o usuário pelo nome, apresenta cartões de resumo do seu perfil — próxima reserva, avisos não lidos e encomendas aguardando retirada, no caso do morador — e atalhos para as funções mais usadas. O mural de avisos, na Figura 16, separa em abas os comunicados da administração e os avisos de encomenda, com destaque para os não lidos.

Figura 15 – Tela de início



Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 16 – Mural de avisos

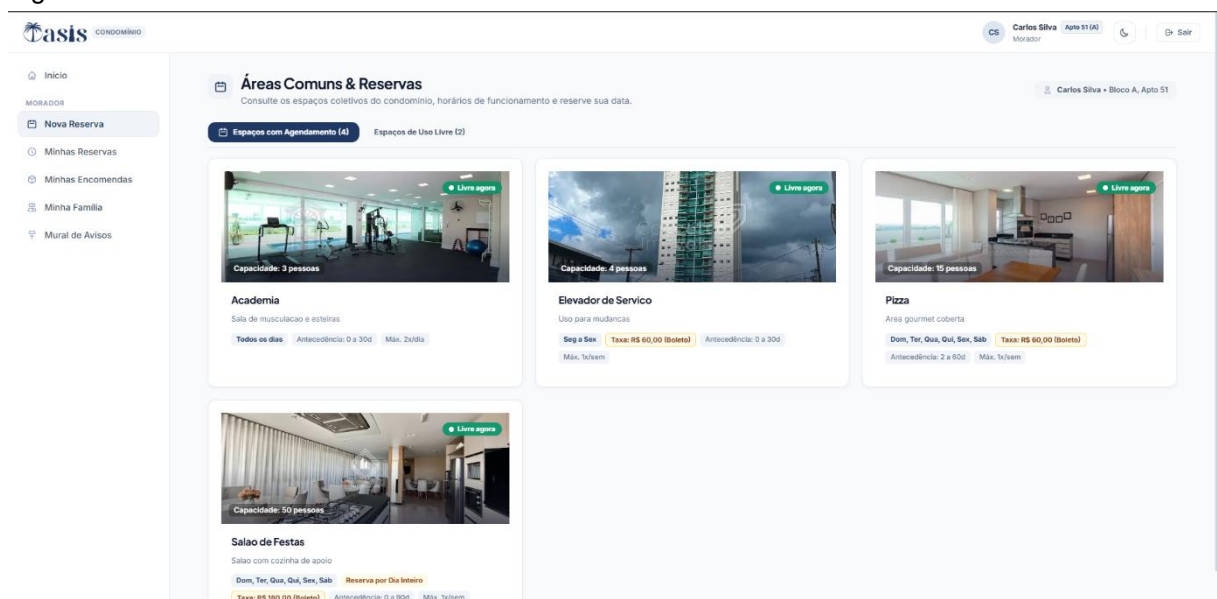


Fonte: Os autores. Captura de tela do sistema em funcionamento.

4.6.3 Telas do perfil Morador

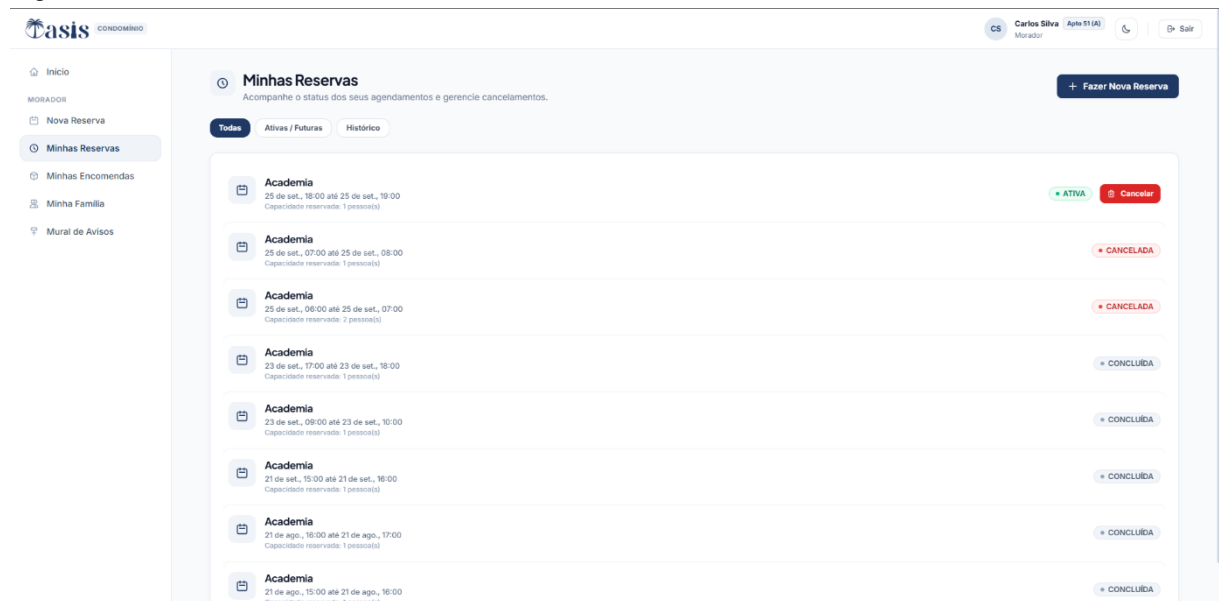
A Figura 17 apresenta a tela de reserva, com a seleção da área, o calendário mensal e a grade de horários, que indica os intervalos livres, parcialmente ocupados e lotados. O cabeçalho identifica o morador em nome de quem a reserva é feita, e o painel lateral reproduz as regras configuradas para a área. A Figura 18 apresenta a lista de reservas do morador, com filtros por situação e o cancelamento dentro do prazo.

Figura 17 – Tela de reserva de áreas comuns



Fonte: Os autores. Captura de tela do sistema em funcionamento.

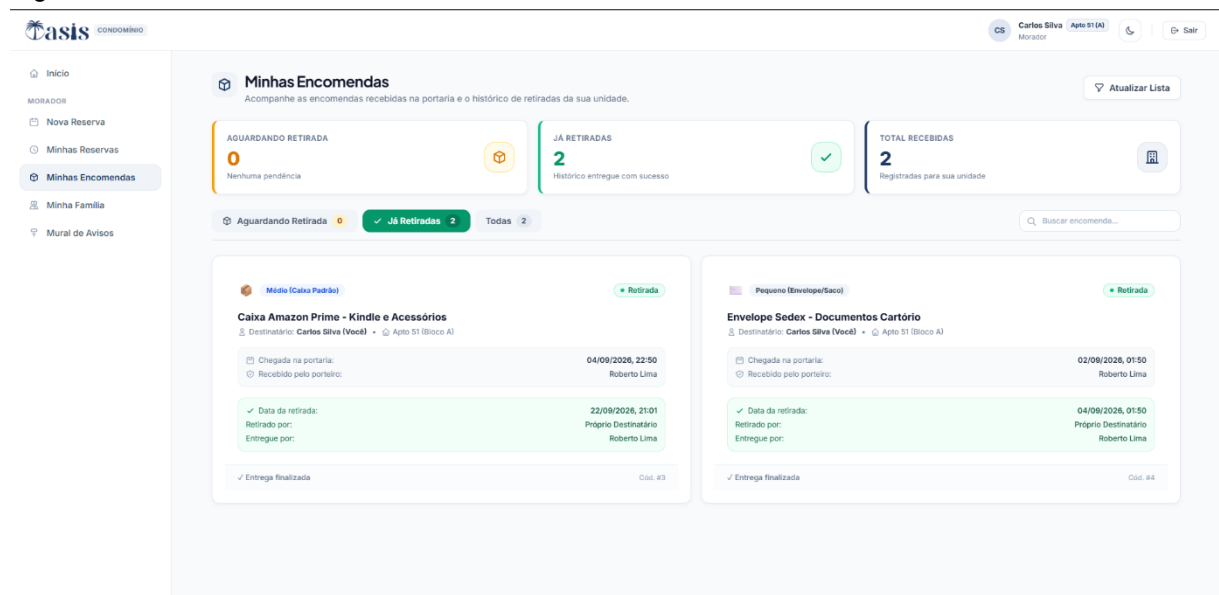
Figura 18 – Tela de reservas do morador



Fonte: Os autores. Captura de tela do sistema em funcionamento.

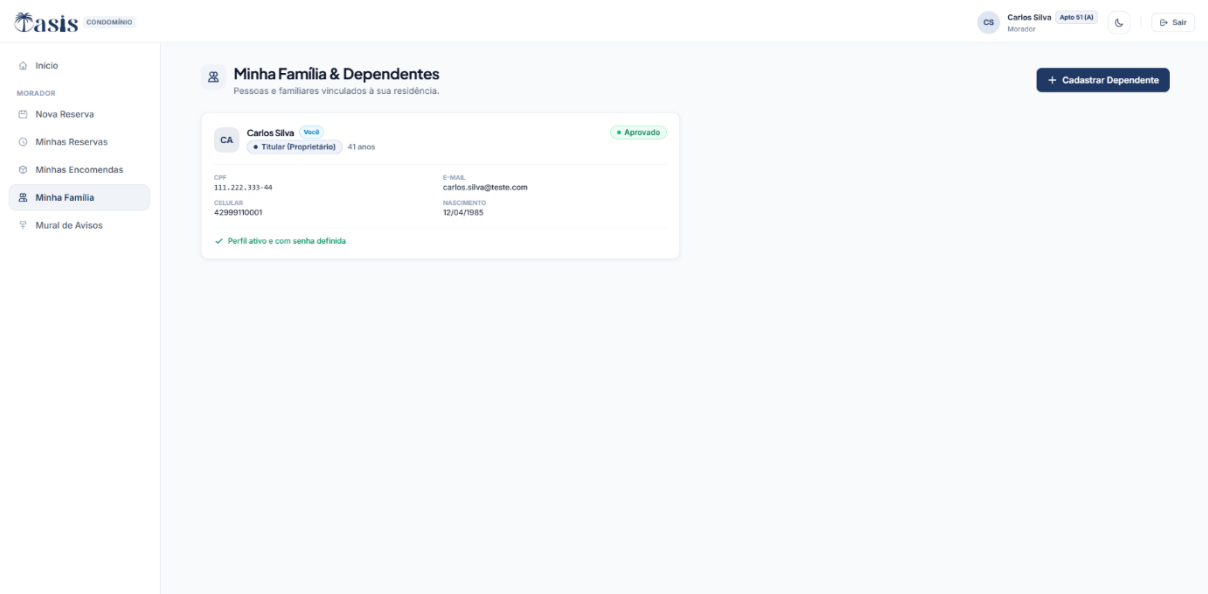
A Figura 19 apresenta a consulta das encomendas do morador, e a Figura 20 apresenta a tela Família, na qual o titular cadastra dependentes, acompanha a aprovação e obtém o código de primeiro acesso de cada um.

Figura 19 – Tela de encomendas do morador



Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 20 – Tela de gestão de dependentes (Família)

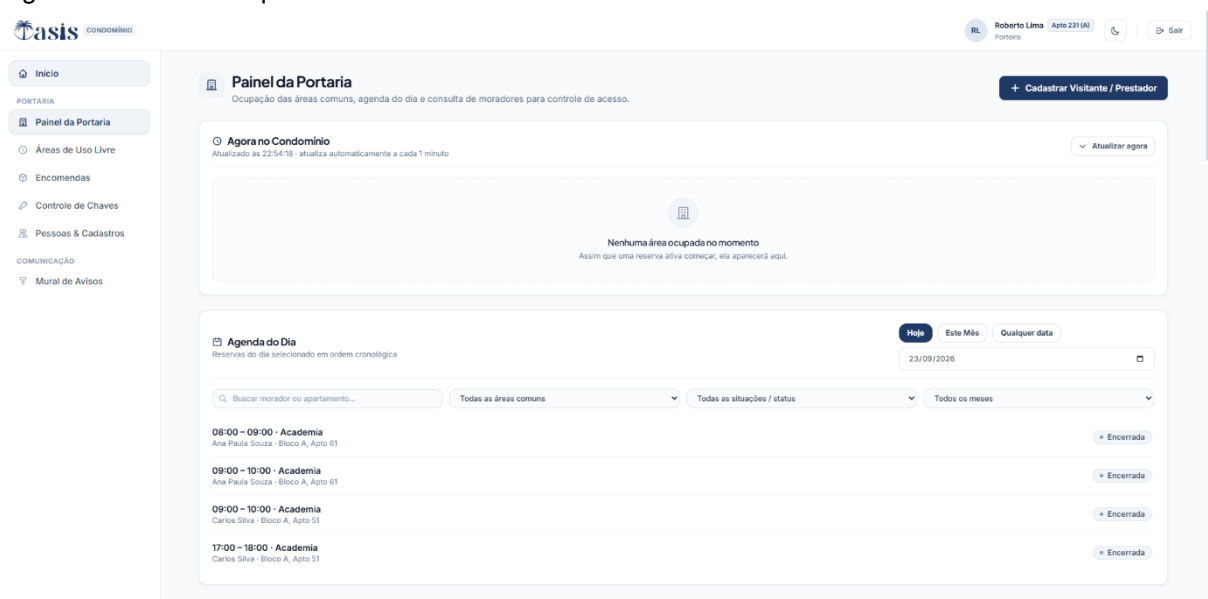


Fonte: Os autores. Captura de tela do sistema em funcionamento.

4.6.4 Telas do perfil Porteiro

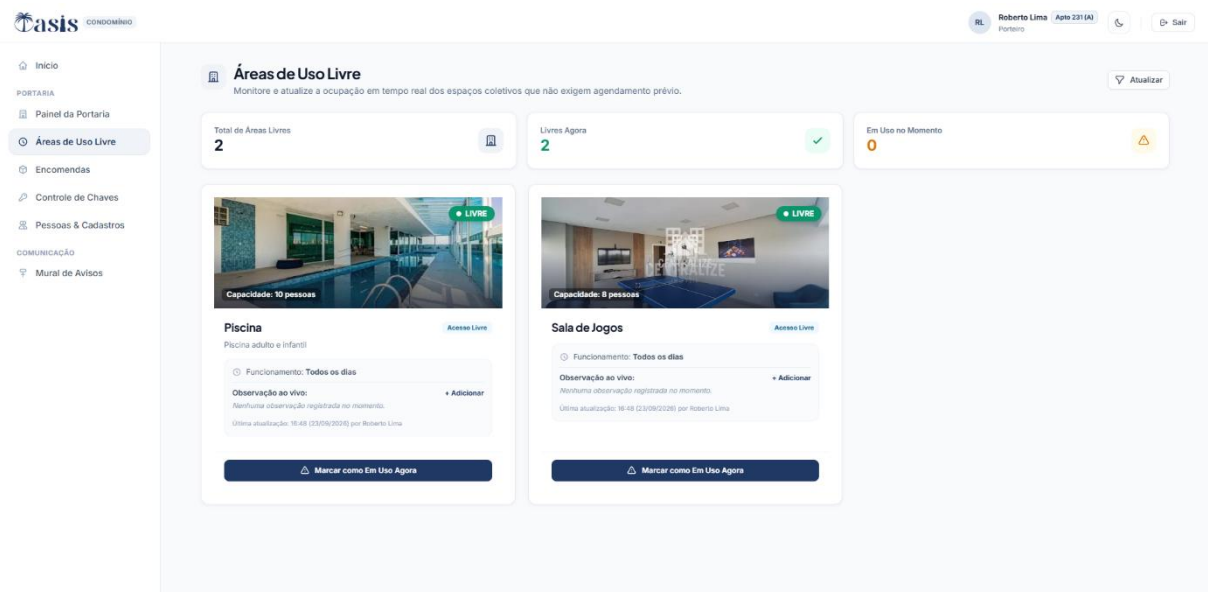
O painel da portaria, na Figura 21, é a tela principal do porteiro: exhibe as áreas ocupadas no momento, com tempo restante e sinalização de atraso, a agenda do dia e a busca de moradores, que mostra a atividade atual, as próximas reservas e o histórico dos últimos 90 dias de cada pessoa, além do acesso ao cadastro completo em modo somente leitura. A Figura 22 apresenta o controle das áreas de uso livre.

Figura 21 – Painel da portaria



Fonte: Os autores. Captura de tela do sistema em funcionamento.

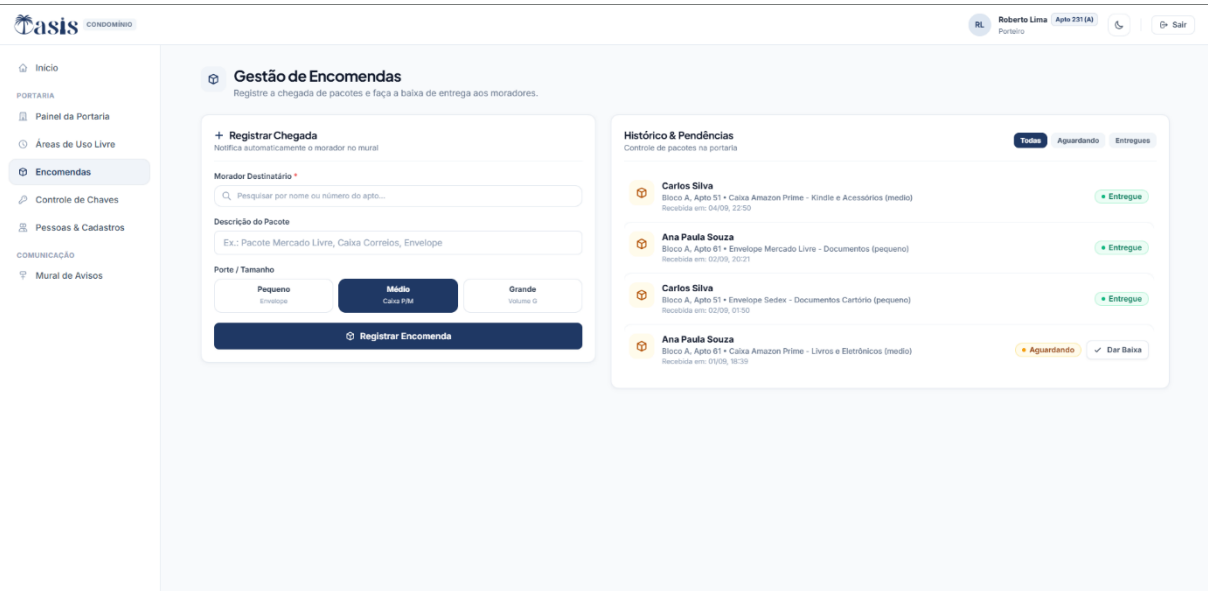
Figura 22 – Tela de áreas de uso livre



Fonte: Os autores. Captura de tela do sistema em funcionamento.

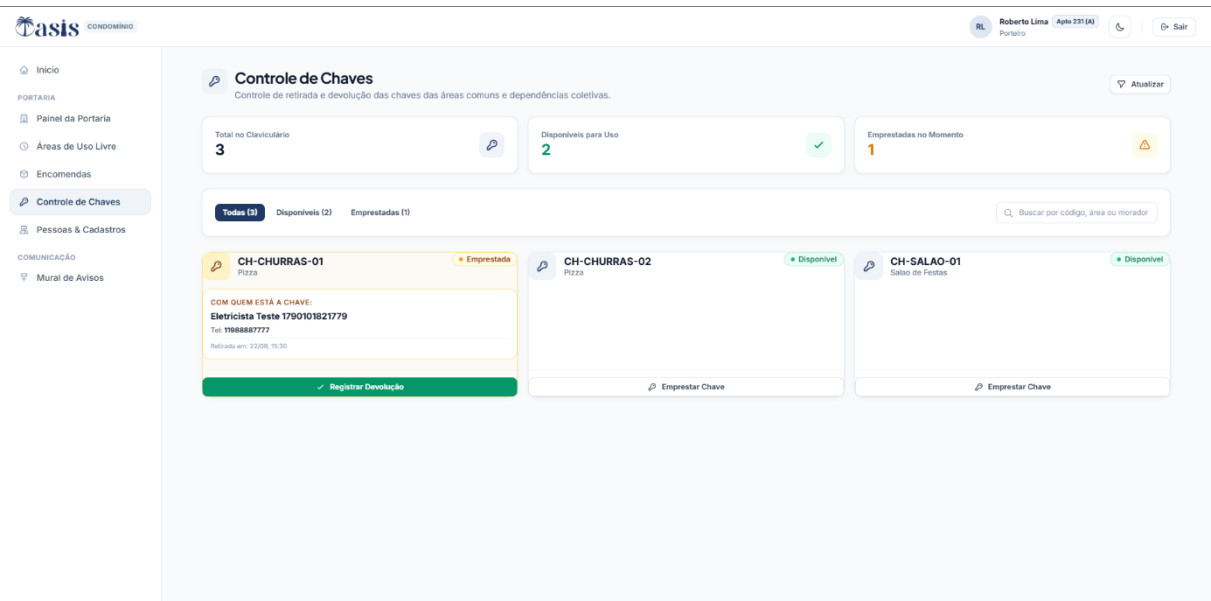
A Figura 23 apresenta o registro de encomendas e a alteração de sua situação, com a indicação obrigatória de quem retirou. A Figura 24 apresenta o controle de empréstimo e devolução de chaves.

Figura 23 – Tela de encomendas da portaria



Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 24 – Tela de controle de chaves

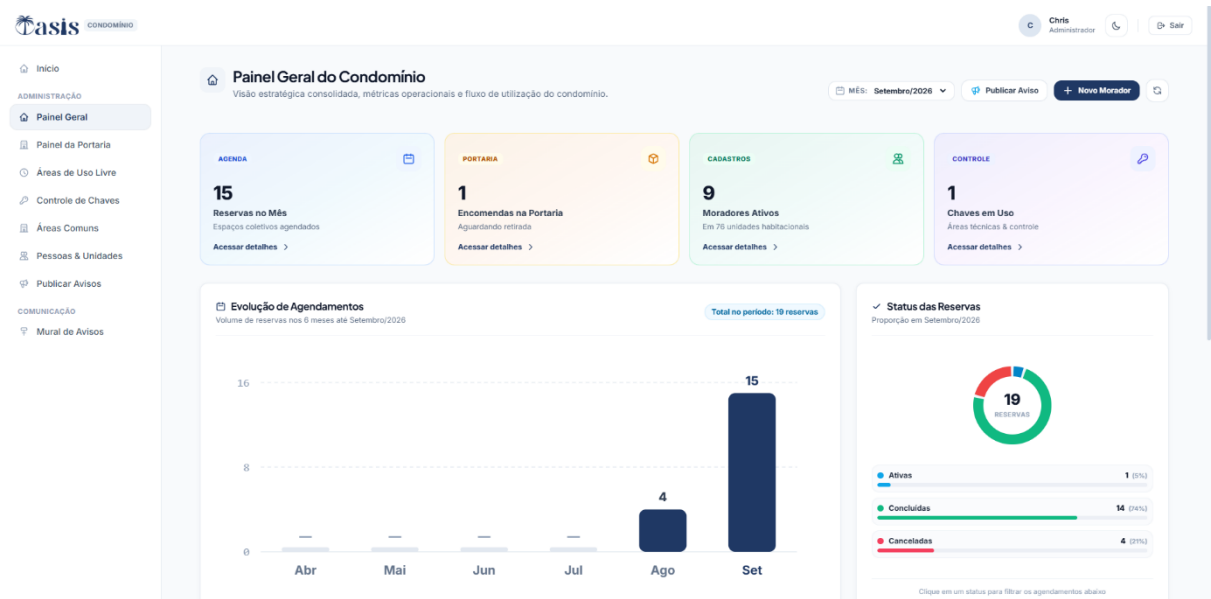


Fonte: Os autores. Captura de tela do sistema em funcionamento.

4.6.5 Telas do perfil Síndico e manutenção de cadastros

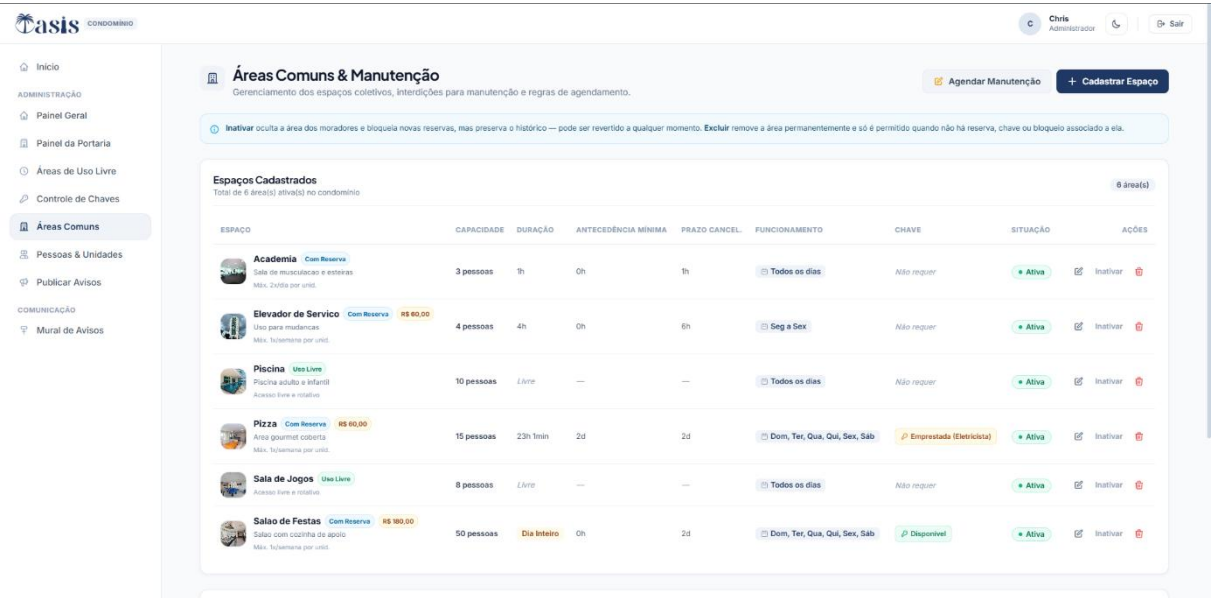
O painel administrativo, na Figura 25, reúne indicadores, gráficos de uso das áreas e filtros por período. A Figura 26 apresenta a gestão de áreas comuns, com as áreas ativas separadas das inativas e as ações de editar, inativar e excluir; a Figura 27 apresenta a gestão de pessoas e unidades, incluindo as aprovações pendentes de dependentes; e a Figura 28 apresenta a publicação de avisos.

Figura 25 – Painel administrativo



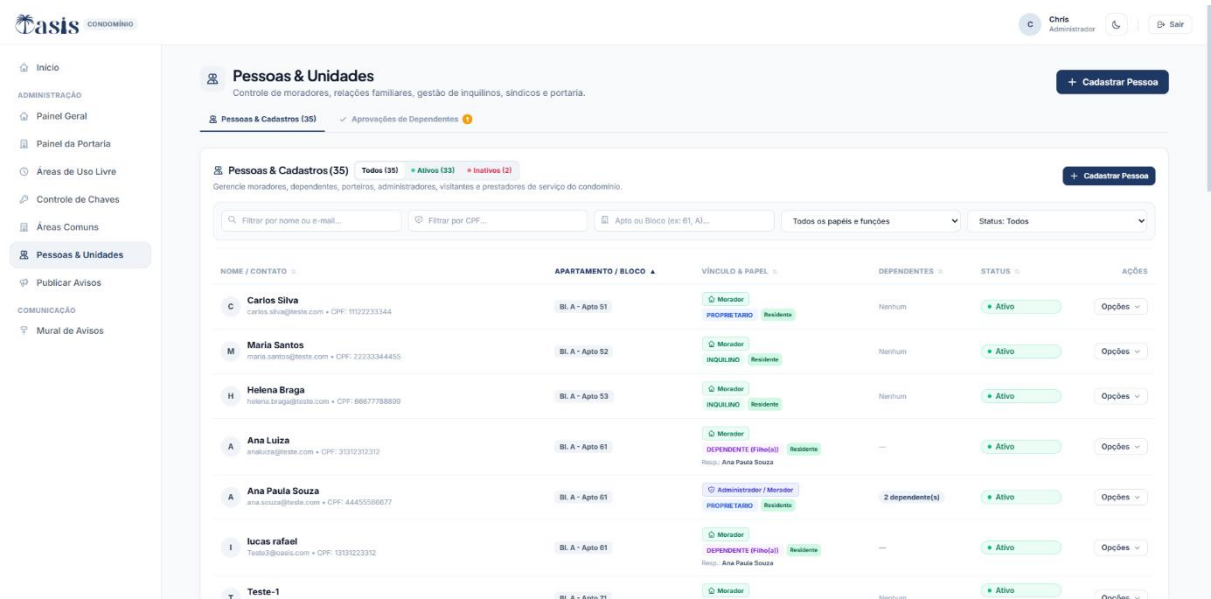
Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 26 – Tela de gestão de áreas comuns



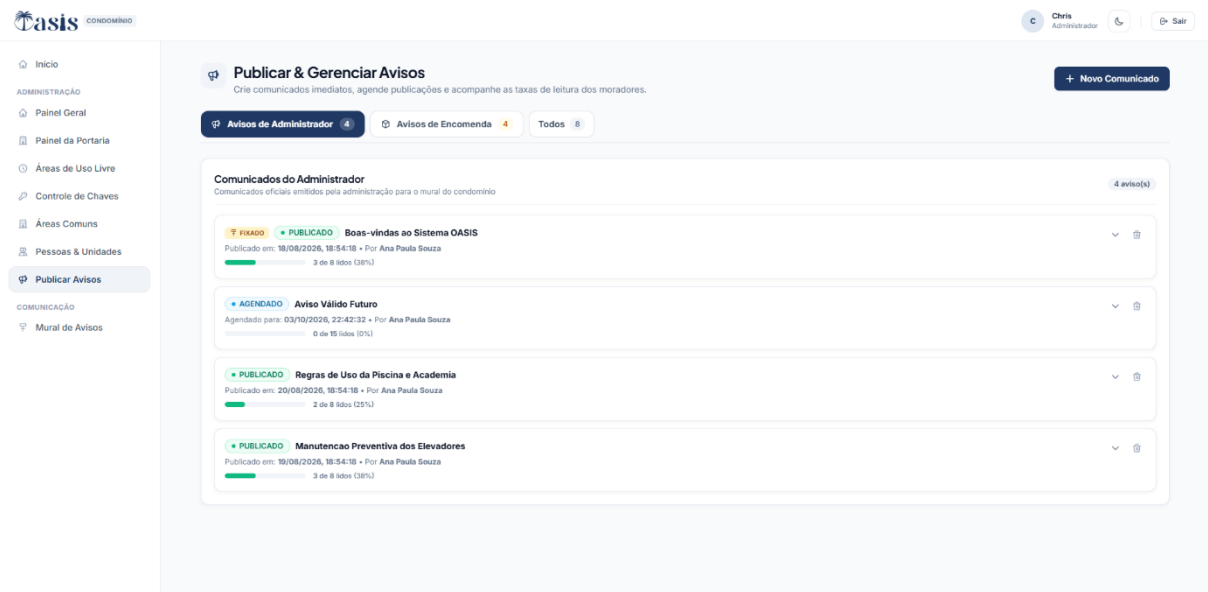
Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 27 – Tela de gestão de pessoas e unidades



Fonte: Os autores. Captura de tela do sistema em funcionamento.

Figura 28 – Tela de publicação de avisos



Fonte: Os autores. Captura de tela do sistema em funcionamento.

O Quadro 46 relaciona as operações de manutenção de cadastros (CRUD) disponíveis no sistema.

Quadro 46 – Operações de manutenção de cadastros (CRUD)

Tela	Entidades mantidas	Operações	Perfil	UC
Pessoas e unidades	PESSOA, PERFIL, PESSOA_UNIDADE, UNIDADE, BLOCO	Incluir, consultar, editar, inativar, reativar, excluir, aprovar dependentes; cadastrar visitantes e prestadores	Síndico	UC11
Família	PESSOA, PESSOA_UNIDADE	Incluir e consultar dependentes	Morador	UC16
Áreas comuns	AREA_COMUM, AREA_HORARIO, AREA_UTENSILIO	Incluir, consultar, editar, inativar, reativar, excluir	Síndico	UC09
Chaves	CHAVE	Incluir, consultar, editar, excluir	Síndico	UC09
Bloqueios e penalidades	BLOQUEIO_AREA, BLOQUEIO_PERFIL	Incluir, consultar, encerrar, excluir	Síndico	UC12
Mural de avisos	AVISO	Incluir, consultar, excluir	Síndico	UC10
Minhas reservas	RESERVA	Incluir, consultar, cancelar	Morador	UC03, UC04
Chaves da portaria	ENTREGA_CHAVE	Incluir, consultar, registrar devolução	Porteiro, Síndico	UC06
Encomendas	ENCOMENDA	Incluir, consultar, alterar status	Porteiro	UC07, UC08
Áreas de uso livre	AREA_COMUM (estado de ocupação)	Consultar, alterar estado	Porteiro, Síndico	UC18

Fonte: Os autores.

4.7 PLANO DE TESTES

4.7.1 Objetivo

Este plano de testes define a estratégia, o escopo, a massa de dados e os casos de teste necessários para verificar e validar o comportamento do sistema

OASIS em conformidade com os casos de uso da seção 4.4.1 e com as regras de negócio da seção 4.5.6.

4.7.2 Escopo

Os testes abrangem os 18 casos de uso, cobrindo os fluxos principais e os principais fluxos alternativos, e a verificação individual das 17 regras implementadas por gatilhos e das regras implementadas na aplicação. Testes de desempenho e testes de intrusão estão fora do escopo deste plano.

4.7.3 Estratégia e critérios de aceitação

A estratégia adotada é a de testes funcionais caixa-preta, em que entradas e saídas são verificadas sem conhecimento da implementação interna, complementada por testes executados diretamente no banco de dados para as regras implementadas por gatilhos. Os critérios de aceitação são: (a) o resultado obtido deve corresponder exatamente ao esperado em cada caso de teste; (b) nenhum caso de prioridade alta pode falhar; (c) os casos de prioridade média devem ter aprovação mínima de 90%. São de prioridade alta os casos de autenticação, integridade das reservas, controle de chaves e rastreabilidade das encomendas.

4.7.4 Massa de dados

Os testes utilizam a carga inicial criada pelos *scripts* da seção 4.5.5, em ambiente executado com o modo de demonstração habilitado, no qual as contas de teste aceitam a senha definida para o ambiente. As datas são expressas em relação à data de referência D, uma segunda-feira de execução dos testes, de modo que o plano possa ser reexecutado em qualquer semana. O Quadro 47 apresenta os usuários e o Quadro 48 apresenta as áreas comuns com as regras configuradas.

Quadro 47 – Massa de dados: usuários do ambiente de testes

Nome	E-mail (login)	Perfil	Bloco / Apto	Vínculo
Carlos Silva	carlos.silva@teste.com	MORADOR	A / 302	Proprietário (titular)
Joao Oliveira	joao.oliveira@teste.com	MORADOR	A / 302	Dependente de Carlos (filho)
Maria Santos	maria.santos@teste.com	MORADOR	A / 201	Inquilina (titular)
Helena Braga	helenabraga@teste.com	MORADOR	B / 105	Inquilina (titular)
Ana Paula Souza	ana.souza@teste.com	SINDICO e MORADOR	C / 401	Proprietária (titular)
Roberto Lima	roberto.lima@teste.com	PORTEIRO	—	—

Fonte: Os autores.

Quadro 48 – Massa de dados: áreas comuns e regras configuradas

Área comum	Cap.	Idade mín.	Slot (min)	Antec. (dias)	Cancel. (h)	Limite	Chave
Academia	10	16	60	1 a 15	24	2 por semana	Não
Piscina	30	—	120	2 a 30	24	2 por semana	Não
Salao de Festas	50	18	360	7 a 90	72	1 por semana	Sim
Churrasqueira	15	18	240	3 a 60	48	1 por semana	Sim
Elevador de Serviço	4	18	120	0 a 30	6	1 por semana	Não

Fonte: Os autores. Todas as áreas admitem uma unidade por horário.

4.7.5 Casos de teste

O Quadro 49 apresenta os 49 casos de teste do sistema, com o caso de uso e a regra relacionados, a pré-condição e os dados de entrada, o resultado esperado e a prioridade.

Quadro 49 – Casos de teste do sistema OASIS

ID	Caso de teste	UC / Regra	Pré-condição e entrada	Resultado esperado	Prior.
CT01	Login válido	UC01	Carlos Silva, conta ATIVO, senha correta	Sessão criada; redirecionamento à tela de início	Alta
CT02	Senha incorreta	UC01	carlos.silva@teste.com com senha errada	Mensagem genérica de credenciais inválidas	Alta
CT03	Conta aguardando primeiro acesso	UC01	Pessoa recém-cadastrada, sem senha	Login recusado; orientação para ativar a conta	Alta
CT04	Limite de tentativas	UC01	Sucessivas senhas erradas para o mesmo e-mail	Tentativas recusadas temporariamente; outro e-mail na mesma rede continua entrando	Alta
CT05	Pessoa com dois perfis	UC01	Login de Ana Paula Souza	Início com conteúdo de síndica; menu com seções de síndica e moradora, sem duplicação	Média
CT06	Primeiro acesso válido	UC14	Código DISPONIVEL e e-mail da pessoa; nova senha	Conta ATIVO; código UTILIZADO; sessão iniciada	Alta
CT07	Código inválido ou usado	UC14	Código já utilizado no CT06	Recusa; nenhuma alteração na conta	Alta
CT08	Recuperação de senha	UC15	E-mail de Carlos; código gerado; nova senha	Senha redefinida; código anterior cancelado	Alta
CT09	Cadastro de pessoa	UC11	Síndica cadastra Bruno Ferreira, apto A-201, INQUILINO	Conta AGUARDANDO_PRIMEIRO_ACESSO e código gerado	Alta
CT10	E-mail duplicado	UC11	Cadastro com carlos.silva@teste.com	Recusa por duplicidade	Alta
CT11	Segundo titular	UC11	Novo PROPRIETARIO para o apto A-302	Recusa: unidade já possui titular	Média
CT12	Administrador único	UC11	Tentativa de inativar Ana, única administradora	Recusa com mensagem explicativa	Alta
CT13	Cadastro de dependente	UC16	Carlos cadastra Lucas Silva, 12 anos, FILHO	Vínculo DEPENDENTE com aprovação PENDENTE	Média

ID	Caso de teste	UC / Regra	Pré-condição e entrada	Resultado esperado	Prior.
CT14	Aprovação de dependente	UC11	Síndica aprova Lucas Silva	Aprovação registrada e código de primeiro acesso gerado	Média
CT15	Responsável inválido	UC16 / RN15	Dependente vinculado a quem não é titular ativo da unidade	Recusa pelo gatilho RN15	Alta
CT16	Cadastro de área	UC09	Quadra, capacidade 20, limite 2 por semana, 2 unidades por horário	Área criada com as regras informadas	Média
CT17	Exclusão sem histórico	UC09	Area criada no CT16, sem reservas nem chaves	Área excluída definitivamente	Média
CT18	Exclusão com histórico	UC09	Tentativa de excluir a Academia	Recusa com a contagem de reservas e oferta de inativação	Alta
CT19	Imagem acima do limite	UC09	Arquivo acima do tamanho máximo ou PDF	Mensagem de tamanho ou formato antes do envio	Média
CT20	Consulta de disponibilidade	UC02	Academia na data D+2 (quarta-feira)	Grade de 06:00 às 22:00 em intervalos de 60 minutos	Alta
CT21	Reserva válida	UC03	Carlos, Academia, D+2, 09:00–10:00, 4 pessoas	Reserva ATIVA	Alta
CT22	Mesma unidade no mesmo horário	UC03 / RN01	Joao, mesma unidade de Carlos, mesmo horário do CT21	Recusa pelo RN01	Alta
CT23	Limite de simultâneas	UC03 / RN01	Maria, Academia, mesmo horário do CT21 (1 unidade por horário)	Recusa: limite de reservas simultâneas atingido	Alta
CT24	Capacidade acumulada	UC03 / RN04	Area com 2 unidades por horário; segunda reserva excedendo a capacidade restante	Recusa com a capacidade restante informada	Alta
CT25	Antecedência e passado	UC03 / RN06	Academia para D (antecedência mínima de 1 dia) e para horário passado	Recusa pelo RN06 nos dois casos	Alta
CT26	Fora do funcionamento	UC03 / RN05	Academia em D+6 (domingo), 15:00–16:00	Recusa pelo RN05	Alta
CT27	Limite semanal	UC03 / RN07	Unidade A-302 com 2 reservas na semana na Academia; terceira tentativa	Recusa pelo RN07	Alta
CT28	Idade mínima	UC03 / RN16	Lucas Silva, 12 anos, reserva a Academia (mínimo 16)	Recusa pelo RN16	Alta
CT29	Área bloqueada	UC12 / RN02	Piscina bloqueada para manutenção em D+5; reserva no período	Recusa pelo RN02	Alta
CT30	Penalidade	UC12 / RN03, RN17	Helena com reserva ativa; síndica aplica penalidade	Reserva cancelada com motivo PENALIDADE; nova reserva recusada	Alta
CT31	Cancelamento no prazo	UC04 / RN08	Reserva do CT21 cancelada em D	Status CANCELADA com data registrada	Alta
CT32	Cancelar concluída	UC04 / RN08	Reserva cujo horário já terminou	Situação CONCLUIDA, sem opção de cancelar	Média
CT33	Empréstimo de chave	UC06 / RN09	CH-SALAO-01 disponível, para Carlos	Status EMPRESTADA	Alta
CT34	Chave indisponível	UC06 / RN09	Novo empréstimo da mesma chave	Recusa, com o responsável atual	Alta
CT35	Devolução	UC06 / RN10	Devolução registrada pelo porteiro	Status DISPONIVEL	Alta
CT36	Chaves da família	UC06	Joao acessa a tela de início após o CT33	Exibe a chave em posse de Carlos	Média
CT37	Registro de encomenda	UC07 / RN11	Porteiro registra pacote MEDIO para Carlos	AGUARDANDO_RETIRADA e aviso na aba de encomendas	Alta
CT38	Retirada sem responsável	UC08 / RN12	Retirada sem informar quem retirou	Recusa pelo RN12	Alta
CT39	Retirada válida	UC08 / RN12	Retirado por PROPRIO	Status RETIRADA com data e responsáveis	Alta
CT40	Encomendas do morador	UC05	Carlos acessa Minhas encomendas	Lista a encomenda do CT37 com a situação atual	Média
CT41	Publicação de aviso	UC10 / RN13	Síndica publica aviso fixado	Um registro de destinatário por perfil ativo	Alta
CT42	Leitura de aviso	UC05 / RN14	Carlos abre o aviso do CT41	Lido com data e hora registradas	Média
CT43	Ocupação agora	UC17	Reserva cobrindo o horário atual	Area listada com morador, unidade e tempo restante	Alta
CT44	Busca de morador	UC17	Porteiro busca "Carlos"	Atividade atual, próximas reservas e histórico de 90 dias	Média
CT45	Área de uso livre	UC18	Porteiro marca área livre como EM_USO	Estado, horário e funcionário registrados	Média
CT46	Painel com filtros	UC13	Síndica filtra o mês corrente	Indicadores e gráficos do período	Média
CT47	Acesso indevido	UC01	Morador acessa /portaria/painel	Acesso negado e redirecionamento	Alta

ID	Caso de teste	UC / Regra	Pré-condição e entrada	Resultado esperado	Prior.
CT48	Cadastro de prestador	UC11	Síndica cadastra Paulo Reparos, PRESTADOR SERVICO, serviço Eletricista	Pessoa registrada sem perfil e sem código de acesso	Média
CT49	Chave para prestador	UC06	Porteiro empresta CH-CHURRAS-01 a Paulo Reparos	Empréstimo registrado na pessoa; chave EMPRESTADA	Média

Fonte: Os autores.

5 CRONOGRAMA

O cronograma apresenta as atividades do projeto distribuídas ao longo do ano letivo de 2026, de março a dezembro, descontado o recesso acadêmico de 20 a 31 de julho. As células preenchidas indicam os meses previstos para cada atividade, e as quatro bancas de avaliação estão assinaladas como atividades. O Quadro 50 apresenta o cronograma completo.

Quadro 50 – Cronograma de atividades do projeto OASIS – 2026

Atividade	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov	Dez
Levantamento de requisitos										
Definição do escopo e do conjunto de tecnologias										
Estimativa de custos e análise de riscos										
Redação e revisão do anteprojeto										
Apresentação do anteprojeto — 1ª banca										
Correções da 1ª banca										
Diagrama de casos de uso										
Modelagem conceitual e lógica do banco de dados										
Diagramas de classes e de sequência										
Scripts SQL, restrições e gatilhos										
Prototipação da interface										
Plano de testes e revisão do documento										
Apresentação do projeto — 2ª banca										
Correções da 2ª banca										
Implementação da API e do banco de dados										
Implementação da interface										
Containerização e infraestrutura										
Autenticação, perfis e dependentes										
Módulos da portaria (encomendas, chaves e painel)										
Atualização do documento e dos diagramas										
Apresentação do sistema — 3ª banca										
Correções da 3ª banca										
Consolidação do banco de dados e separação dos modos de demonstração e produção										
Revisão de segurança e testes de integração										
Ajustes de interface e responsividade										
Execução do plano de testes										
Implantação e homologação com a cliente										
Treinamento da cliente e manual do usuário										
Documentação final — 4ª banca										

Fonte: Os autores. As células preenchidas indicam os meses previstos para cada atividade.

6 CONCLUSÃO

A análise do cenário atual do condomínio evidenciou que a solução vigente não atende às necessidades da administração e dos moradores, em especial quanto à reserva individual de espaços, ao controle de encomendas e chaves, à comunicação oficial e ao acompanhamento do uso das áreas comuns. O OASIS foi concebido como resposta direta a essas demandas.

No terceiro bimestre, o projeto deixou o papel e foi implementado. O sistema atende hoje aos 18 casos de uso descritos neste documento, sobre um banco de dados de 18 tabelas cujas regras críticas — 17 no total — são garantidas por gatilhos, independentemente da camada de aplicação. Durante a implementação, decisões de projeto foram revistas com base na experiência prática: a autenticação passou a ser própria, com ativação de conta por código, o que eliminou a dependência de um serviço externo; o acesso ao banco passou a ser feito com SQL direto, coerente com a opção por regras no próprio banco; e o controle de reservas foi ampliado para contemplar reservas simultâneas, capacidade acumulada, limites por período e idade mínima.

O empacotamento do sistema em contêineres permite reproduzir o ambiente completo com um único comando, e as medidas de segurança adotadas — armazenamento das senhas apenas como resumo criptográfico, limitação de tentativas e controle de acesso por perfil em todas as rotas — preparam o sistema para uso real. As etapas restantes, previstas no cronograma para o quarto bimestre, concentram-se na consolidação do banco de dados, na separação entre os modos de demonstração e de produção, na execução do plano de testes e na implantação.

Conclui-se, portanto, que o desenvolvimento do OASIS é viável e pertinente, com potencial de impacto positivo direto na organização, na segurança e na qualidade de vida da comunidade condominial atendida.

6.1 TRABALHOS FUTUROS

As funcionalidades a seguir estão fora do escopo deste projeto e constituem a próxima etapa de evolução do sistema:

- Integração com a plataforma Intelbras, para que as reservas, os bloqueios e as penalidades já registrados no sistema liberem ou impeçam o acesso físico por biometria e reconhecimento facial;
- Notificações *push* nos dispositivos dos moradores, complementando os avisos do mural;
- Envio do código de recuperação de senha por e-mail;
- Abertura e acompanhamento de chamados pelos moradores;
- Assistente automatizado de atendimento via *WhatsApp* sobre os serviços do condomínio;

- Relatório de uso da própria aplicação, com acessos, telas mais utilizadas e horários de pico.

7 REFERÊNCIAS

DOCKER INC. Docker documentation. 2026. Disponível em: <https://docs.docker.com>. Acesso em: set. 2026.

FOWLER, M. UML distilled: a brief guide to the standard object modeling language. 3. ed. Boston: Addison-Wesley, 2004.

GLASSDOOR. Salários de Desenvolvedor Pleno no Brasil. 2026. Disponível em: https://www.glassdoor.com.br/Salarios/desenvolvedor-pleno-salario-SRCH_KO0,19.htm. Acesso em: mar. 2026.

HEUSER, C. A. Projeto de banco de dados. 6. ed. Porto Alegre: Bookman, 2009.

NESTJS. NestJS: a progressive Node.js framework. 2026. Disponível em: <https://docs.nestjs.com>. Acesso em: set. 2026.

NODE-POSTGRES. node-postgres documentation. 2026. Disponível em: <https://node-postgres.com>. Acesso em: set. 2026.

NODE.JS. Node.js documentation. 2026. Disponível em: <https://nodejs.org/en/docs>. Acesso em: set. 2026.

OWASP FOUNDATION. Password storage cheat sheet. 2026. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: set. 2026.

POSTGRES SQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 documentation. 2026. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: set. 2026.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

REACT. React: the library for web and native user interfaces. 2026. Disponível em: <https://react.dev>. Acesso em: set. 2026.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.

8 RESPONSABILIDADES